UNIT 1

GENERAL ISSUES AND OVERVIEW OF AI

Books

- **Text Books**

1. Elaine Rich and Kevin Knight: Artificial Intelligence- Tata McGraw Hill.
2. Dan W. Patterson, Introduction to Artificial Intelligence and Expert Systems- Prentice Hall of India.

- **Reference Books**

1. Nils J. Nilsson: Principles of Artificial Intelligence- Narosa Publishing house.
2. Artificial Intelligence : A Modern Approach, Stuart Rusell, Peter Norvig, Pearson Education, 2nd Edition
3. Artificial Intelligence, Winston, Patrick, Henry, Pearson Education
4. Artificial Intelligence and Expert System, Gopal Krishna , Janakiraman

Syllabus

- Introduction to AI,
- Problem Solving, State space search,
- Blind search:
  - Depth first search,
  - Breadth first search,
- Informed search:
  - Heuristic function,
  - Hill climbing search,
  - Best first search,
  - A* & AO* Search,
  - Constraint satisfaction.
- Game tree
- Evaluation function,
- Mini-Max search,
- Alpha-beta pruning,
- Games of chance.

Introduction to AI

What is AI ?

- Artificial Intelligence is the science and engineering of making intelligent machines.
- Artificial Intelligence is the study of how to make computers do things which, at the moment, people do better.
- Artificial Intelligence is the branch of computer science that is concerned with the automation of intelligent behavior.
- Artificial Intelligence is the study and design of intelligent agents, where an intelligent agent is a system that perceives its environment and takes actions that maximize its chance of success.

What is AI ?

- Artificial Intelligence is concerned with the design of intelligence in an artificial device.

- The term was coined by McCarthy in 1956.

- There are two ideas in the definition.
  - Intelligence
  - Artificial

- The term artificial is easy to understand. But it's very difficult to define intelligence.

What is Intelligence?

- Intelligence is what we use when we don't know what to do.
- Intelligence relates to tasks involving higher mental processes.

Examples: Creativity, Solving problems, Pattern recognition, Classification, Learning, Induction, Deduction, Building analogies, Optimization, Language processing, Knowledge and many more.

Approaches to AI

- Hard or Strong AI
- Soft or Weak AI
- Applied AI
- Cognitive AI

Hard or Strong AI

- Strong AI refers to a machine that approaches or supersedes human intelligence.
  - if it can do typically human tasks,
  - If it can apply a wide range of background knowledge and
  - If it has some degree of self-consciousness
- Strong AI aims to build machines whose overall intellectual ability is indistinguishable from that of a human being.

Soft or Weak AI

- Weak AI refers to the use of software to study or accomplish specific problem solving or reasoning tasks that do not encompass the full range of human cognitive abilities. Example: a chess program such as Deep Blue

- Weak AI does not achieve self-awareness; it demonstrates wide range of human level cognitive abilities; it is merely an intelligent, a specific problem-solver.

Applied AI

- Aims to produce commercially viable "smart" systems such as, for example, a security system that is able to recognize the faces of people who are permitted to enter a particular building.
- Applied AI has already enjoyed considerable success.

Cognitive AI

- Computers are used to test theories about how the human mind works--for example, theories about how we recognize faces and other objects, or about how we solve abstract problems.

Cognitive science

- Aims to develop, explore and evaluate theories of how the mind works through the use of computational models.

- The important is not what is done but how it is done; means intelligent behavior is not enough, the program must operate in an intelligent manner.

- Example: the chess programs are successful, but say little about the ways humans play chess.

Cybernetics

- Cybernetics" comes from a Greek word meaning "the art of steering".
- Cybernetics is about having and taking action to achieve that goal. Knowing whether you have reached your goal (or at least are getting closer to it) requires "", a concept that comes from cybernetics.
- Cybernetics grew from a desire to understand and build systems that can achieve goals, whether complex human goals or just goals like maintaining the temperature of a room under changing conditions.

Goals of AI

- The definition of AI gives four possible goals to pursue:
  1. Systems that think like humans
  2. Systems that think rationally
  3. Systems that act like humans
  4. Systems that act rationally
- Traditionally, all four goals have been followed and the approaches were:

	Human-like	Rationally
Think	(1) Cognitive science Approach	(2) Laws of thought Approach
Act	(3) Turing Test Approach	(4) Rational agent Approach

- Most of AI works falls into category 2 and 4.

Systems that think like humans

- Most of the time it is a black box where we are not clear about our thought process.
- One has to know functioning of brain and its mechanism for possessing information.
- It is an area of cognitive science.
  - The stimuli are converted into mental representation.
  - Cognitive processes manipulate representation to build new representations that are used to generate actions.
- Neural network is a computing model for processing information similar to brain.

Systems that act like humans

- The overall behaviour of the system should be human like.

- It could be achieved by observation.

Systems that think rationally

- Such systems rely on logic rather than human to measure correctness.
- For thinking rationally or logically, logic formulas and theories are used for synthesizing outcomes.
- For example,
  - given John is a human and all humans are mortal then one can conclude logically that John is mortal
- Not all intelligent behavior are mediated by logical deliberation.

Systems that act rationally

- Rational behavior means doing right thing.
- Goal is to develop systems that are rational and sufficient.

General AI Goals

- Replicate human intelligence
- Solve knowledge intensive tasks
- Make an intelligent connection between perception and action
- Enhance human-human, human-computer and computer to computer interaction/communication

AI Goal

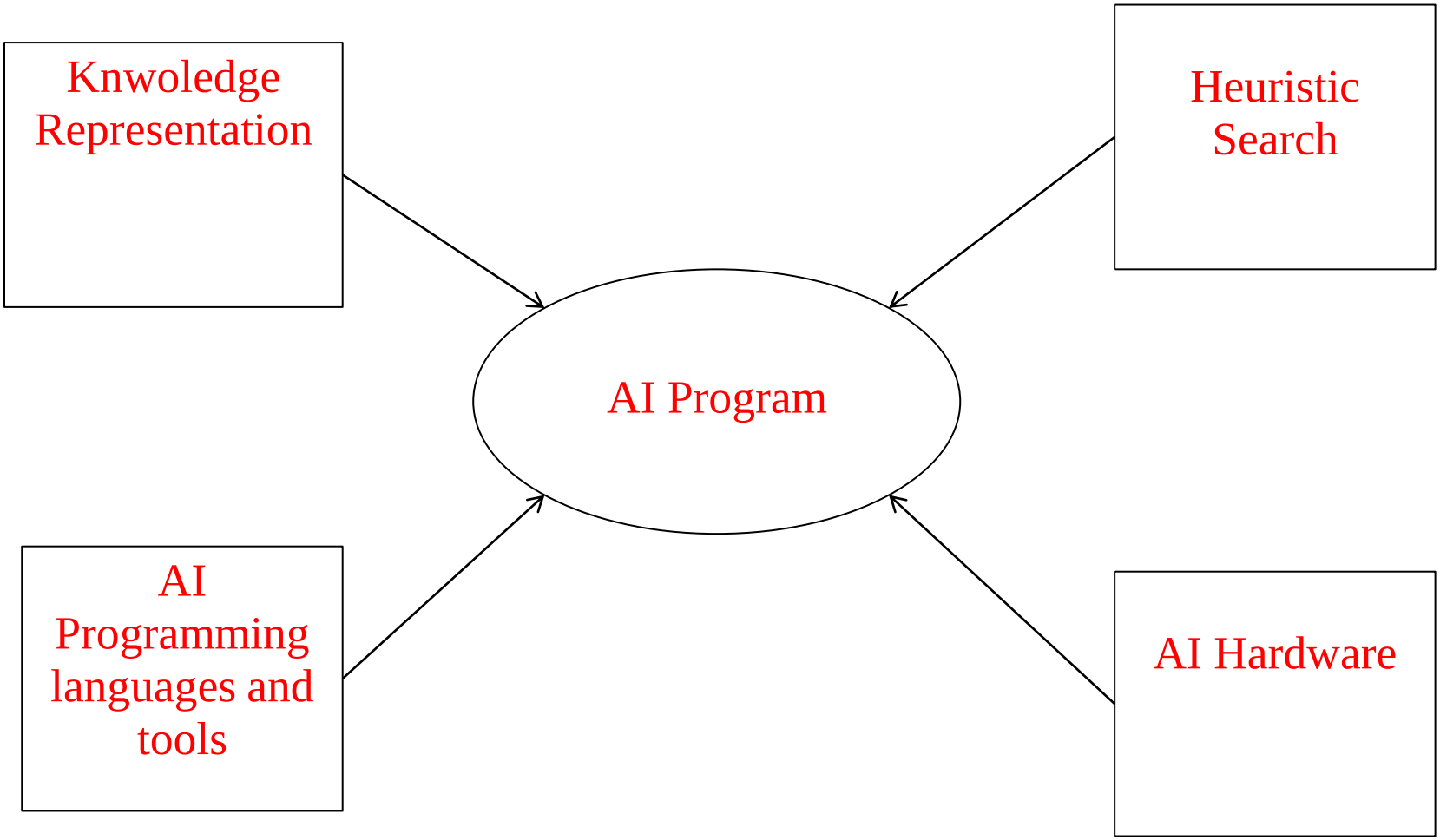
Engineering based AI Goal

- Develop concepts, theory and practice of building intelligent machines
- Emphasis is on system building

Science based AI Goal

- Develop concepts, mechanisms and vocabulary to understand biological intelligent behavior.
- Emphasis is on understanding intelligent behavior.

Major components of an AI system



Major components of an AI system

- The quality of the result depends on how much knowledge the system possesses. The available knowledge must be represented in a very efficient way. Hence, knowledge representation is vital component of the system.
- It is not merely enough that knowledge is represented efficiently. The inference process should also be equally good for satisfactory results. The inference process is broadly divided into brute and heuristic search procedure.
- Today, just like we have specialized languages and programs for data processing and scientific applications, we encounter specialized languages and tools for AI programming. AI languages provide the basic functions for AI programming and tools for the right environment.
- Today, most of the AI programs in India are implemented on Von Neumann machines only. However dedicated workshops have emerged for AI programming.

Applications area of AI

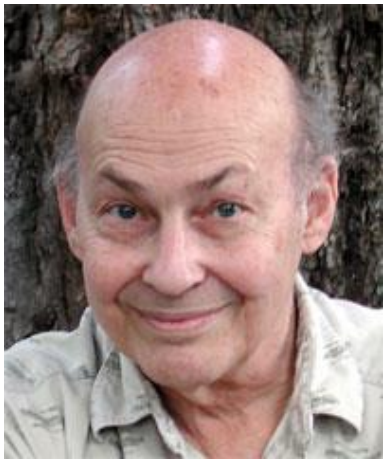
- Perception
  - Machine vision
  - Speech understanding
  - Touch (*tactile* or *haptic*) sensation
- Robotics
- Natural Language Processing
  - Natural Language Understanding
  - Speech Understanding
  - Language Generation
  - Machine Translation
- Planning
- Expert Systems
- Machine Learning
- Theorem Proving
- Symbolic Mathematics
- Game Playing

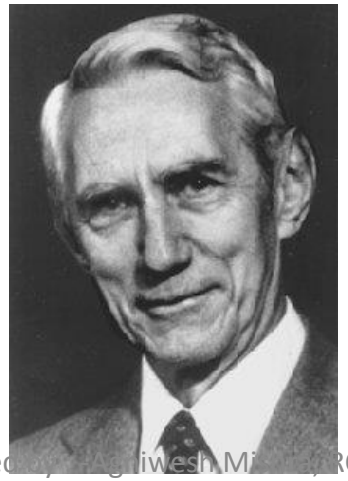
The History of AI

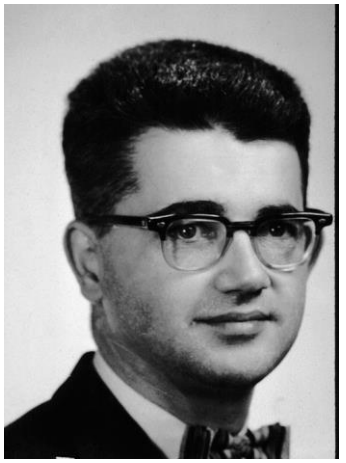
i. Gestation of AI (1943-1956)
ii. Early enthusiasm, great expectations (1952-1969)
iii. A dose of reality (1966-1974)
iv. Knowledge-based systems (1969-1979)
v. AI becomes an industry (1980-1988)
vi. Evolving systems (1986-present)
vii. Recent events (1987-present)

Gestation of AI

- First work
  - McCulloch and Pitts (1943): build a Boolean circuit model of brain
- First neural network computer
  - Minsky and Edmonds (1951) build "SNARC" neural network computer
- First chess programs
  - Shannon (1950) and Turing (1953)
- Official birthplace of AI
  - The 1956 Dartmouth workshop organized by John McCarthy

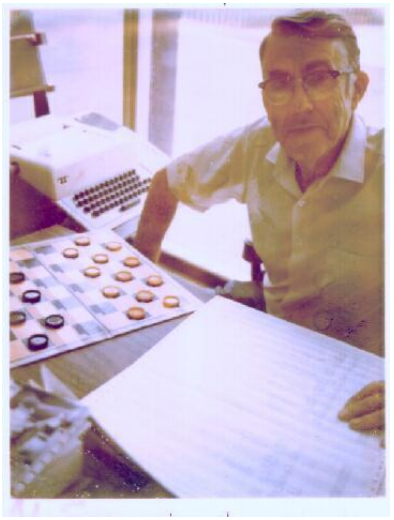






Early enthusiasm, great expectations

- Newell and Simon's early work
  - Logic Theorist(1956)
  - General Problem Solver
- Samuel's checkers program(1952-1956)
- The MIT connection
  - McCarthy's LISP (1958)
  - Minsky's microworlds (1963)



A dose of reality

- Things were not so easy…
  - AI discovers computational complexity(1966-1974)
  - Only syntactic manipulation and little knowledge
  - A lot of AI problems are intractable
  - Basic structures had fundamental limitations

Can AI only be used for toy problems?

Knowledge-based systems(1969-79)

- Expert systems
  - Use knowledge to suit larger reasoning steps
  - Inferring molecular structures: DENDRAL (1969)
  - Medical diagnosis: MYCIN(1974)
  - Geological system: PROSPECTOR
- Knowledge representation schemes
  - Production System (Newell)
  - Frame theory (Minsky)
  - Conceptual Dependency (Schank)
  - PROLOG (Colmerauer)

AI becomes an industry

- The first commercial expert system
  - DEC's R1 (1982)
- The "Fifth Generation" project
  - A Japanese initiative to take the lead in AI research
  - Parallel reasoning machine
  - Running Prolog like machine code

Evolving systems(1990)

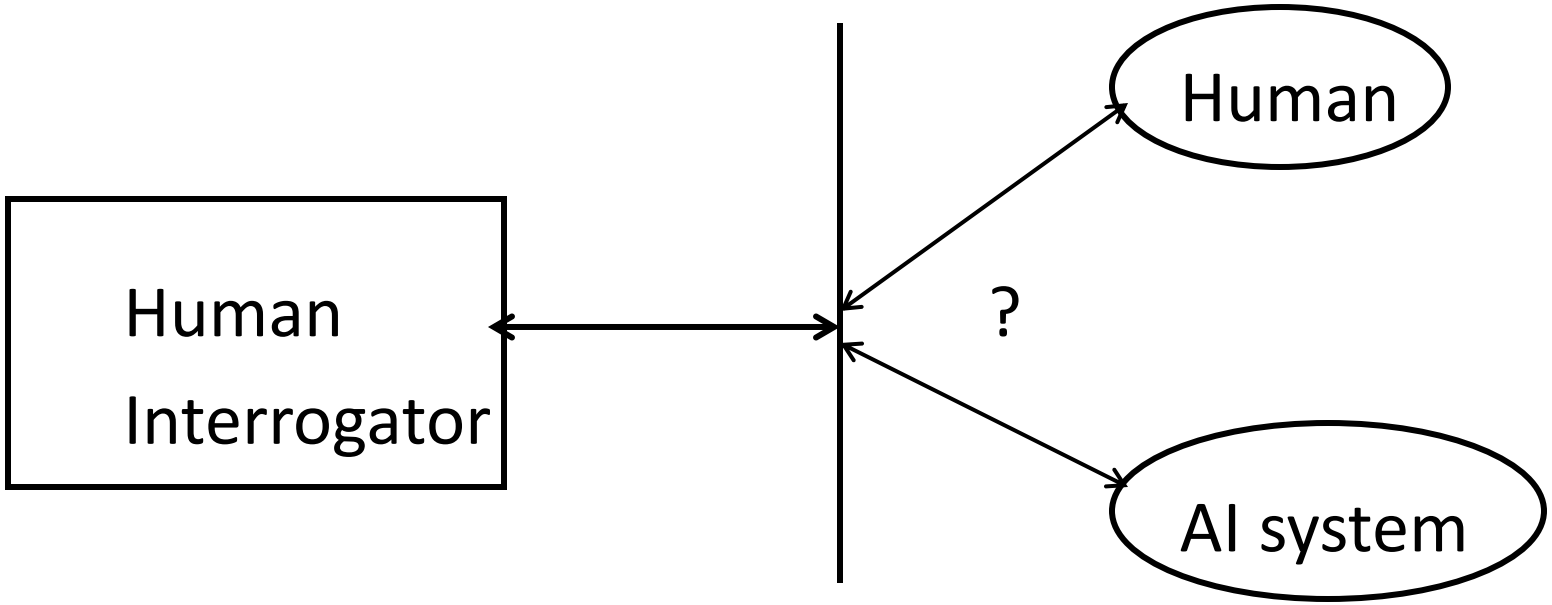
- Machine learning
  - An attempt to solve the knowledge acquisition bottleneck
  - Reinvention of the back propagation learning algorithm
- Expert system design is very time-consuming
  - Can machine learning fix that problem?
- Intelligent tutoring
- Case based reasoning
- Multi agent planning
- Scheduling
- Natural language
- Virtual reality, games
- Genetic algorithms
  - Evolutionary programming

Recent events

- Based on existing theories instead of proposing new ones
  - *Speech recognition* based on hidden Markov models
  - *Planning* based from the start on a simple framework
  - *Belief networks* for reasoning about the combination of uncertain evidence
- This has lead to robust methods and workable research agendas
- Is the “whole agent” problem within reach?

The Turing Test

Turing proposed operational test for intelligent behavior in 1950.



Alan M. Turing, (1912 - 1954)

- Major contributor to the code-breaking work at Bletchley Park (*Enigma*), during World War II.
- Major contributor to the early development of computers.
- Foresaw *Artificial Intelligence* & devised the *Turing Test*.

Turing test

- To conduct Turing test, we need two people and the machine to be evaluated. One person plays the role of the interrogator, who is in a separate room from the computer and the other person.
- The interrogator can ask questions of either the person or the computer by typing questions and receiving typed responses. however, the integrator knows only as A and B and aims to determine which is the person and which is the machine.
- The goal of the machine is to fool the interrogator into believing that it is the person. If the machine succeeds at this, then we will conclude that the machine can think. The machine is allowed to do whatever it can to fool the interrogator.

Information, Knowledge, Intelligence

- **Information** is a message that contains relevant meaning, implication, or input for decision and/or action. Information comes from both current (communication) and historical (processed data or 'reconstructed picture') sources. In essence, the purpose of information is to aid in making decisions and/or solving problems or realizing an opportunity.
- **Knowledge** is the cognition or recognition (know-what), capacity to act (know-how), and understanding (know-why) that resides or is contained within the mind or in the brain. The purpose of knowledge is to better our lives.
- **Intelligence** (also called intellect) is an umbrella term used to describe a property of the mind that encompasses many related abilities, such as the capacities to reason, to plan, to solve problems, to think abstractly, to comprehend ideas, to use language, and to learn.

Human intelligence Vs. Artificial intelligence

Human intelligence	Artificial intelligence
Human intelligence revolves around adapting to the environment using a combination of several cognitive processes.	The field of Artificial intelligence focuses on designing machines that can mimic human behavior.
Human Intelligence is organically based,	Artificial Intelligence is silicon based.
Human intelligence is separated through the five senses.	Computer intelligence is separated into binary code
Human memory...Five access points	Artificial intelligence...Just one access point....
The human mind on the other hand works by asociation, which is not always logical in a technical sense.	Artificially Intelligence works along pre-set formulas and ways, it is rather straight forward.

Intelligent Computing Vs. conventional computing

	Intelligent Computing		Conventional Computing
1	Does not guarantee a solution to a given problem.	1	Guarantees a solution to a given problem.
2	Results may not be reliable and consistent	2	Results are consistent and reliable.
3	Programmer does not tell the system how to solve the given problem.	3	Programmer tells the system exactly how to solve the problem
4	Can solve a range of problems in a given domain.	4	Can solve only one problem at a time in a given domain

The AI Problem

The AI Problem

- Much of the early work in the field focused on **formal tasks**, such as game playing and theorem proving.
- Another early foray into AI focused on the sort of problem solving that we do every day called as **mundane tasks**, such as commonsense reasoning.
- The problem area where AI is now flourishing most as a practical discipline are primarily the domains that require only specialized expertise without the assistance of commonsense knowledge, called as **expert systems**.

Formal Tasks

- Games
  - Chess
  - Backgammon
  - Checkers-Go
- Mathematics
  - Geometry
  - Logic
  - Internal calculus
  - Proving properties of programs

Mundane Tasks

- Perception
  - Vision
  - Speech
- Natural language
  - Understanding
  - Generation
  - Translation
- Commonsense reasoning
- Robot control

Expert Tasks

- Engineering
  - Design
  - Fault finding
  - Manufacturing planning
- Scientific analysis
- Medical diagnosis
- Financial analysis

Limits of AI Today

Today's AI systems have been able to achieve limited success in some of these tasks.

- In Computer vision, the systems are capable of face recognition
- In Robotics, we have been able to make vehicles that are mostly autonomous.
- In Natural language processing, we have systems that are capable of simple machine translation.
- Today's Expert systems can carry out medical diagnosis in a narrow domain
- Speech understanding systems are capable of recognizing several thousand words continuous speech
- Planning and scheduling systems had been employed in scheduling experiments with the Hubble Telescope.
- The Learning systems are capable of doing text categorization into about a 1000 topics
- In Games, AI systems can play at the Grand Master level in chess (world champion), checkers, etc.

What can AI systems NOT do yet?

- Understand natural language robustly (e.g., read and understand articles in a newspaper)
- Surf the web
- Interpret an arbitrary visual scene
- Learn a natural language
- Construct plans in dynamic real-time domains
- Exhibit true autonomy and intelligence

What is an AI technique

What is an AI technique

AI technique is a method that exploits knowledge which should be represented in such a way that –

i. The knowledge captures generalization or we can say that situations that share important properties and grouped together rather than to represent separately each individual situation.

ii. It can be understood by people who provide it. In many AI domains most of the knowledge, a programs has, must ultimately be provided by people in terms they understand.

iii. It can easily be modified to correct errors and to reflect changes in the world and in our world view.

iv. It can be used in a great many situations even if it is not totally accurate or complete.

v. It can be used to help overcome its own sheer bulk by to narrow the range of possibilities that must usually be considered.

Examples of AI problems

1. Tic-Tac toe
2. Water jug problem
3. 8-puzzle problem
4. 8-queen problem
5. Chess problem
6. Missionaries and cannibals problem
7. Tower of Hanoi problem
8. Traveling salesman problem
9. Magic square
10. Language understanding problems
11. Monkey and Banana Problem
12. Crypt arithmatic puzzle
13. Block World problem

Problem solving /Problem representation

Problem solving

- Problem solving is a process of generating solutions from observed data.
- Key element of problem solving
  - State: A state is a representation of problem at a given moment.
  - State space: Contains all the possible states for a given problem.
  - Operators: the available actions performed is called operators.
  - Initial state: position from which the problem-solving process may start.
  - Goal state: solution to the problem.

General Problem solving

- To build a system, to solve a praticular problem, there are four things:
  1. Define the problem precisely (apply the State Space representation).
  2. Analyze the problem.
  3. Isolate and represent the task knowledge that is necessary to solve the problem.
  4. Choose the best problem solving technique(s) and apply it to the particular problem.

Problem Characteristics

To choose an appropriate method for a particular problem:

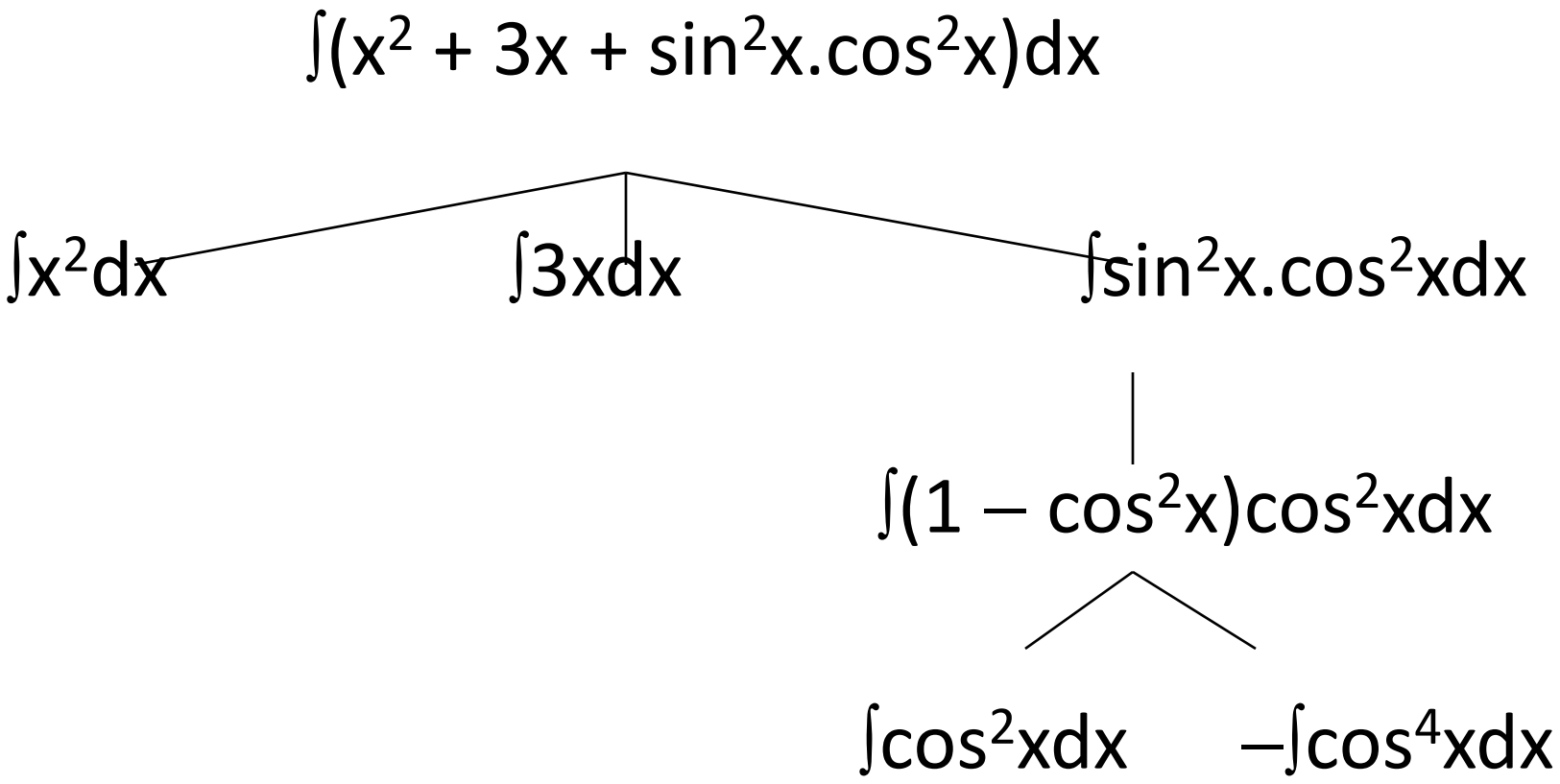
1. Is the problem decomposable?
2. Can solution steps be ignored or undone?
3. Is the universe predictable?
4. Is a good solution absolute or relative?
5. Is the solution a state or a path?
6. What is the role of knowledge?
7. Does the task require human-interaction?

1. Is the problem decomposable?

- Can the problem be broken down to smaller problems to be solved independently?

- Decomposable problem can be solved easily.

1. Is the problem decomposable?



2. Can solution steps be ignored or undone?

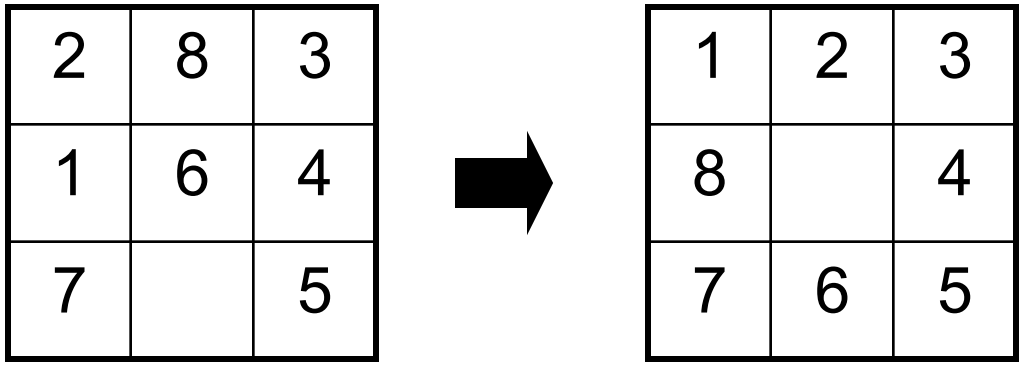
Theorem Proving

A lemma that has been proved can be ignored for next steps.

Ignorable!

2. Can solution steps be ignored or undone?

The 8-Puzzle



Moves can be undone and backtracked.

Recoverable!

2. Can solution steps be ignored or undone?

Playing Chess

Moves cannot be retracted.

Irrecoverable!

2. Can solution steps be ignored or undone?

- Ignorable problems can be solved using a simple control structure that never backtracks.
- Recoverable problems can be solved using backtracking.
- Irrecoverable problems can be solved by recoverable style methods via planning.

3. Is the universe predictable?

The 8-Puzzle

Every time we make a move, we know exactly what will happen.

Certain outcome!

3. Is the universe predictable?

Playing Bridge

We cannot know exactly where all the cards are or what the other players will do on their turns.

Uncertain outcome!

3. Is the universe predictable?

- For certain-outcome problems, planning can used to generate a sequence of operators that is guaranteed to lead to a solution.
- For uncertain-outcome problems, a sequence of generated operators can only have a good probability of leading to a solution.

  Plan revision is made as the plan is carried out and the necessary feedback is provided.

4. Is a good solution absolute or relative?

1. Marcus was a man.
2. Marcus was a Pompeian.
3. Marcus was born in 40 A.D.
4. All men are mortal.
5. All Pompeians died when the volcano erupted in 79 A.D.
6. No mortal lives longer than 150 years.
7. It is now 2004 A.D.

4. Is a good solution absolute or relative?

1. Marcus was a man.
2. Marcus was a Pompeian.
3. Marcus was born in 40 A.D.
4. All men are mortal.
5. All Pompeians died when the volcano erupted in 79 A.D.
6. No mortal lives longer than 150 years.
7. It is now 2004 A.D.

Is Marcus alive?

4. Is a good solution absolute or relative?

1. Marcus was a man.
2. Marcus was a Pompeian.
3. Marcus was born in 40 A.D.
4. All men are mortal.
5. All Pompeians died when the volcano erupted in 79 A.D.
6. No mortal lives longer than 150 years.
7. It is now 2004 A.D.

Is Marcus alive?

Different reasoning paths lead to the answer. It does not matter which path we follow.

4. Is a good solution absolute or relative?

The Travelling Salesman Problem

We have to try all paths to find the shortest one.

4. Is a good solution absolute or relative?

- Any-path problems can be solved using heuristics that suggest good paths to explore.

- For best-path problems, much more exhaustive search will be performed.

5. Is the solution a state or a path?

Finding a consistent intepretation

"The bank president ate a dish of pasta salad with the fork".

- "bank" refers to a financial situation or to a side of a river?
- "dish" or "pasta salad" was eaten?
- Does "pasta salad" contain pasta, as "dog food" does not contain "dog"?
- Which part of the sentence does "with the fork" modify? What if "with vegetables" is there?

No record of the processing is necessary.

5. Is the solution a state or a path?

The Water Jug Problem

The path that leads to the goal must be reported.

5. Is the solution a state or a path?

- A path-solution problem can be reformulated as a state-solution problem by describing a state as a partial path to a solution.

- The question is whether that is natural or not.

6. What is the role of knowledge

Playing Chess

Knowledge is important only to constrain the search for a solution.

Reading Newspaper

Knowledge is required even to be able to recognize a solution.

7. Does the task require human-interaction?

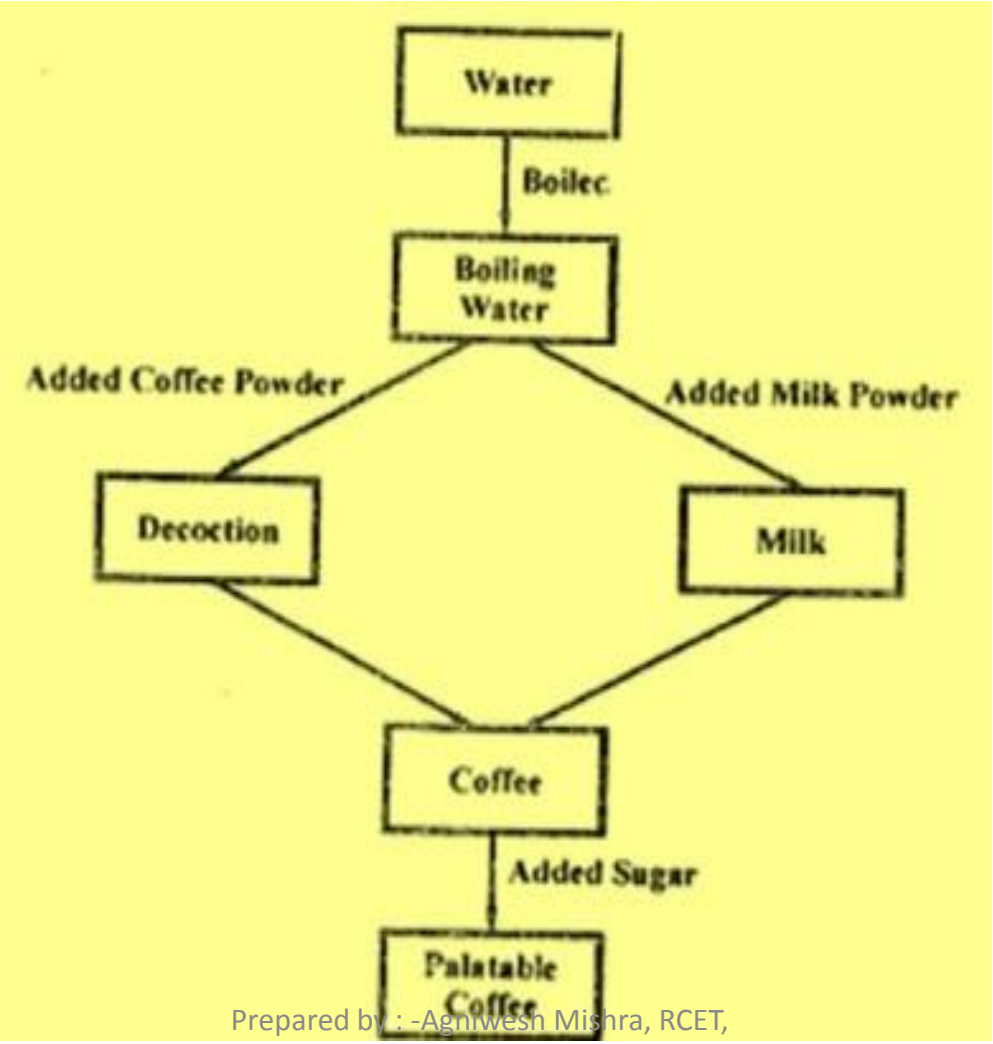
- Solitary problem, in which there is no intermediate communication and no demand for an explanation of the reasoning process.

- Conversational problem, in which intermediate communication is to provide either additional assistance to the computer or additional information to the user.

State space representation

- Before a solution can be found, the prime condition is that the problem must be very precisely defined. By defining it properly, one converts the abstract problem into real workable states that are really understood.
- A set of all possible states for a given problem is known as the **state space** of the problem.State space representations are highly beneficial in AI because they provide all possible states, operations and goals.
- If the entire state space representations for a problem is given, it is possible to trace the path from the initial state to the goal state and identify the sequence of operators necessary for doing it.
- The major deficiency of this method is that it is not possible to visualize all states for a given problem. Moreover, the resources of the computer system are limited to handle huge state-space representation.

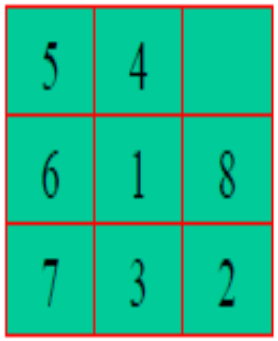
State space representation of coffee making



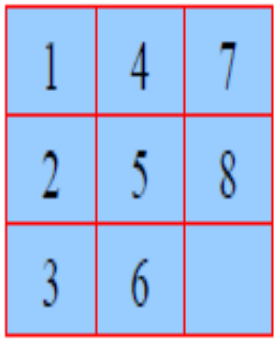
8- puzzle problem

- In the 8-puzzle problem we have a 3×3 square board and 8 numbered tiles. The board has one blank position.
- Tiles can be slid to adjacent blank positions. We can alternatively and equivalently look upon this as the movement of the blank position up, down, left or right.
- The objective of this puzzle is to move the tiles starting from an initial position and arrive at a given goal configuration.

Initial and goal state



Initial State



Goal State

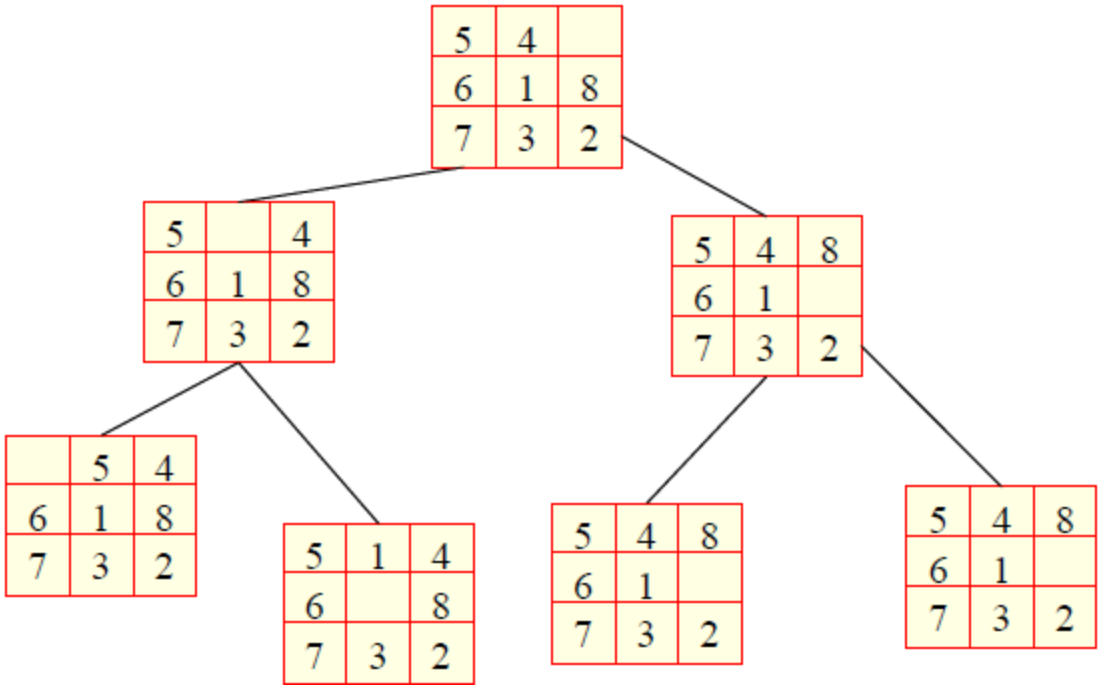
The start state is some (almost) random configuration of the tiles

The goal state is as shown

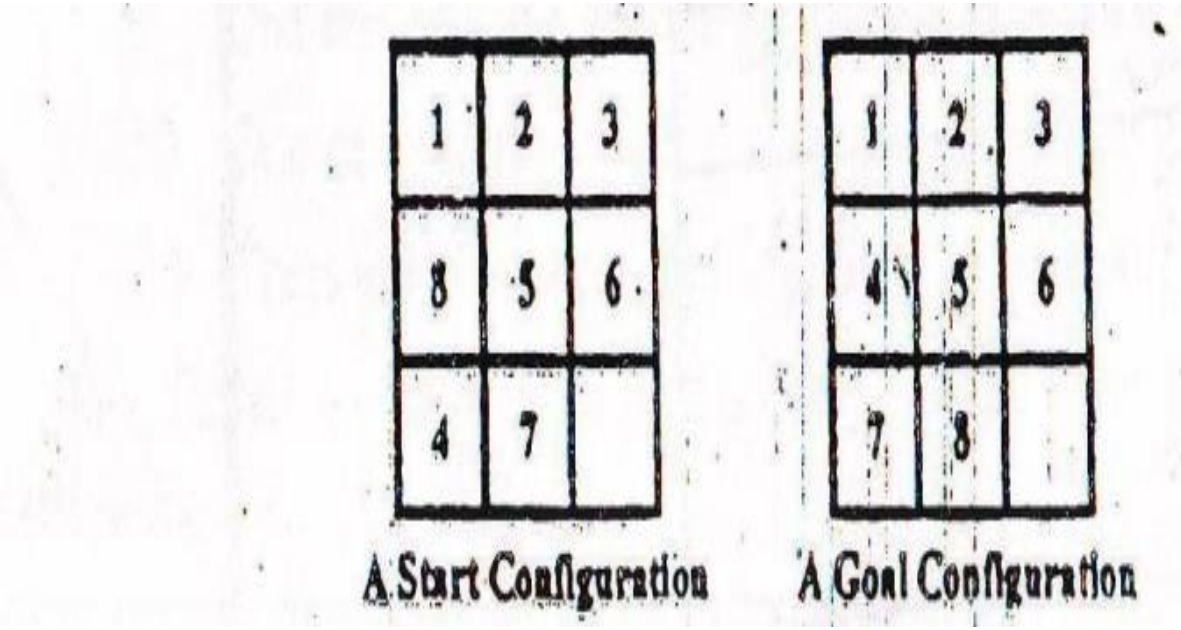
Operators are

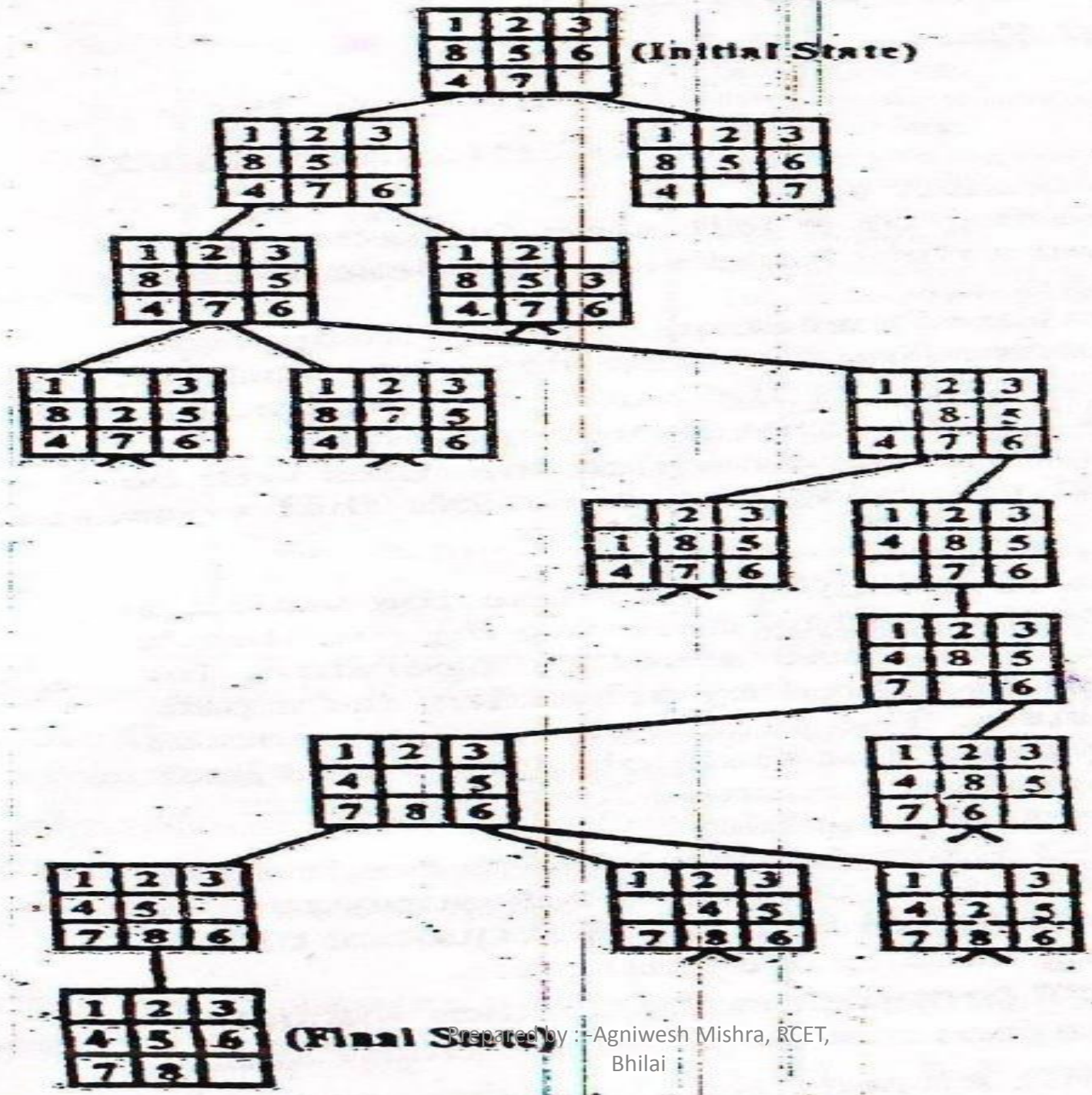
- Move empty space up
- Move empty space down
- Move empty space right
- Move empty space left

State space of the 8 puzzle problem



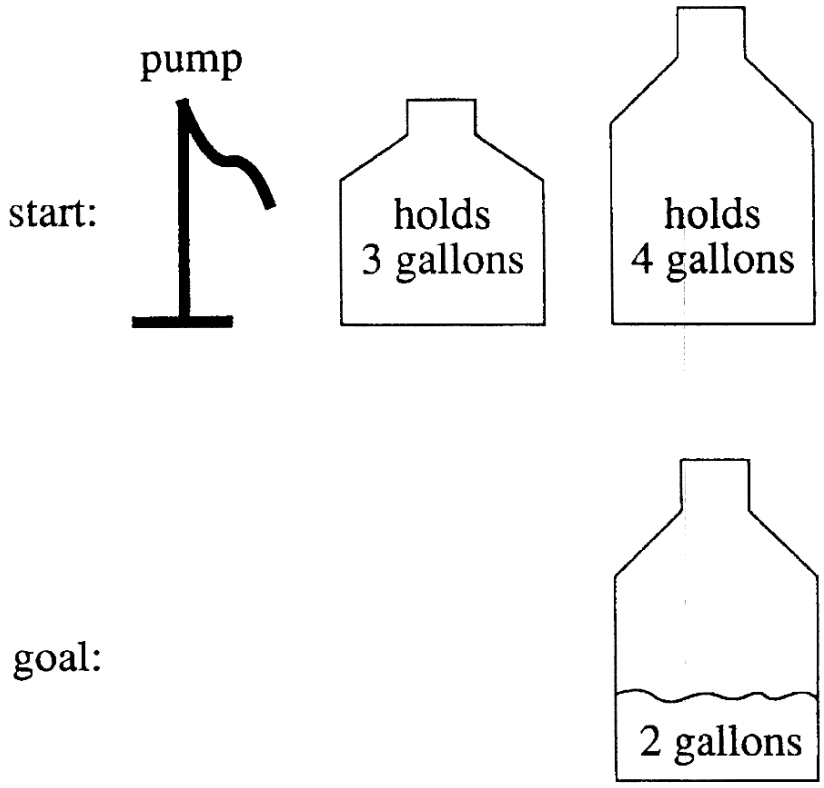
8- puzzle problem




(Initial State)
(Final State)

A Water Jug Problem

- You have a 4-gallon and a 3-gallon water jug
- You have a pump with an unlimited amount of water
- You need to get exactly 2 gallons in 4-gallon jug



Puzzle-solving as Search

- State representation**: (x, y)**
  - x: Contents of four gallon
  - y: Contents of three gallon

- Start state**: (0, 0)**
- Goal state **(2, n)**
- Operators
  - Fill 3-gallon from pump, fill 4-gallon from pump
  - Fill 3-gallon from 4-gallon , fill 4-gallon from 3-gallon
  - Empty 3-gallon into 4-gallon, empty 4-gallon into 3-gallon
  - Dump 3-gallon down drain, dump 4-gallon down drain

State Space Search: Water Jug Problem...

1. (x, y) → (4, y) if x is less than 4, fill the 4 gallon jug
   if $x < 4$
2. (x, y) → (x, 3) fill the 3-gallon jug
   if $y < 3$
3. (x, y) → (x – d, y) pour some water out of the 4 gallon jug
   if $x > 0$
4. (x, y) → (x, y – d) pour some water out of the 3 gallon jug
   if $y > 0$
5. (x, y) → (0, y) empty the 4-gallon jug on the ground
   if $x > 0$
6. (x, y) → (x, 0) empty the 3-gallon jug on the ground
   if $y > 0$

Water Jug Problem...

7.	$(x, y) \rightarrow (4, y - (4 - x))$ if $x + y \geq 4, y > 0$	pour water from the 3- gallon jug into the 4-gallon jug until the 4-gallon jug is full
8.	$(x, y) \rightarrow (x - (3 - y), 3)$ if $x + y \geq 3, x > 0$	pour water from the 4- gallon jug into the 3-gallon jug until the 3-gallon jug is full
9.	$(x, y) \rightarrow (x + y, 0)$ if $x + y \leq 4, y > 0$	pour all the water from the 3-gallon jug into the 4-gallon jug
10.	$(x, y) \rightarrow (0, x + y)$ if $x + y \leq 3, x > 0$	pour all the water from the 4-gallon jug into the 3-gallon jug
11.	$(0, 2) \rightarrow (2, 0)$	pour the 2 gallons from the 3-gallon jug into the 4-gallon jug
12.	$(2, y) \rightarrow (0, y)$	empty the 2 gallons in the 4 gallon jug on the ground

State Space Search: Water Jug Problem...

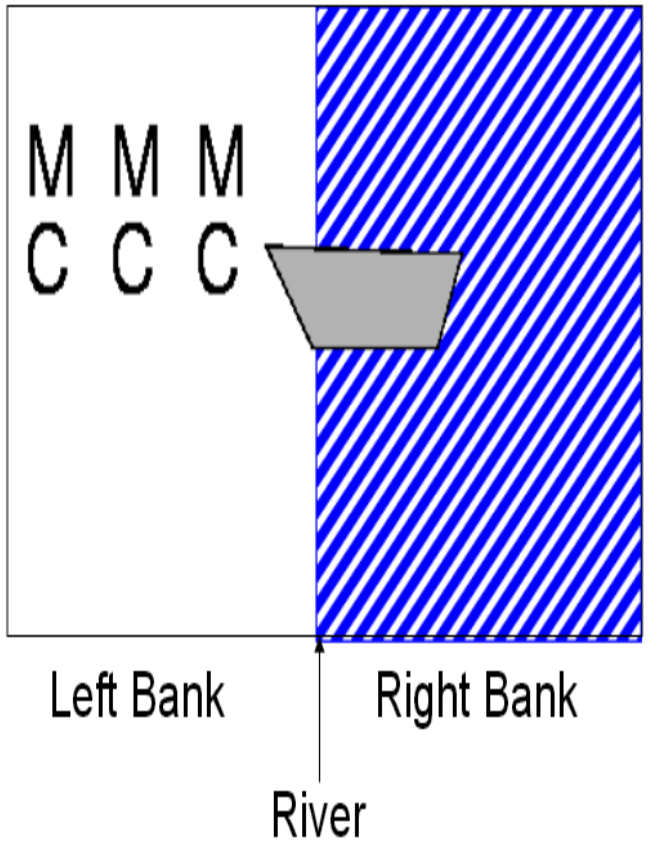
1. Current state = (0, 0)
2. Loop until reaching the goal state (2, 0)

   - Apply a rule whose left side matches the current state
   - Set the new current state to be the resulting state

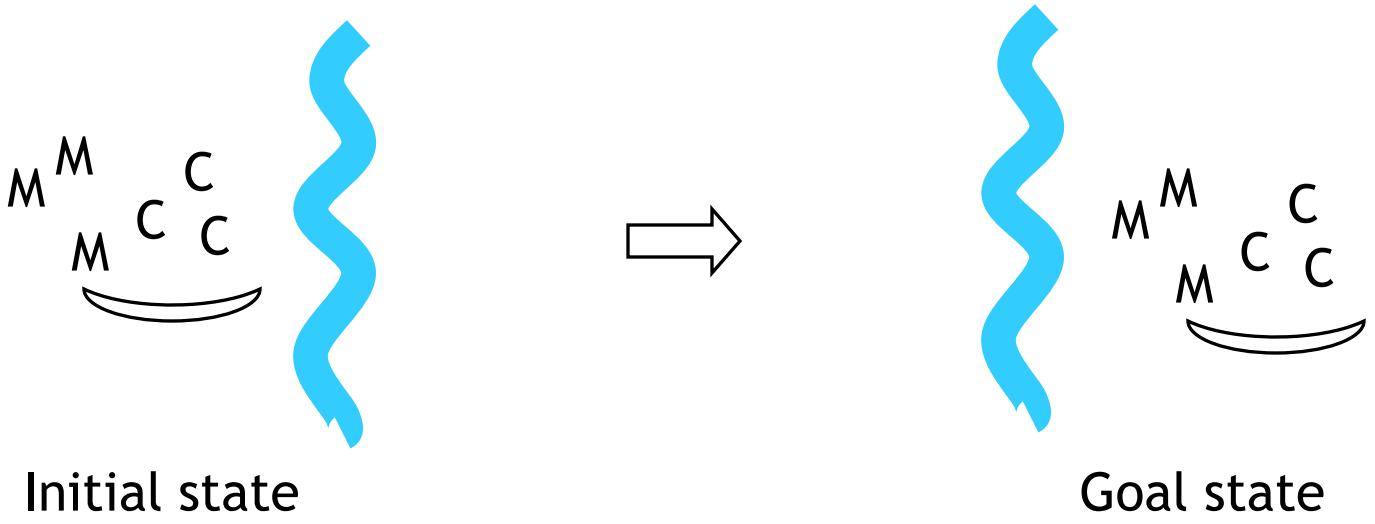
Gallons in the 4-Gallon Jug	Gallons in the 3-Gallon Jug	Rule Applied
0	0	2
0	3	9
3	0	2
3	3	7
4	2	5
0	2	9
2	0	

Missionaries and cannibals

The missionaries and cannibals wish to cross a river
They have a boat that can hold two people
It is unsafe to have cannibals outnumber missionaries



Missionaries and cannibals



Initial state

Goal state

States

- A state can be represented by the number of missionaries and cannibals on each side of the river
- State(#m, #c, riverbank)

#m → no. of missionaries

#c → no. of cannibals

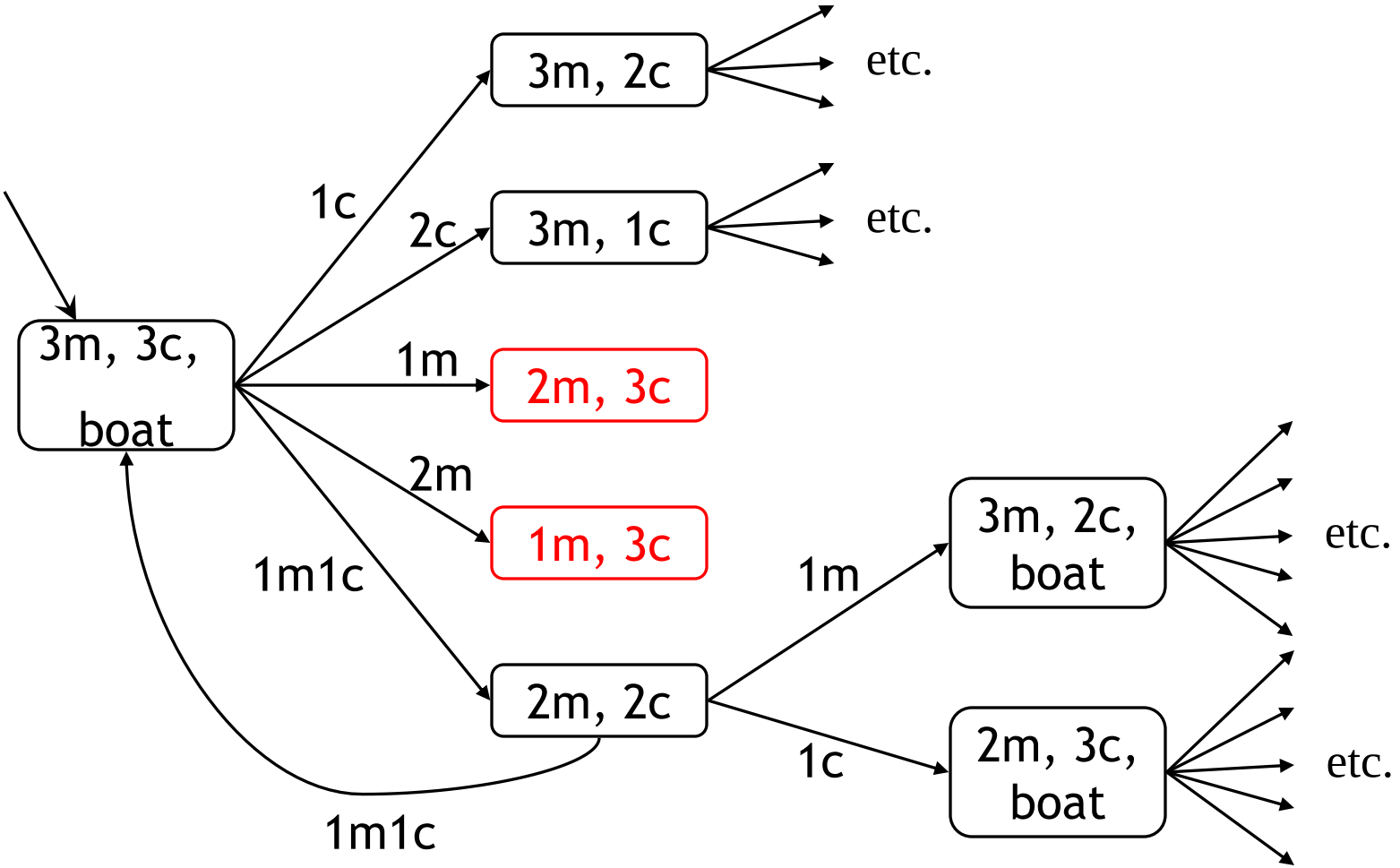
Bankofriver→ indicates whether the boat in the left bank or right bank 0→ left bank, 1→right bank

- Initial state (3,3,0)
- Goal state: (0,0,0)

Operations

- An operation takes us from one state to another
- Here are five possible operations:
  - boat takes 1 missionary across river (1m)
  - boat takes 1 cannibal across river (1c)
  - boat takes 2 missionaries across river (2m)
  - boat takes 2 cannibals across river (2c)
  - boat takes 1 missionary and 1 cannibal across river (1m1c)

The state space



Solution

1. (3, 3,0) → (0,0,1) Initial state
2. (3, 1,0) → (0,2,1) two cannibal will go
3. (3,2,0) → (0,1,1) one cannibal will return
4. (3,0,0) →(0,3,1) two cannibal will go
5. (3,1,0) →(0,2,1) one cannibal will return
6. (1,1,0) →(2,2,1) two Missionaries will go
7. (2,2,0) →(1,1,1) one cannibal and one Missionaries will return
8. (0,2,0) →(3,1,1) two Missionaries will go
9. (0,3,0) → (3,0,1) one cannibal will return
10. (0,1,0) →(3,2,1) two cannibal will go
11. (0,2,0) → (3,1,1) one cannibal will return
12. (0,0,0) →(3,3,1) two cannibal will go

Farmer river crossing Problem

- A farmer with his wolf, goat and cabbage come to the edge of a river they all want to cross the river. There is a boat at the rivers edge, but only the farmer can row, the boat can carry two things at a time. if the wolf is ever left alone with the goat, the wolf will eat the goat, similar, if the goat is left alone with the cabbage, the goat will eat the cabbage.

Solution

- A set of all possible states for a problem is known as the state space of the problem. There are 16 sub-states of the man , wolf, goat and cabbage.
- Some of the 16 states such as GC-MW are fatal and may never be entered by the system.

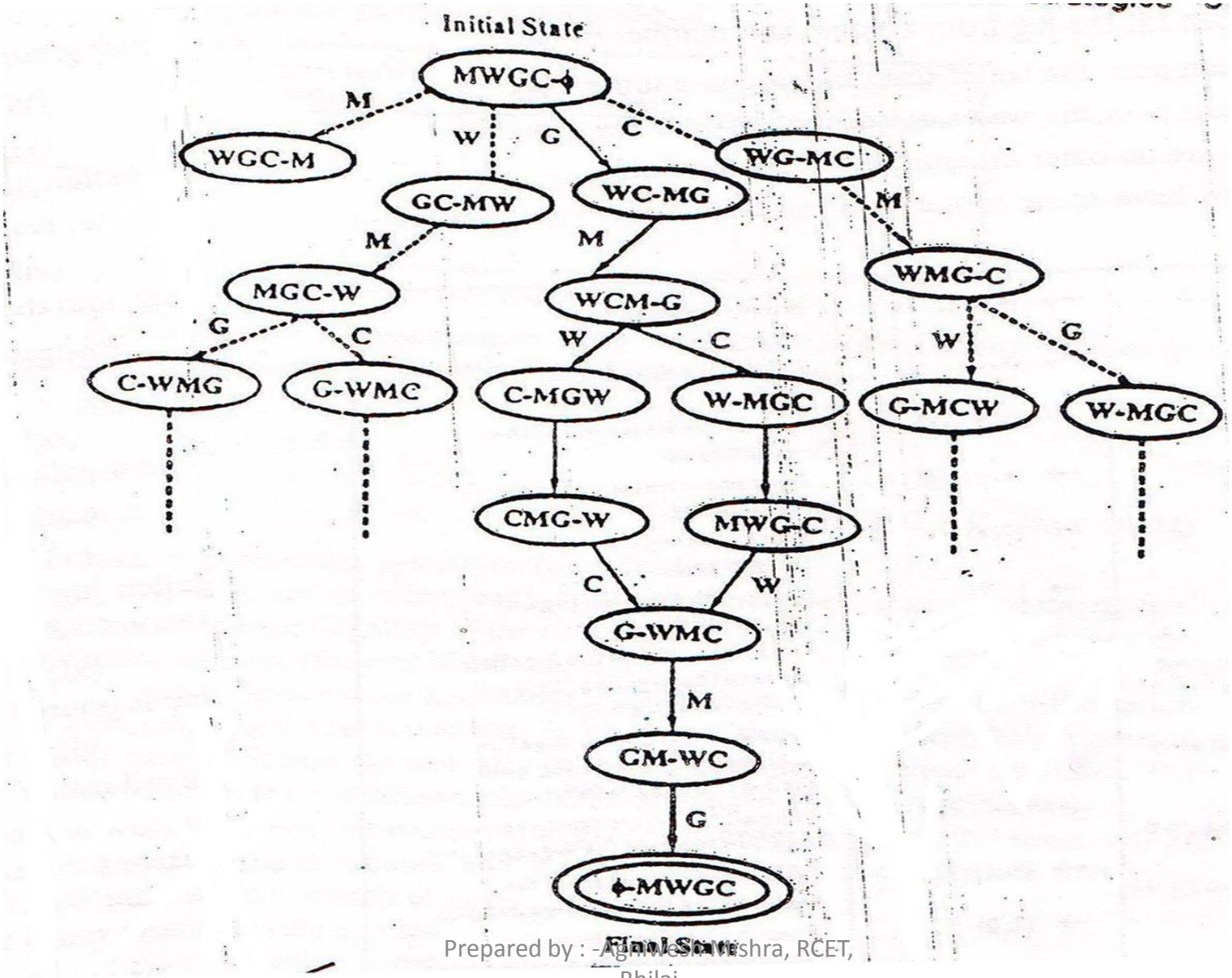
MWGC-ϕ → initial state

ϕ –MWGC → goal state

Sequence of steps for crossing river

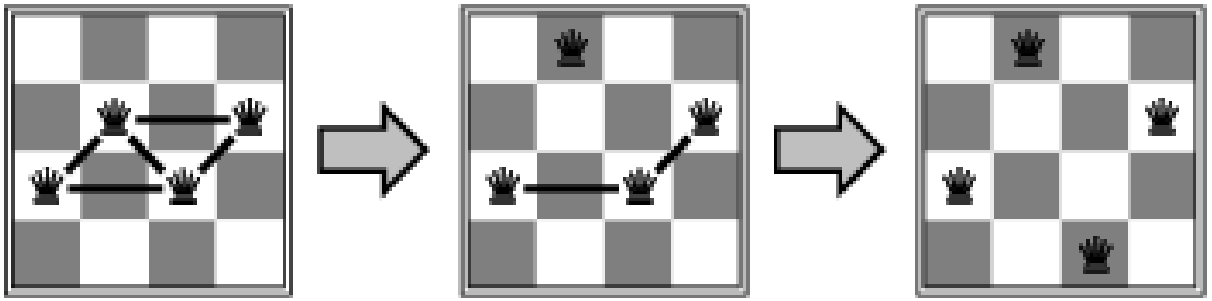
1. First man crosses the river with goat.
2. Man comes back.
3. Man takes either cabbage (or wolf) with him to another side.
4. Man comes back with goat to first side.
5. Man takes wolf (or cabbage) with him to another side.
6. Man comes back.
7. Man takes goat with him to another side.

State space representation



n-queens

- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Actions: move queen in column
- Goal test: no attacks
- Evaluation: *h(n)* = number of attacks



- Given random initial state, we can solve *n*-queens for large *n* with high probability

Search and Control strategies

Production system

Since search forms the core of many intelligent processes, it is useful to structure AI programs in a way that facilitates describing and performing the search process. Production systems provide such structures.

A production system consist of:-

- ✓ A set of rules, each consisting of a left side (pattern) that determines the applicability of the rule and a right side describing the operation to be performed.
- ✓ One or more knowledge/databases that contain whatever information is appropriate for the particular task.
- ✓ A control strategy that specifies the order in which the rules will be compared to the database and a way of resolving the conflicts that arise when several rules match at once.
- ✓ A rule applier which is the computational system that implements the control strategy and applies the rules.

Classes of Production System

- **Monotonic production system:** the application of a rule never prevents the later application of another rule that could also have been applied at the time the first rule was selected.

- **Non-monotonic production system:** Is one in which this is not true.

- **Partially commutative production system:** the application of a particular sequence of rules transforms state x into state y, then any permutation of those rules that is allowable also transforms state x into state y.

- **Commutative production system:** system that is both monotonic and partially commutative.

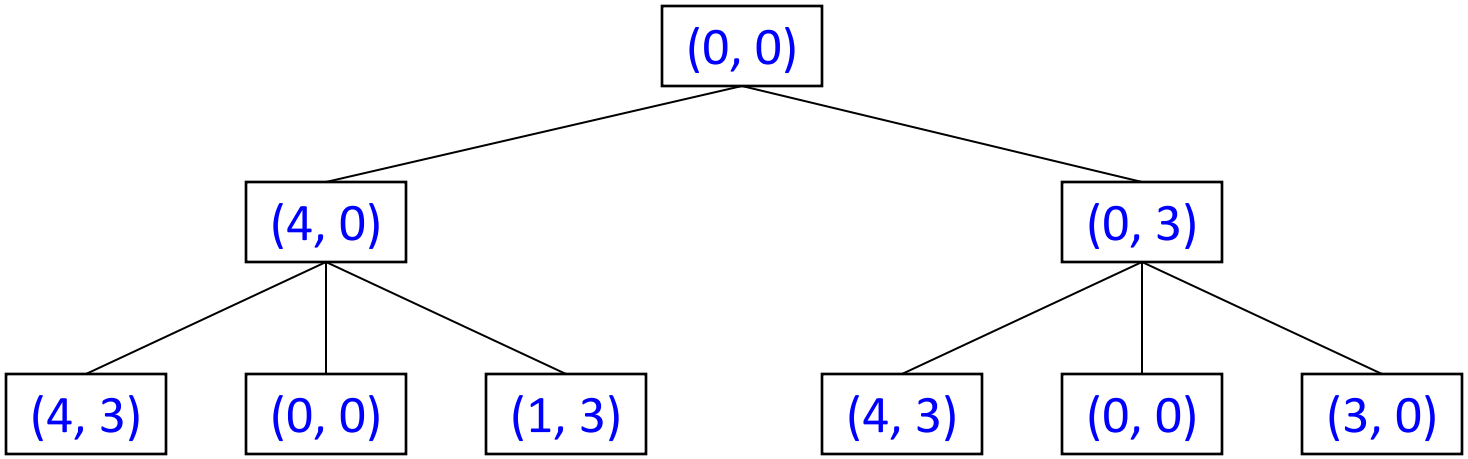
Control strategies

- A control strategy that specifies the order in which the rules will be applied.
- Control strategies help us to overcome the abnormal situations, when there are more than one rules or fewer than one rule will have its left sides match the current state.
- Requirement for control strategy

i. **A good control strategy causes motion**

ii. **A good control strategy is systematic**

Example



Search Strategies

1. Uninformed search (blind search)(Exhaustive search)(Brute force)
   Having no information about the number of steps from the current state to the goal.

2. Informed search (heuristic search)
   More efficient than uninformed search.

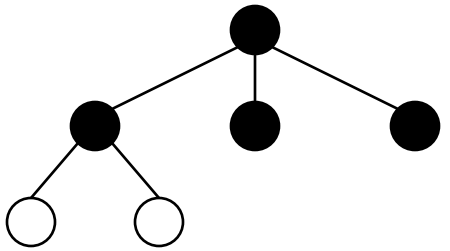
Brute Force or Uninformed Search Strategies

- These are commonly used search procedure which explore all the alternatives during the search process.
- They do not have any domain specific knowledge.
- They need the initial state, the goal state and a set of legal operators.
- The strategy gives the order in which the search space is searched
- The followings are example of uninformed search
  - Depth First Search (DFS)
  - Breadth First Search (BFS)

Search Strategies: Blind Search

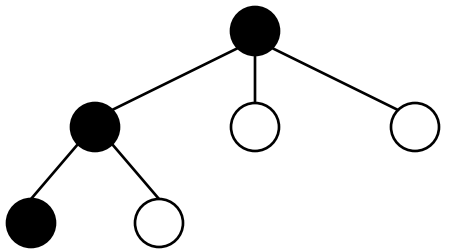
- Breadth-first search

  Expand all the nodes of one level first.



- Depth-first search

  Expand one of the nodes at the deepest level.



Depth First Search

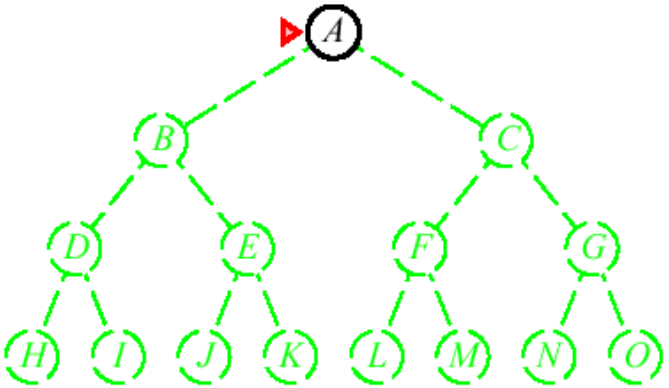
- The search begins by expanding the initial node, generate all successors of the initial node and test them.
- Depth-first search always expands the deepest node in the current frontier of the search tree.
- Depth-first search uses a **LIFO** approach.

Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

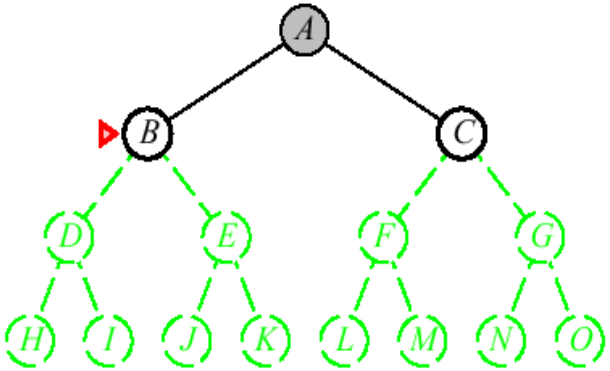


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

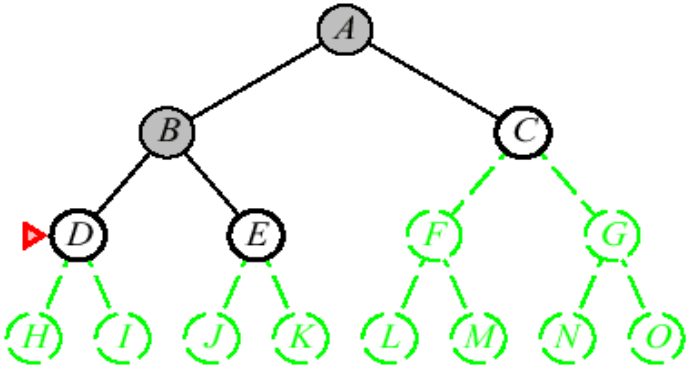


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

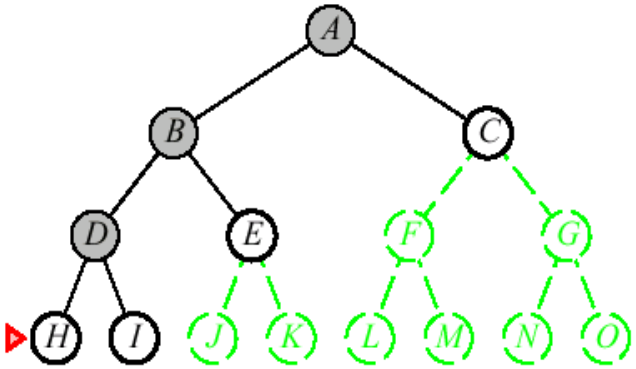


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

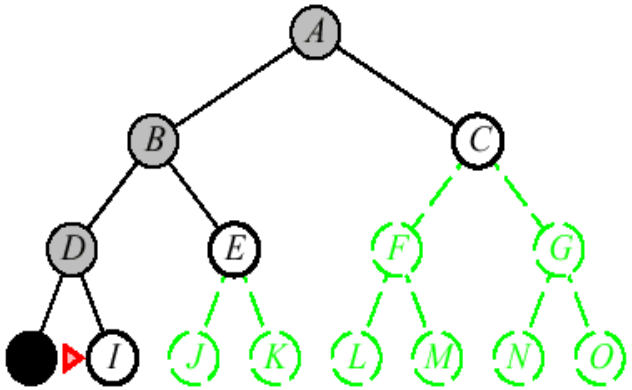


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

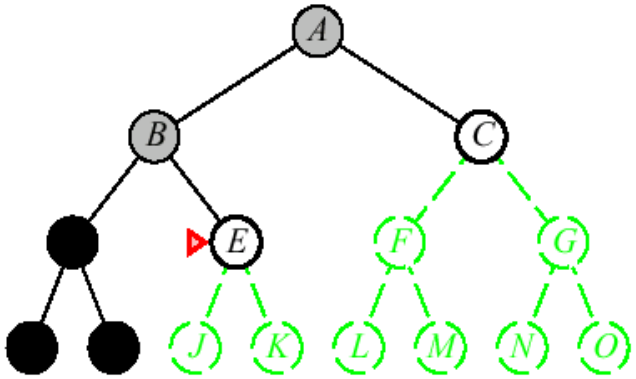


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

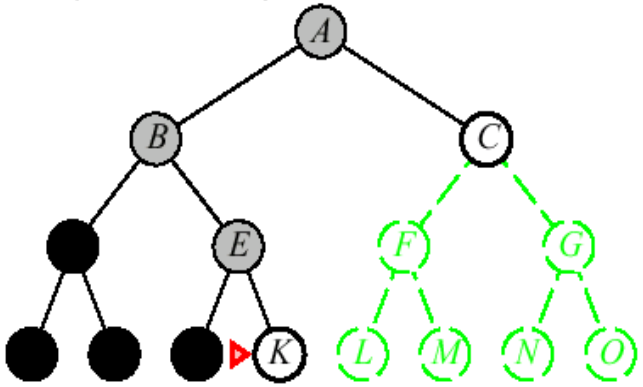


Depth-first search

Expand deepest unexpanded node

Implementation:

$fringe$ = LIFO queue, i.e., put successors at front

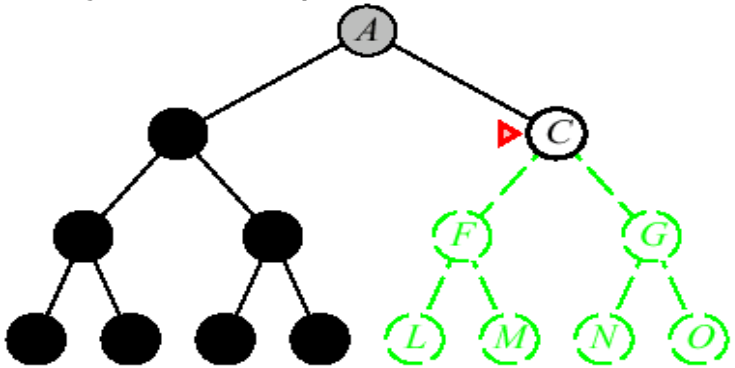


Depth-first search

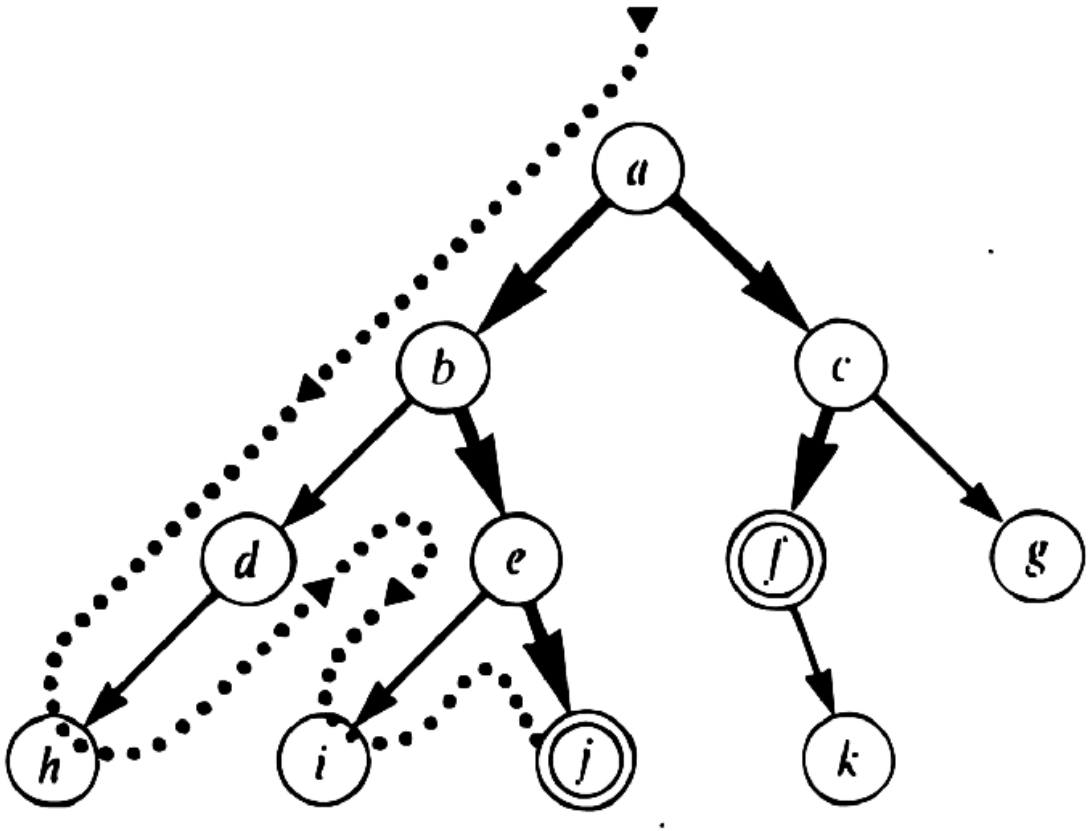
Expand deepest unexpanded node

Implementation:

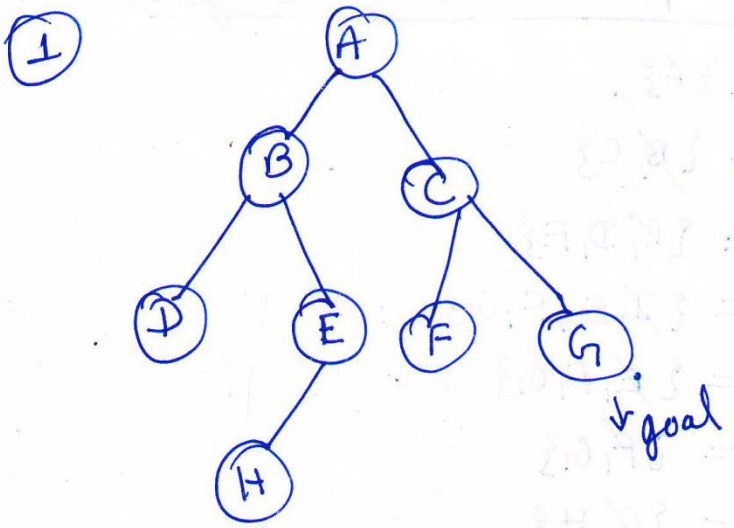
$fringe$ = LIFO queue, i.e., put successors at front



Depth-first search



DFS Example



START = {A}

= {~~B~~, C}

= {~~D~~, E, C}

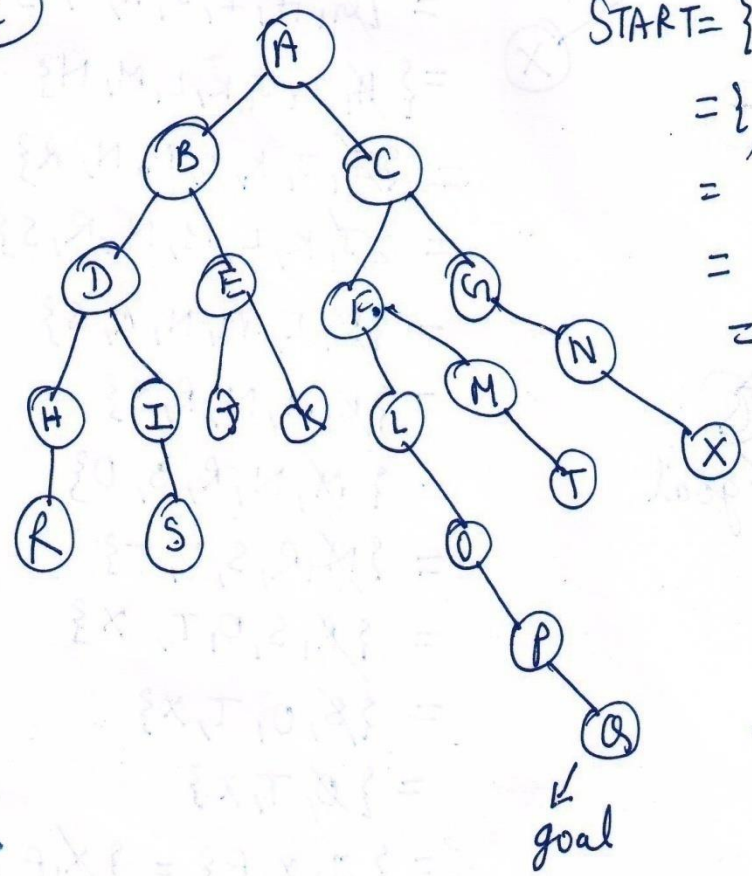
= {~~E~~, C}

= {~~H~~, C}

= {~~F~~C}

= {F, G}

(2)



START = {A}

= {B, C}

= {D, E, C}

= {H, I, E, C}

= {R, I, E C}

= {I, E, C}

= {S, E, C}

= {E, C}

= {K, C}

= {C}

= {F, G}

= {M, G} = {P, M, G}

Algorithm for Depth First Search

1. If the initial state is a goal state, quit and return success.
2. Otherwise, do the following until success or failure is signaled:
   a) Generate a successor, E, of the initial state. If there are no more successors, signal failure.
   b) Call Depth-First Search with E as the initial state.
   c) If success is returned, signal success. Otherwise continue in this loop.

Time and space complexity

Time Complexity :

$$1 + b + b^2 + b^3 + \ldots + \ldots\ldots b^{d.}$$

Hence Time complexity = O (b^{d})

Where b-> branching factor

d-> Depth of a tree

Space Complexity :

Hence Space complexity = O (d)

Advantages of Depth-First Search

i. It requires less memory since only the nodes of the current path are stored.
ii. By chance, it may find a solution without examining much of the search space at all.

Disadvantages of Depth-First Search

i. Determination of the depth until which the search has to proceed. This depth is called cut-off depth.
ii. If the cut-off depth is smaller, solution may not be found.
iii. If cut-off depth is large, time complexity will be more.
iv. And there is no guarantee to find a minimal solution, if more than one solution exists.

Breadth First Search

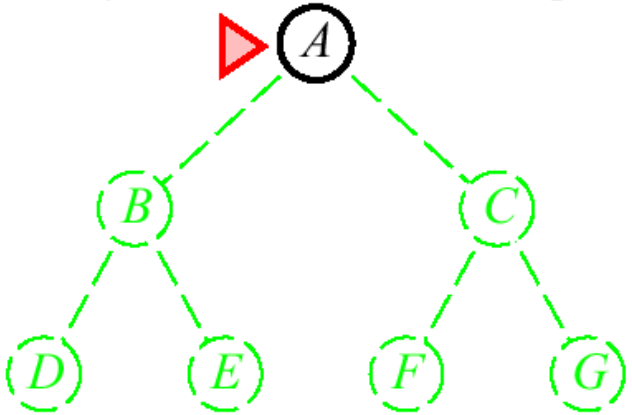
- Searching processes level by level unlike depth first search which goes deep into the tree.
- An operator is employed to generate all possible children of a node.

Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

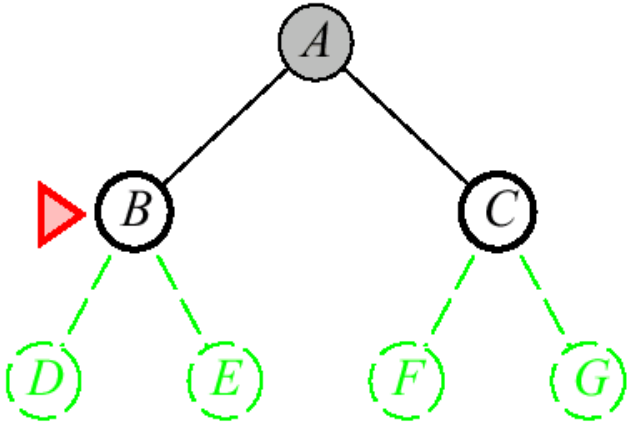


Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

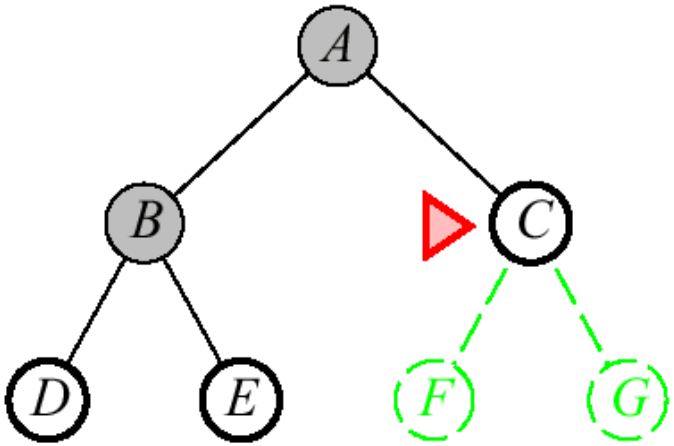


Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

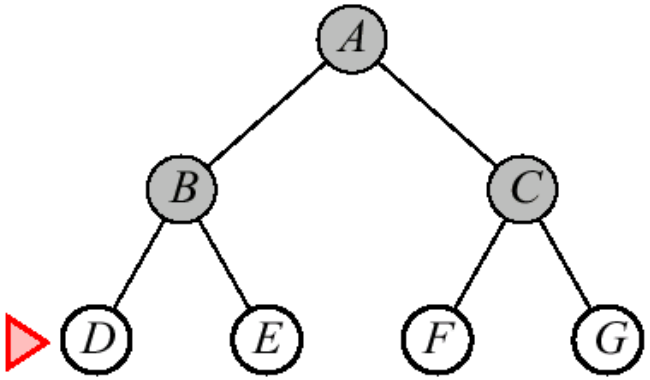


Breadth-first search

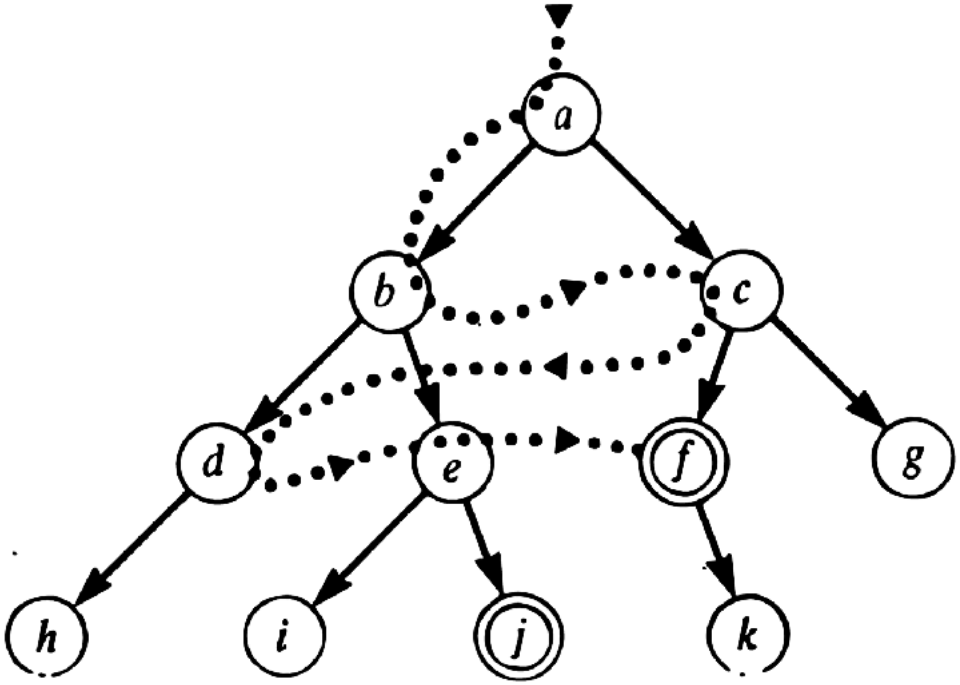
Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end



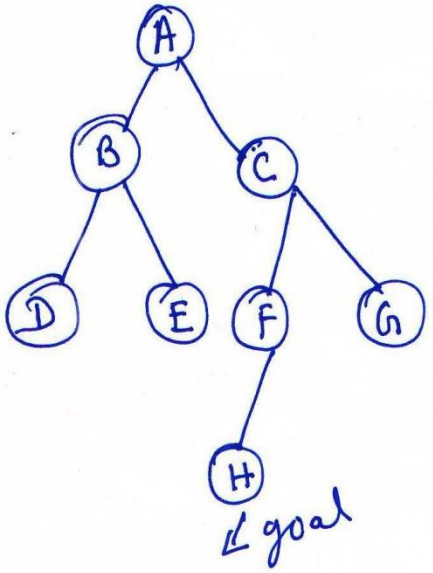
Breadth-first search



- Guaranteed to find shortest solution first
- best-first finds solution a-c-f
- depth-first finds a-b-e-j

BFS Example

(1)



Steps to search the goal

START = {~~A~~}

= {~~B~~, C}

= {~~C~~, D, E}

= {~~D~~, E, F, G}

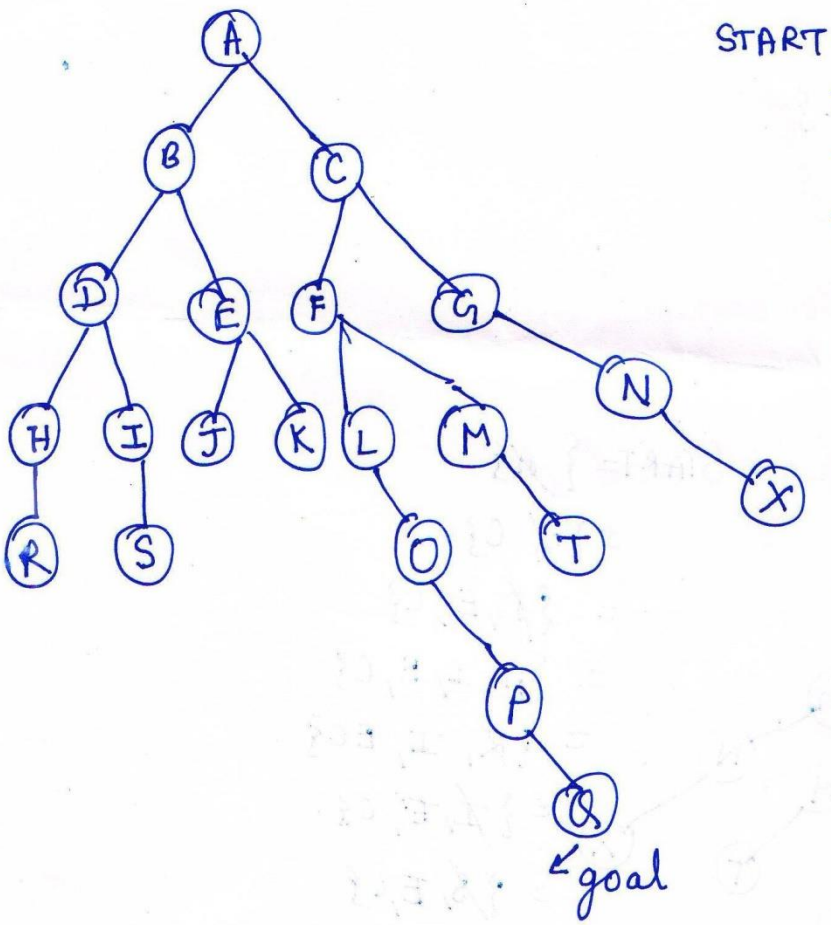
= {~~E~~, F, G}

= {~~F~~, G}

= {~~G~~, H}

= {H}

(2)



START = {A}
= {B, C}
= {C, D, E}
= {D, E, F, G}
= {E, F, G, H, I}
= {F, G, H, I, J, K}
= {G, H, I, J, K, L, M}
= {H, I, J, K, L, M, N}
= {I, J, K, L, M, N, R}
= {J, K, L, M, N, R, S}
= {K, L, M, N, R, S}
= {L, M, N, R, S}
= {M, N, R, S, O}
= {N, R, S, O, T}
= {R, S, O, T, X}
= {S, O, T, X}
= {O, T, X}
= {T, X, P} = {X, P,}
= {P}
= {Q}

Algorithm of Breadth First Search

1. Create a variable called Node-LIST and set it to the initial state.
2. Until a goal state is found or Node-LIST is empty:
   a) Remove the first element from Node-LIST and call it E. if Node-LIST was empty, quit.
   b) For each way that each rule can match the state described in E do:
      i. Apply the rule to generate a new state,
      ii. If the new state is a goal state, quit and return this state.
      iii. Otherwise, add the new state to the end of Node-LIST.

Time and space complexity

Time Complexity :

$$1 + b + b^2 + b^3 + \ldots + \ldots\ldots b^{d.}$$

Hence Time complexity = O (b^d)

Space Complexity :

$$1 + b + b^2 + b^3 + \ldots + \ldots\ldots b^{d.}$$

Hence Space complexity = O (b^d)

Advantages of Breadth-First Search

i. Breadth first search will never get trapped exploring the useless path forever.
ii. If there is a solution, BFS will definitely find it out.
iii. If there is more than one solution then BFS can find the minimal one that requires less number of steps.

Disadvantages of Breadth-First Search

i. It requires more memory
ii. Searching process remembers all unwanted nodes which is of no practical use for the search.

DFS Vs BFS

DFS	BFS
It require less memory because only the nodes on the current path are stored.	It require more memory because all the tree that has so far been generated must be stored.
It is one in which by luck solution can be found without examining much of the search space at all.	While in BFS all parts of the tree must be examined to level n before any nodes on level n+1 can be examined.
It does not give optimal solution.	It gives optimal solution.
DFS may find a long path to a solution in one part of the tree, when a shorter path exists in some other, unexplored part of the tree.	BFS guarantees to find a solution if it exists. Furthermore if there are multiple solutions, then a minimal solution will be found.
Time complexity: $O(b^d)$ where b : branching factor, d: depth	Time complexity: $O(b^d)$ where b : branching factor, d: depth
Space complexity: O(d) , d: depth	Space complexity: $O(b^d)$ where b : branching factor, d: depth

Informed Search

- Informed search tries to reduce the amount of search that must be done by making intelligent choices for the nodes that are selected for expansion.
- In general this is done using a **heuristic function.**

Heuristic Function

- A **heuristic function** is a function that ranks alternatives in various search algorithms at each branching step based on the available information (heuristically) in order to make a decision about which branch to follow during a search.

- Well designed heuristic functions can play an important part in efficiently guiding a search process toward a solution. Sometimes very simple heuristic functions can provide a fairly good estimate of whether a path is any good or not. In other situations, more complex heuristic functions should be employed.

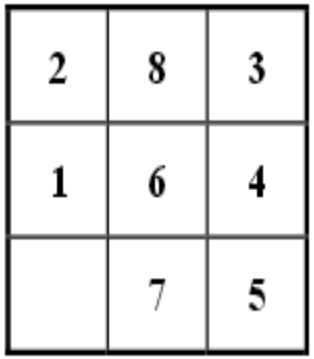
Heuristic Example : 8-puzzle

The first picture shows the current state and the second picture the goal state.

Heuristics → is the number of tiles out of place.

h(n) = 5

because the tiles 2, 8, 1, 6 and 7 are out of place.



Initial State

Goal state

Heuristic Search Algorithm

- Algorithms that use a heuristic function are as follows
  - Hill Climbing
  - Best First Search
  - A*
  - AO*
  - Constraint satisfaction

Hill Climbing

- This algorithm also called discrete optimization algorithm.
- It utilizes a simple heurisitic function.
- Hill Climbing = Depth First Search + Heuristic Function
- There is practically no difference between hill climbing and depth first search except that the childern of the node that has been exapanded are sorted by the remaining distance.

Implementation of Hill Climbing

- There are two ways to implement hill climbing
  - Simple hill climbing
  - Steepest-Ascent hill climbing or gradient search

Simple hill climbing algorithm

1. Evaluate the initial state if goal then return (success) . Else continue with initial state as the current state .
2. Loop until a solution is found or until there are no new operator to apply to current node :
    a) Select a new operator and apply current state to produce a new state .
    b) Evaluate the new state.
        i. if it is a goal then return (success) .
        ii. if not goal but better than current state then make it the current state .
        iii. if it is not better than current state then continue the loop.

Steepest-Ascent hill climbing

1. Evaluate the initial state . If it is also a goal state , then return it and quit . Otherwise , continue with the initial state as current state .
2. Loop until a solution is found or until a complete iteration produces no change to current state :

   a) Let SUCC be a state such that any possible successor of the current state will be better than SUCC .

   b) For each operator that applies to the current state do :

      i) Apply the operator and generate a new state .

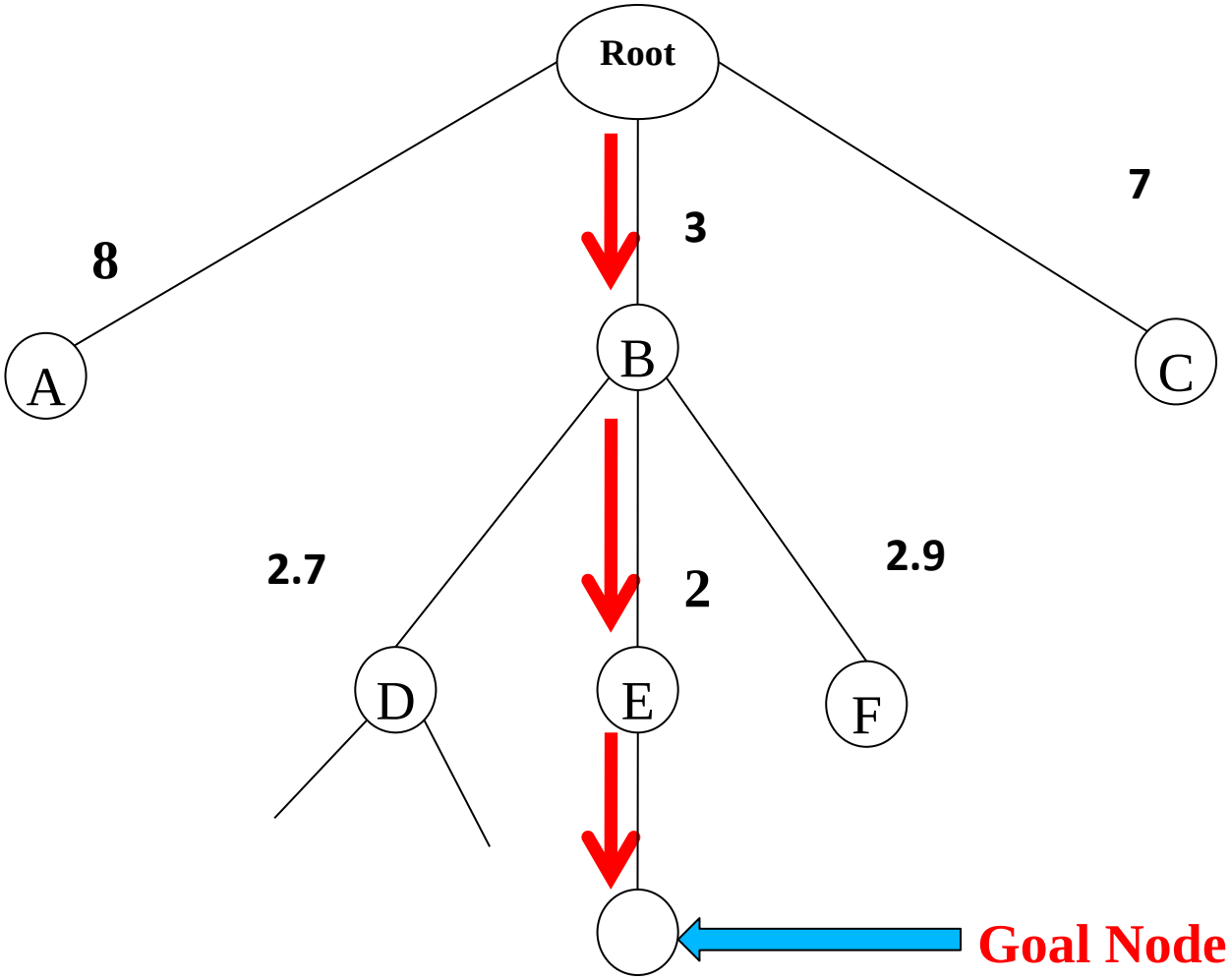
      ii) Evaluate the new state . if it is a goal state , return it and quit . If not, compare it to SUCC . If it is better , then set SUCC to this state . if it is not better , then leave SUCC alone .

      iii) if the SUCC is better than current state , then set current state to SUCC .

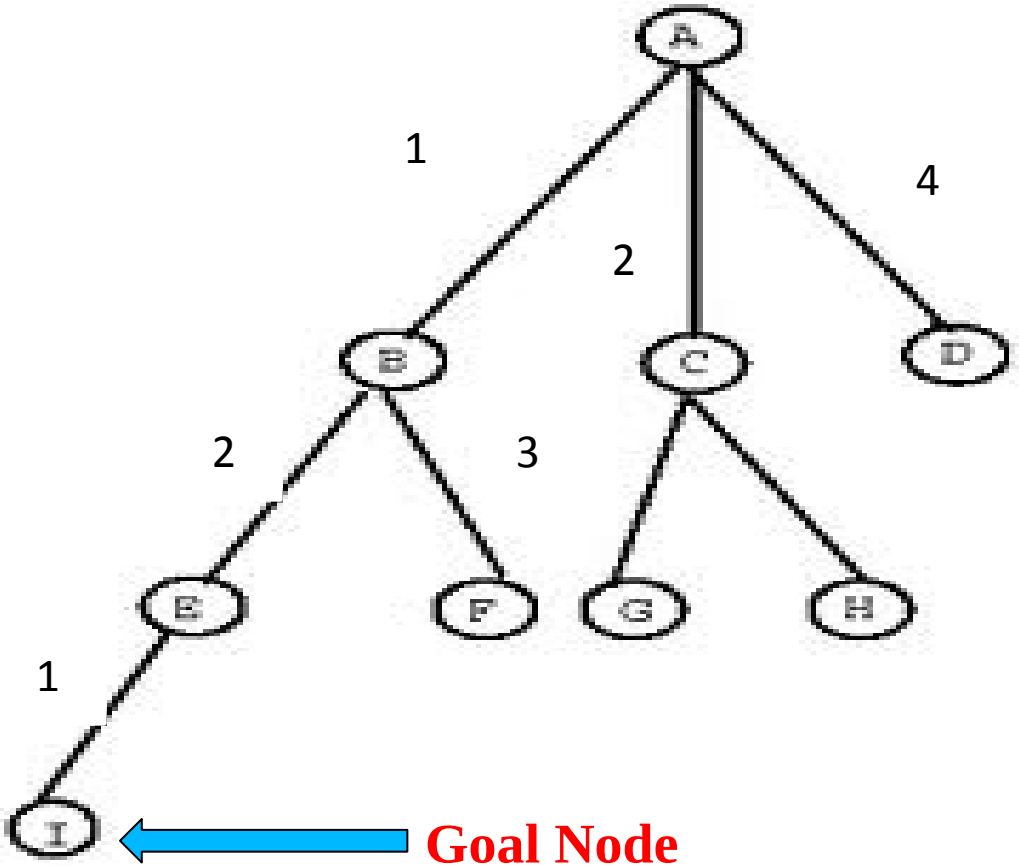
Difference between simple & steepest-ascent hill climbing

- Steepest-ascent hill climbing or gradient search considers all the moves from the current state and selects the best one as the next state.
- In the simple hill climbing the first state that is better than the current state is selected.

Search Tree for Hill Climbing



Example



A

B C D

(1) (2) (4)

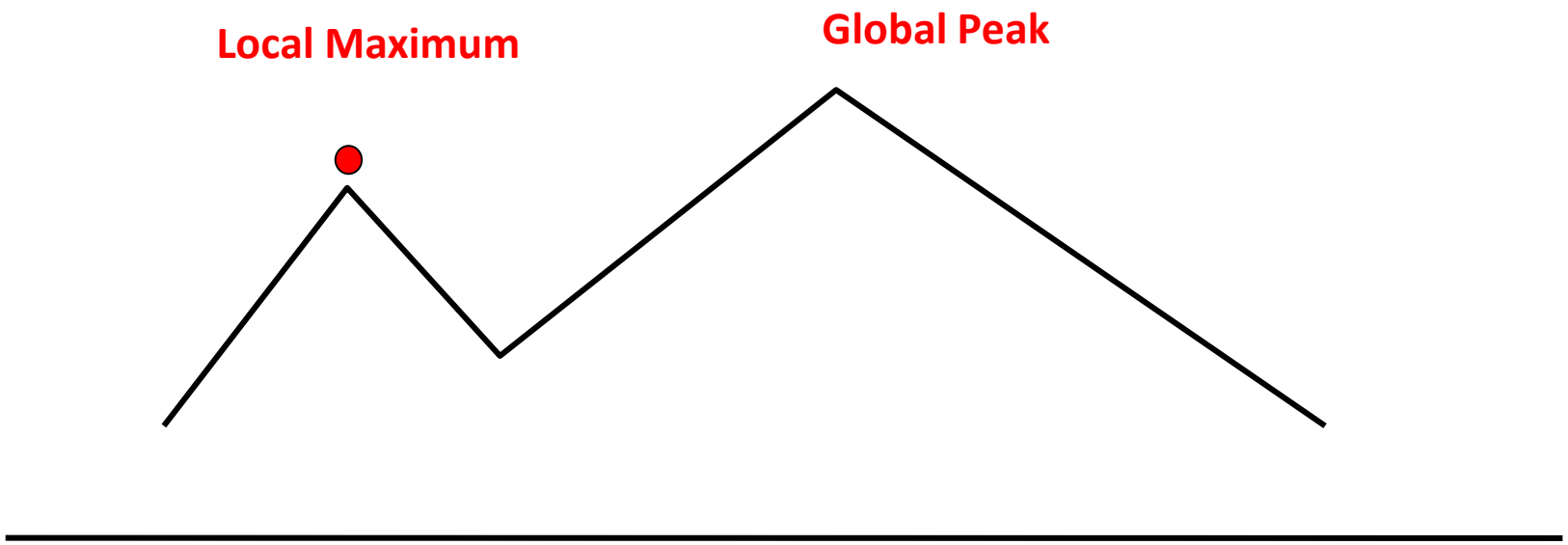
E F

(2) (3)

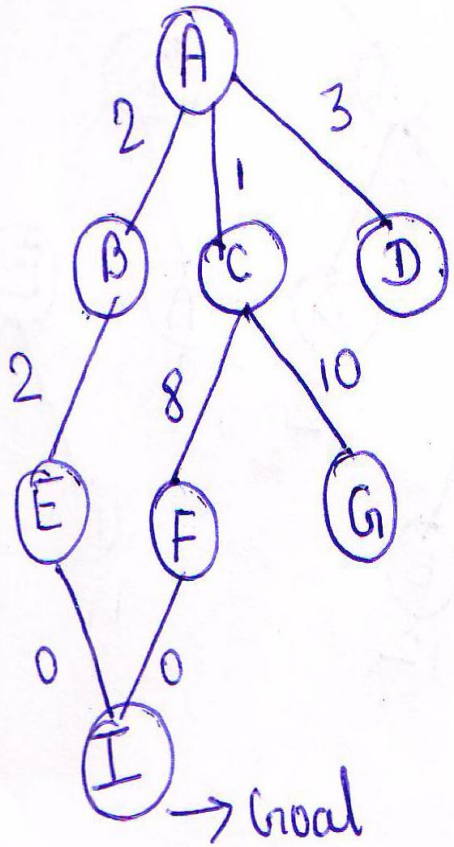
I

Problems with Hill Climbing Technique

- **Local Maximum** : A state that is better than all its neighbours but not so when compared to states to states that are farther away.



Problem of local Maxima



Through minimum path also we are not able to reach the optimum goal state.

Ex: A → C → F → I

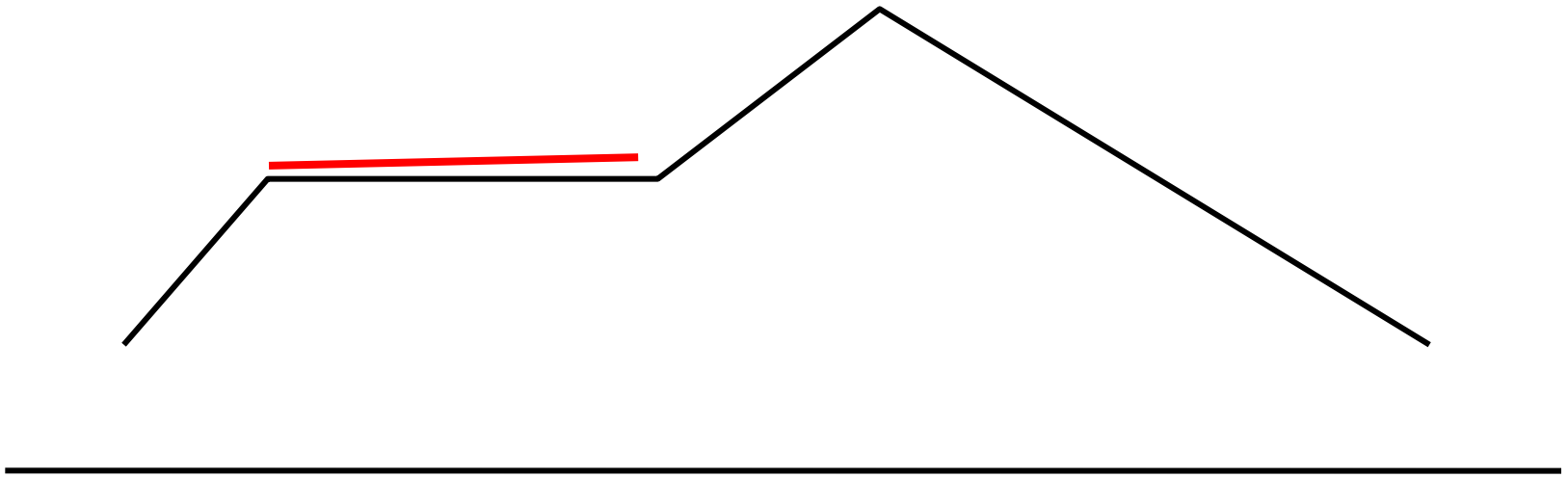
Total cost = 1 + 8 + 0 = 9

A → B → E → I

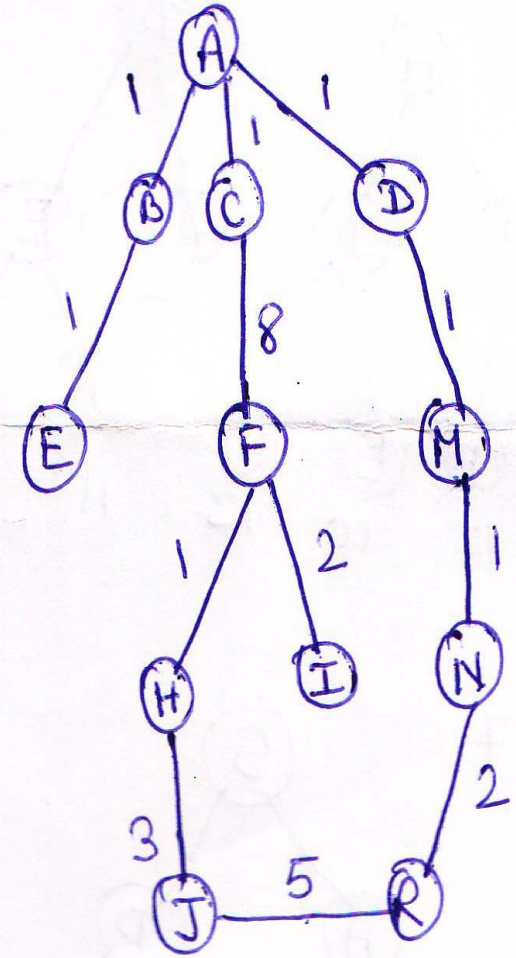
Total cost = 2 + 2 + 0 = 4

Problem with Hill Climbing Technique

Plateau :A flat area of the search space in which all neighbouring states have the same value.



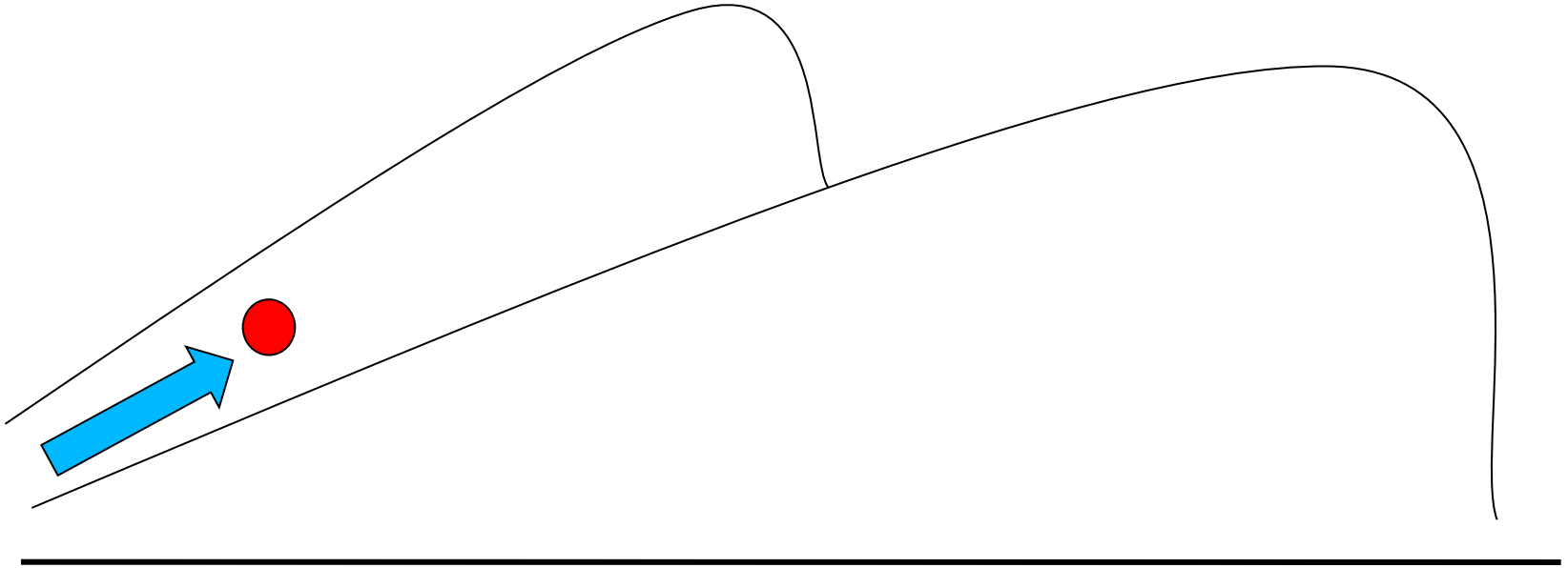
Problem of plateau



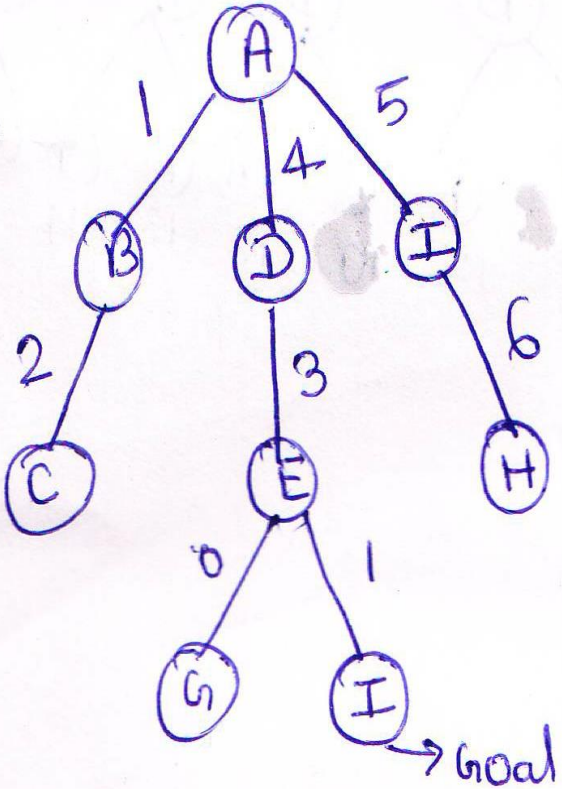
If the value of the path is same, there will be a confusion of choosing the next path for solving the problem.

Problem with Hill Climbing Technique

Ridge :The orientation of the high region, compared to the set of available moves, makes it impossible to climb up. However, two moves executed serially may increase the height.



Problem of Ridge



There is no way to reach toward the goal.

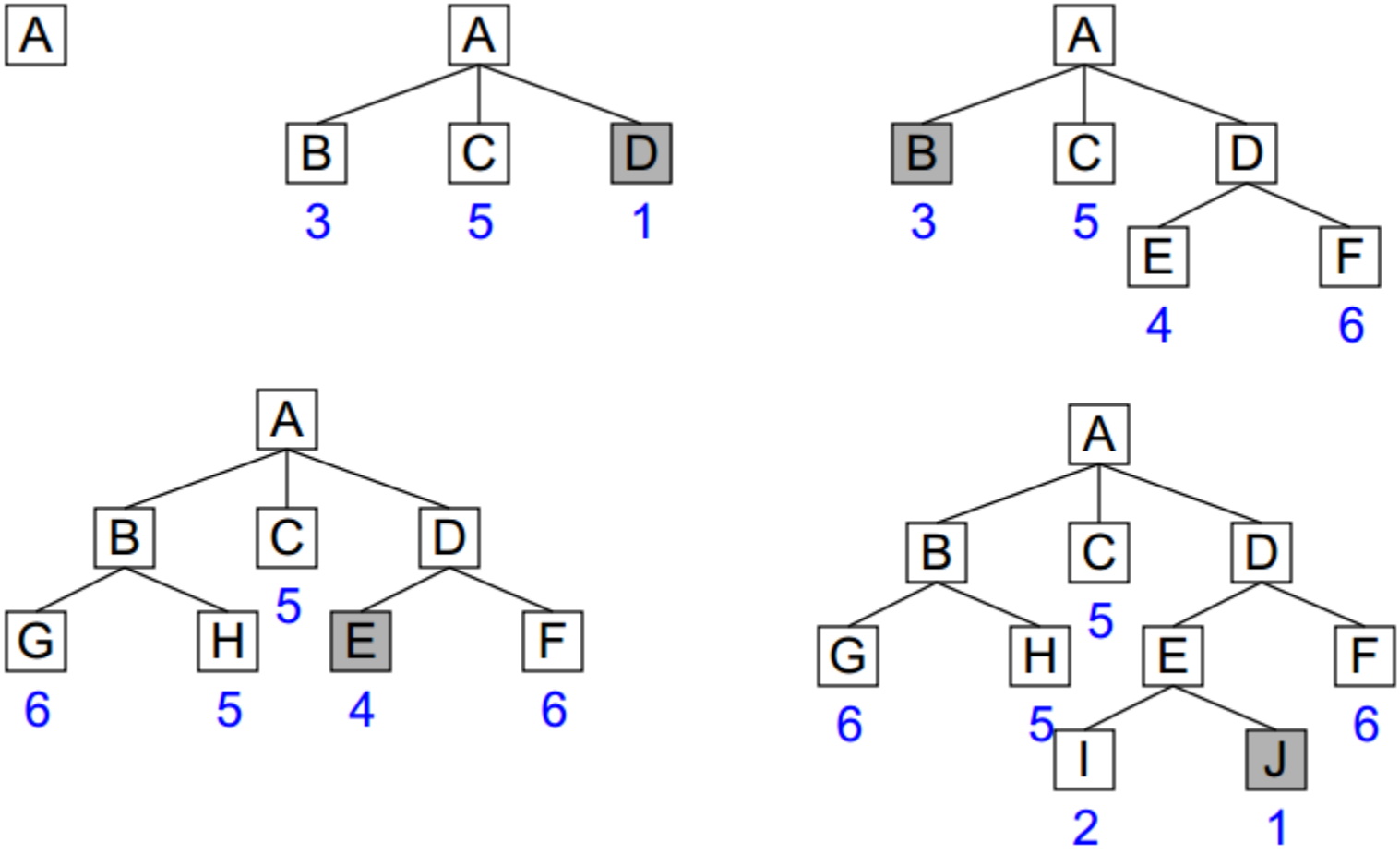
Methods to overcome these problems

- Backtracking for local maximum. Backtracking helps in undoing what has been done so far and permits to try totally different path to attain the global peak.
- A big jump is the solution to escape from the plateau. A huge jump is recommended because in a plateau all neighboring points have the same value.
- Trying different paths at the same time is the solution for circumventing ridges.

Best First Search

- It is a way of combining the advantages of both depth-first search and breadth first search into a single method.
- One way of combining the DFS and BFS is to follow a single path at a time, but switch paths whenever some competing path looks more promising than the current one does.
- At each step of the best-first search process, we select the most promising of the nodes we have generated so far. This is done by applying an appropriate heuristic function to each of them. We then expand the chosen node by using the rules to generate its successors. If one of them is a solution, we can quit. If not, all those new nodes are added to the set of nodes generated so far. Again the most promising node is selected and the process continues.

Best First Search Example



List to maintain in Best-First Search

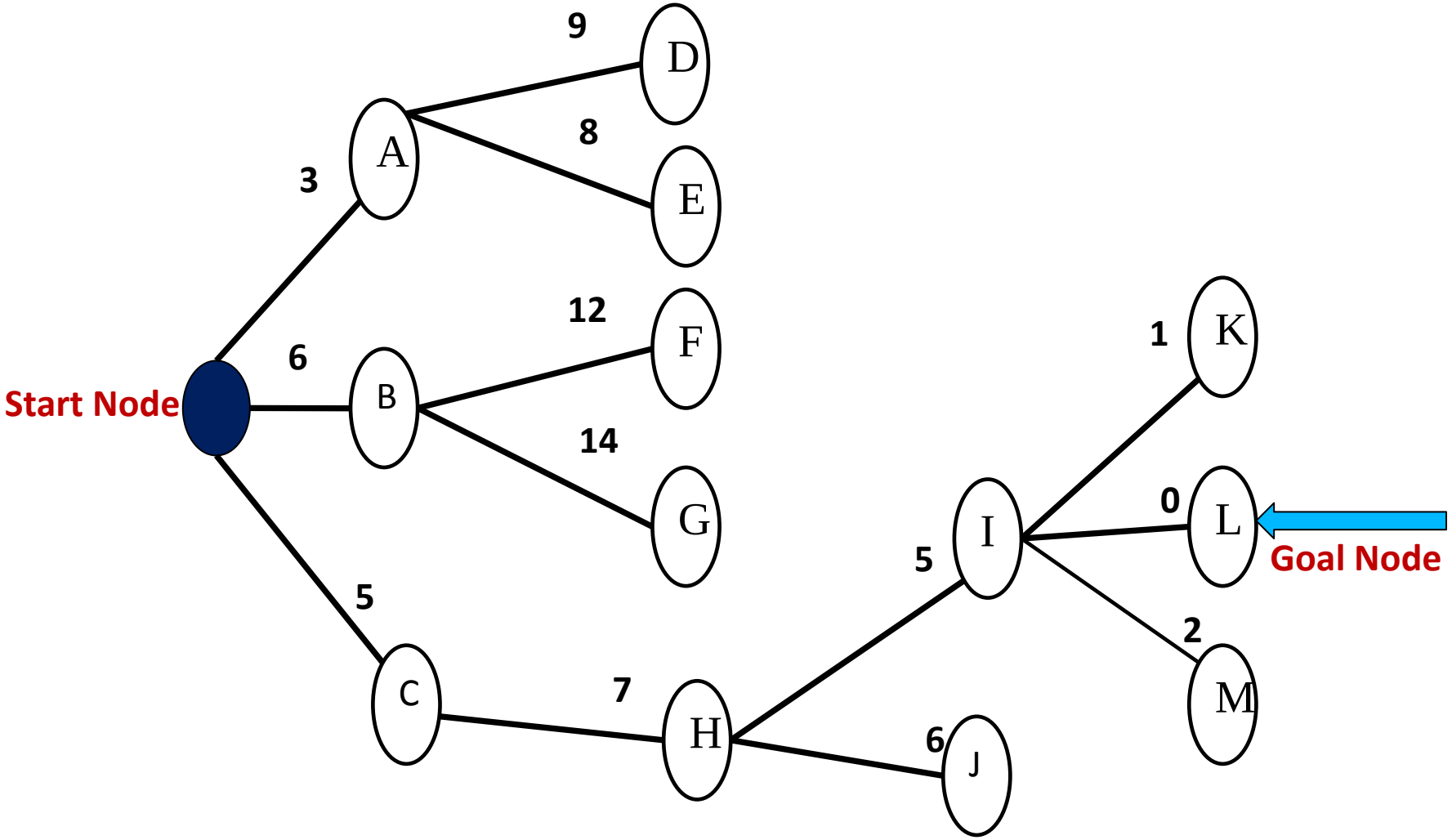
- **OPEN:** nodes that have been generated, but have not examined. This is organized as a priority queue.

- **CLOSED:** nodes that have already been examined. Whenever a new node is generated, check whether it has been generated before.

Algorithm of Best First Search

1. OPEN = {initial state}.
2. Loop until a goal is found or there are no nodes left in OPEN do:
    a. Pick the best node in OPEN
    b. Generate its successors.
    c. For each successor do:
        i. If it has not been generated before, evaluate it, add it to OPEN, and record its parent.
        ii. If it has been generated before, change the parent if this new path is better than the previous one. In that case, update the cost of getting to this node and to any successors that this node may already, have.

A Sample tree for best first search



Search process of best first search

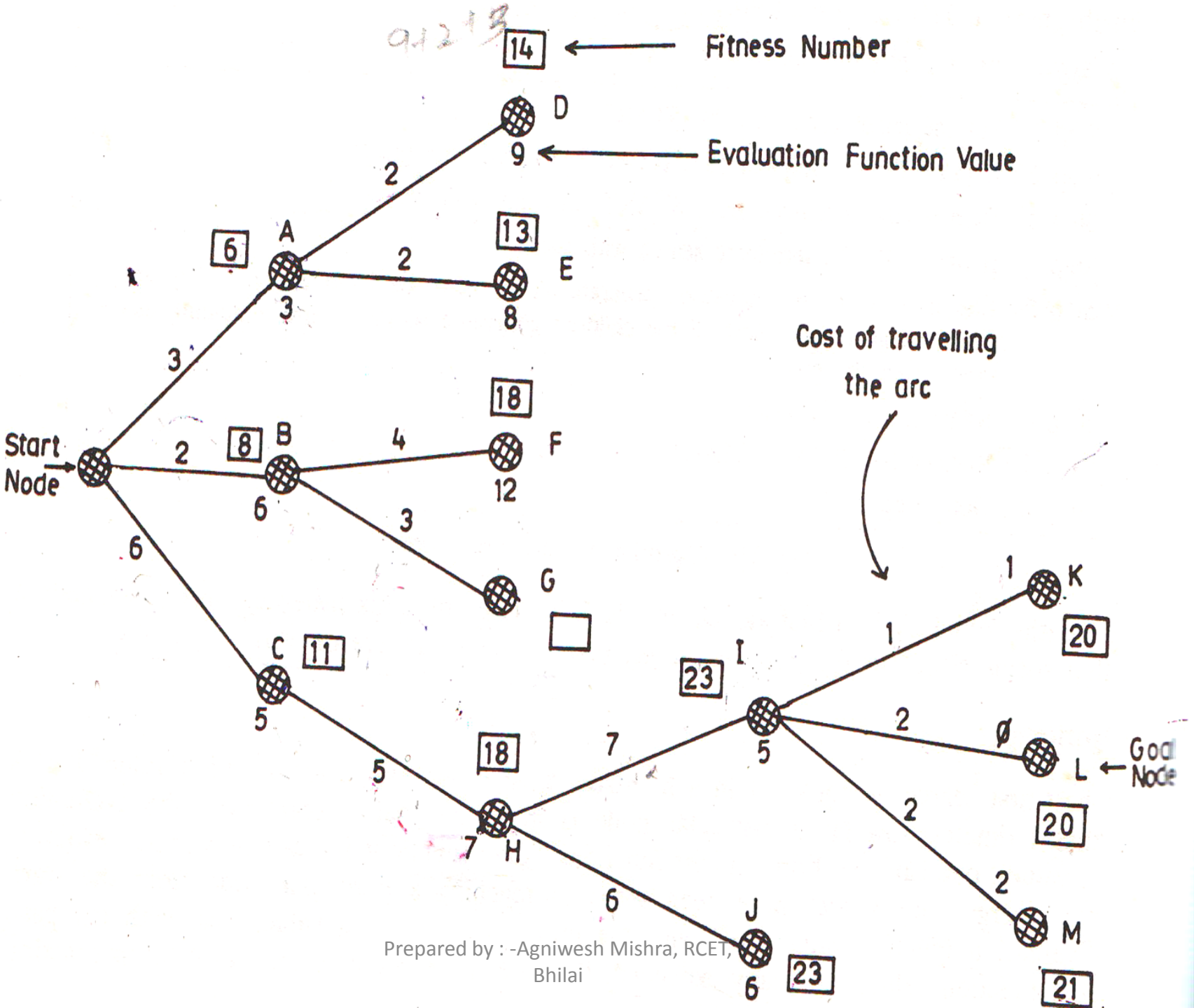
Step	Node being expanded	Children	OPEN List	CLOSE List
1	S	(A:3)(B:6)(C:5)	(A:3)(B:6)(C:5)	(A;3)
2	A	(D:9)(E:8)	(B:6)(C:5) (D:9)(E:8)	(C:5)
3	C	(H:7)	(B:6) (D:9) (E:8) (H:7)	(B:6)
4	B	(F:12) (G:14)	(D:9) (E:8) (H:7) (F:12) (G:14)	(H:7)
5	H	(I:5) (J:6)	(D:9) (E:8) (F:12) (G:14) (I;5) (J:6)	(I:5)
6	I	(K:1) (L:0) (M:2)	(D:9) (E:8) (F:12) (G:14) (J:6) (K:1)(L:0)(M:2)	Search stops as goal is reached

A*

- A* algorithm was given by Hart, Nilsson and Rafael in 1968.

A* is a best first search algorithm with
$f(n) = g(n) + h(n)$
where
$g(n)$ = sum of edge costs from start to n
$h(n)$ = estimate of lowest cost path from n to goal

$f(n)$ = actual distance so far + estimated distance remaining


14
Fitness Number
D
9
Evaluation Function Value
2
6
A
2
13
E
3
8
3
Cost of travelling
the arc
18
Start
Node
2
8
B
4
F
12
6
3
G
6
C
11
1
K
20
23
I
1
5
2
ø
L
Goal
Node
20
18
7
5
5
7
H
2
2
J
M
6
23
21

Example

Obtain the fitness number for node K

F(n)=g(n)+h(n)

=(cost function involved from start node S to node K)+(evaluation function for K)

=6+5+7+1+1

=20

A* Algorithm

1. Initialize: set OPEN= {s}, CLOSED={ }

   g(s)=0, f(s)=h(s)

2. Fail: if OPEN ={ }, Terminate & fail.

3. Select: select the minimum cost state, n, from OPEN. Save n in CLOSED.

4. Terminate: if n $\in$ G, terminate with success, and return f(n).

5. Expand: for each successor , m, of n

   If m $\notin$[open U closed]

   Set g(m) =g(n) + C(n,m)

   Set f(m) =g(m) + h(m)

   Insert m in OPEN.

   If m $\in$[open U closed]

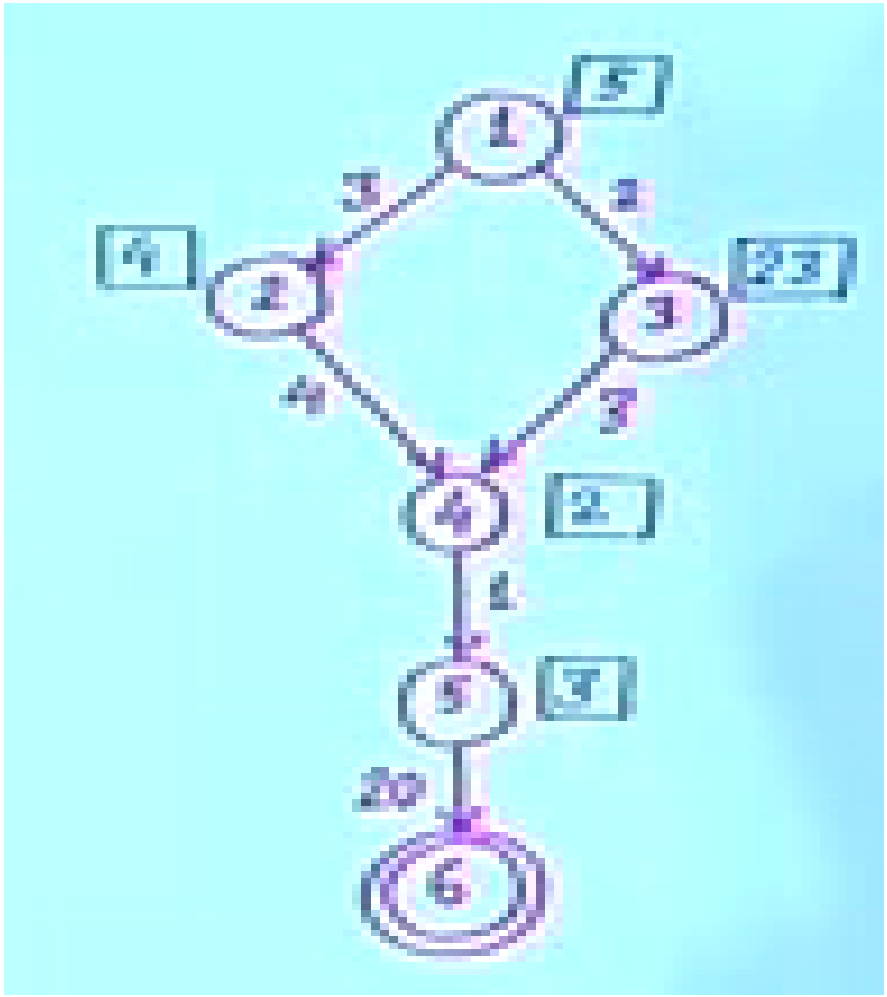
   Set g(m) =min{g(m) ,g(n)+ C(n,m)}

   Set f(m) =g(m) + h(m)

   If f(m) has decreased and m $\in$CLOSED, move m to OPEN

6. Loop: go to step 2

Example



OPEN
1 (5)
2 (7) 3 (25)
3 (25) 4 (9)
3 (25) 5 (11)
3 (25) 6 (28)
4 (7) 6 (28)
6 (28) 5 (9)
6 (28) 6 (26)

CLOSED
1 (5) 2 (7) 4 (9)
5 (11) 3 (25) 4 (7) 5 (9)

Merit and demerit of A* Algorithm

Merits

A* is both complete and admissible. Thus A* always finds an optimal path, if one exists.

Demerits

It is costly if the computation cost is high.

Problem Reduction

Problem Reduction

- Sometimes problems only seem hard to solve. A hard problem may be one that can be reduced to a number of simple problems...and, when each of the simple problems is solved, then the hard problem has been solved.

- Problem reduction may be defined as planning how best to solve a problem that can be recursively decomposed into subproblems in multiple ways.

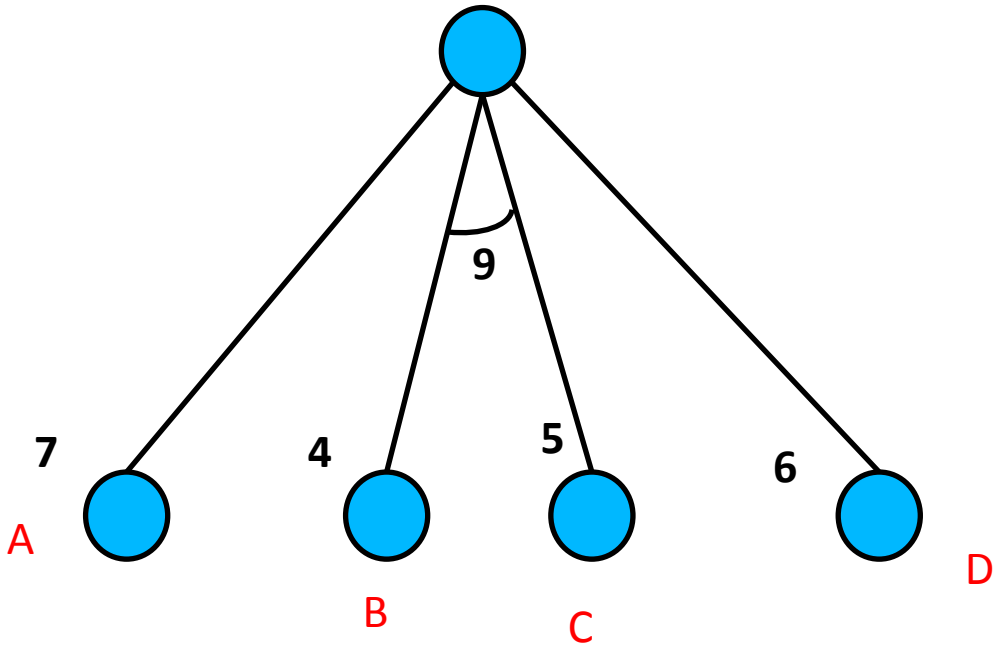
AND or OR

- The complex problem and the sub problem , there exist two kinds of relationships.
  - AND relationship
  - OR relationship.
- In AND relationship, the solution for the problem is obtained by solving all the sub problems.
- In OR relationship, the solution for the problem is obtained by solving any of the sub problem.
- An arc (‿) connecting different branches is called AND.

AND/OR graphs

- Real life situations do not exactly decompose into either AND tree or OR tree but are always a combination of both.
- AND/OR graph is useful for representating the solutions of problem that can be solve by decomposing them into a set of smaller problems.
- A* algorithm is not adequate for AND/OR graphs.
- AO* algorithm is used for AND/OR graphs.

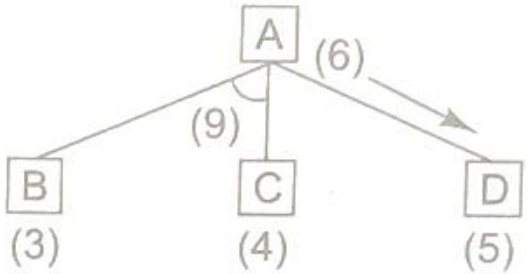
AND/OR tree



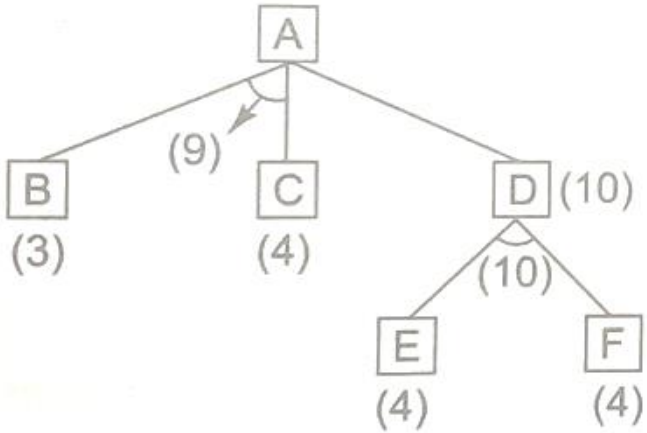
Example

Before step 1

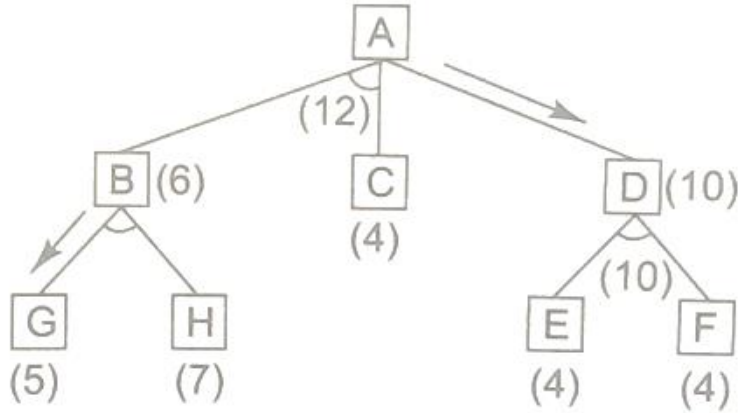
Before step 2



Before step 3



Before step 4



AO* Algorithm

1. Initialize: set $G^* = \{s\}$, f(s)=h(s)

if $s \in T$, label s as SOLVED

2. Terminate: if s is SOLVED, then terminate.
3. Select: select a non-terminal leaf node n from the marked sub tree.
4. Expand: make explicit the successors of n

   For each new successor, m:

   set f(m)=h(m)

   if m is terminal, label m SOLVED
5. Cost revision: call cost-revise(n)
6. Loop: Go to step 2

Cost-revise(n)

1. create Z={n}
2. If Z={} return
3. Select a node m from z such that m has no descendants in Z.
4. If m is an AND node with successors.

   r_1 r_2 r_3 …….. r_k

   Set f(m) = $\sum$ [f(r_i) + C(m, r_i)]

   Mark the edge to each successor of m.

   If each successor is labeled SOLVED, then label m as SOLVED.

5. If m is an OR node with successors.

   r_1 r_2 r_3 …….. r_k

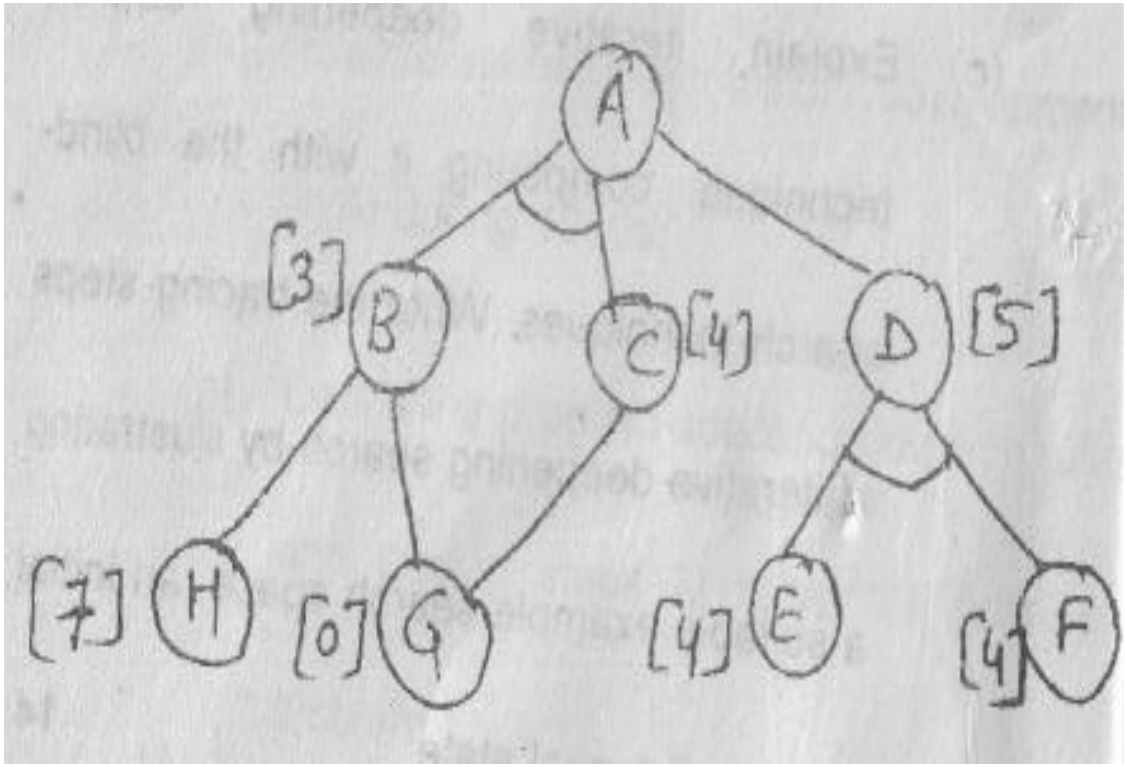
   Set f(m) = min[f(r_i) + C(m, r_i)]

   Mark the edge to best successor of m.

   If the marked successor is labeled SOLVED, then label m as SOLVED.

6. If the cost or label of m has changed, then insert those parents of m into z for which m is a marked successor.

Example



Illustrate the operation of AO* search upon the following search space.

Constraint Satisfaction

- Many AI problems can be viewed as problems of constraint satisfaction.

Examples

- Scheduling
- Timetabling
- Supply Chain Management
- Graph colouring
- Puzzles

Constraint Satisfaction Problem(CSP)

- A CSP consists of
  - A set of variables, X
  - For each variable x_i in X, a domain D_i
  - D_i is a finite set of possible values
- A solution is an assignment of a value in D_i to each variable x_i such that every constraint satisfied.

Crypt-arithmetic puzzle

```
  SEND
+ MORE
------
 MONEY
```

- We have every letters standing for a digit and every letter stands for a different digit.
- We have to find an assignment of letters to digits such that a given arithmetic formula is correct.
- Variables are D, E, M, N, O, R, S, Y
- Domains are
  - {0, 1,2,3,4,5,6,7,8,9} for D, E,N,O,R,Y
  - {1,2,3,4,5,6,7,8,9} for S, M

Constraints for this problem

Constraint 1:

We can write one long constraint for the sum.

1000* S + 100* E+10* N+ D

+ 1000* M+ 100* O+10* R+ E

……

10000*M + 1000* O+ 100*N+ 10* E+ Y

Constraint 2:

alldifferentr(D, E, M, N, O, R, S, Y)

These two constraints express the problem precisely.

Solution

- **Rules for propagating constraints generates the following constraints:**

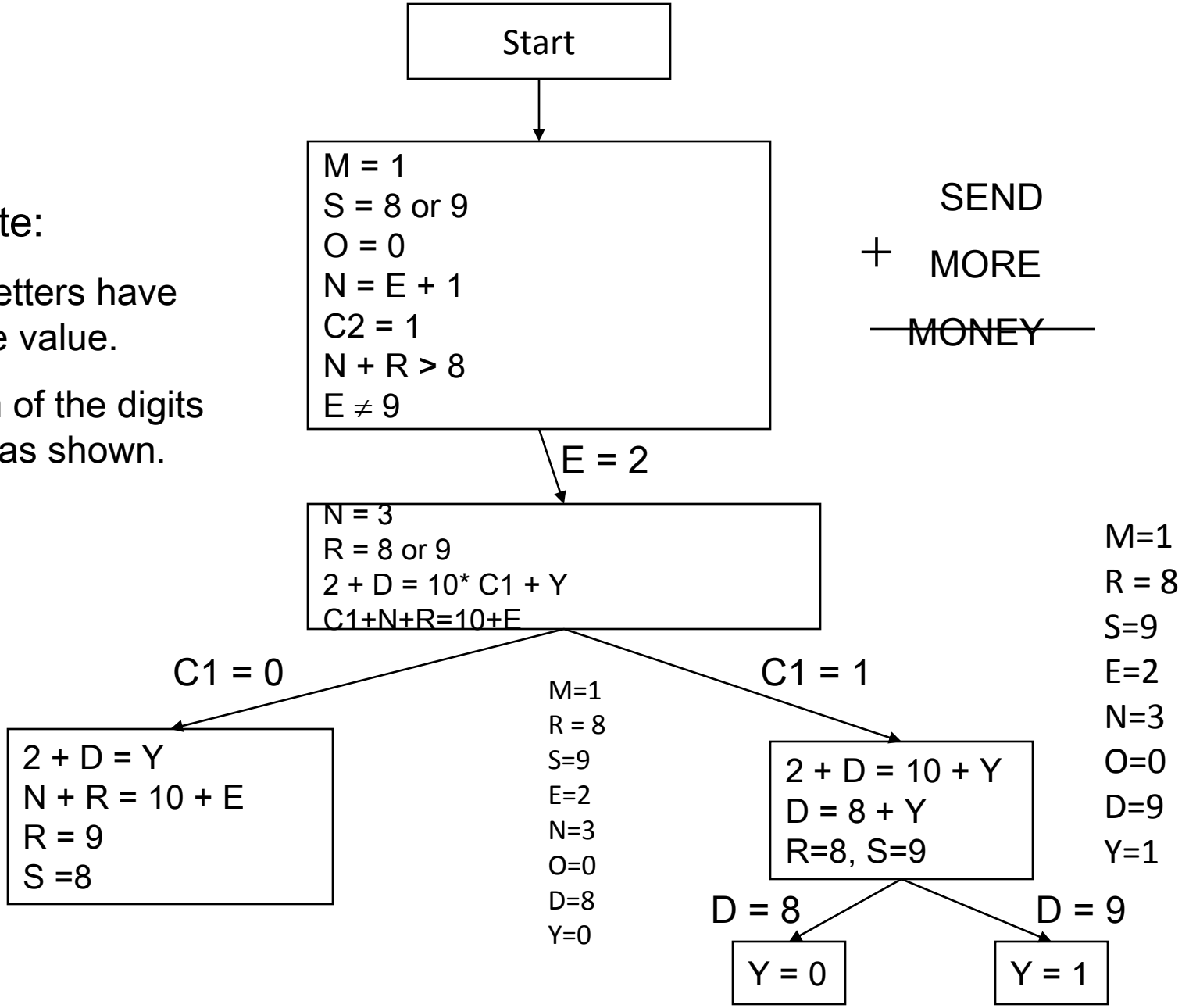
1. M = 1, since two single-digit numbers plus a carry can not total more than 19.
2. S = 8 or 9, since S+M+C3 > 9 (to generate the carry) and M = 1, S+1+C3>9, so S+C3 >8 and C3 is at most 1.
3. O = 0, since S + M(1) + C3(<=1) must be at least 10 to generate a carry and it can be most 11. But M is already 1, so O must be 0.
4. N = E or N=E+1, depending on the value of C2. But N cannot have the same value as E. So N = E+1 and C2 is 1.
5. In order for C2 to be 1, the sum of N + R + C1 must be greater than 9, so N + R must be greater than 8.
6. N + R cannot be greater than 18, even with a carry in so E cannot be 9.

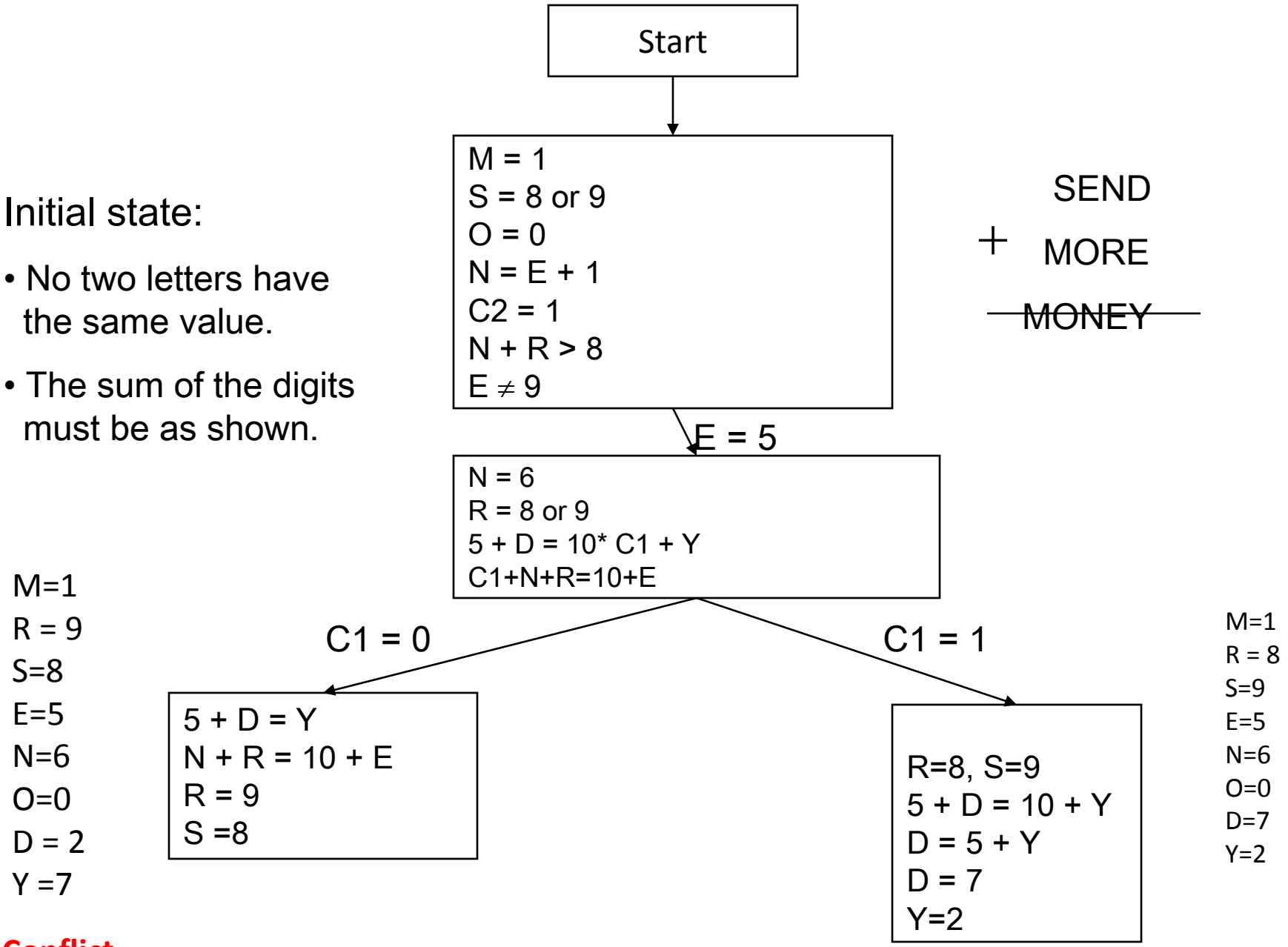
Solution...

- Suppose E is assigned the value 2.
- The constraint propagator now observes that:
- N = 3 since N = E + 1.
- R = 8 or 9, since R + N (3) + C1 (1 or 0) = 2 or 12. But since N is already 3, the sum of these nonnegative numbers cannot be less than 3. Thus R + 3 +(0 or 1) = 12 and R = 8 or 9.
- 2 + D = Y or 2 + D = 10 + Y, fro the sum in rithmost column.

Initial state:

- No two letters have the same value.
- The sum of the digits must be as shown.





Initial state:

- No two letters have the same value.
- The sum of the digits must be as shown.

Conflict

Final solution

- M=1
- R = 8
- S=9
- E=5
- N=6
- O=0
- D=7
- Y=2
- C1=1
- C2=1
- C3=0
- C4=1

SOLVE

- US +AS=ALL
- SHE +THE=BEST
- CROSS+ROADS=DANGER
- DAYS+TOO=SHORT

Solve

CROSS

+ROADS

.....

DANGER

Solution

Rule 1: Well you can see that the DANGER has one more letter than CROSS and ROADS, and the extra letter is D. That means that C + R equals something more than 10. Which also means D is 1.

Rule 2: Oh look, S + S = R. That means that R must be even. We have a choice of 4, 6 and 8, because if R was 2, S would have to be 1, and D is already 1. Let's try 6 for the value of R, because we need high numbers if we want C + R to equal something more than 10. Oh look, if R is 6 and S is R divided by 2, then S must be 3!

Rule 3: S+D=E, 3+1=4, So, E=4

Rule 4: And since we now only have 4 spots in the key left, we choose the highest number for C, which is 9. Again, we need high numbers to make C + R equal something more than 10.

Rule 5: In the equation, O + A = G. We have 2, 5, 7 and 8 vacant. Let's play around with these letters. Let's see if we can find an equation in there. Yes! There is an equation there. 5 + 2 = 7! So G must equal 7. We know that 9 + 6 = 15, but it's missing the 5! So, A must equal 5. In turn, this leads to O having to be 2 (do the maths, O + 5 = 7). And last of all, O is 2, so 6 + 2 (6 + O) = N But 6 + 2 = 8, so now N is 8. We now have the following equation...

- 96233
  62513

  158746

  And the following key...

- C=9
- R=6
- O=2
- S=3
- A=5
- D=1
- E= 4
- N=8
  G=7

DAYS **+ TOO** **……** **SHORT**	**9871** **655** **……** **10526**
US +AS …… ALL	85 15 …… 100
SHE +THE …….. BEST	634 834 …….. 1468
CROSS +ROADS …… DANGER	96233 62513 …… 158746

Games in Artificial Intelligence

Why has game playing been a focus of AI?

- Games have well-defined rules, which can be implemented in programs
- Interfaces required are usually simple
- Many human expert exist to assist in the developing of the programs.
- Games provide a structured task wherein success or failure can be measured with least effort.

Game Playing (Basic strategy)

- John von Neumann is acknowledged as father of game theory.
- The term Game means a sort of conflict in which n individuals or groups (known as players) participate.
- Game theory denotes strategy for game.
- Grow a search tree
- Only one player move at each turn
- At the leaf position, when the game is finish, assign the utility to player.

Two-player games

Usual conditions:

- Each player has a global view of the board
- Zero-sum game: any gain for one player is a loss for the other

Major components of a game playing program

Two major components

- **Plausible move generator:** plausible move generator is used to generate the set of possible successor positions.
- **Static evaluation function generator (utility function):** based on heuristics, this generates the static evaluation function value for each and every move that is being made. The static evaluation function gives a snapshot of a particular move.

Game Tree

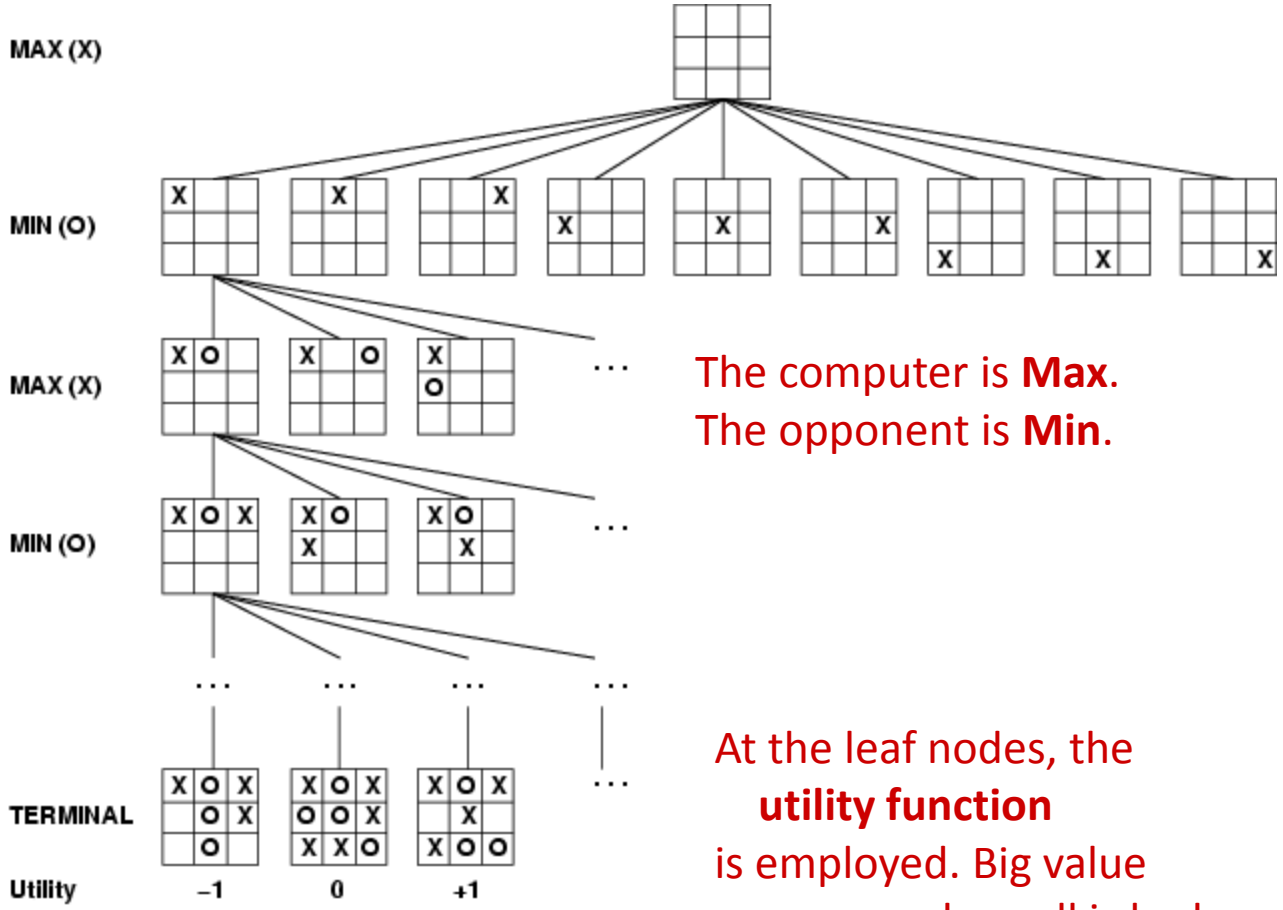
computer's turn

opponent's turn

computer's turn

opponent's turn

leaf nodes are evaluated



The computer is **Max**.
The opponent is **Min**.

At the leaf nodes, the **utility function** is employed. Big value means good, small is bad.

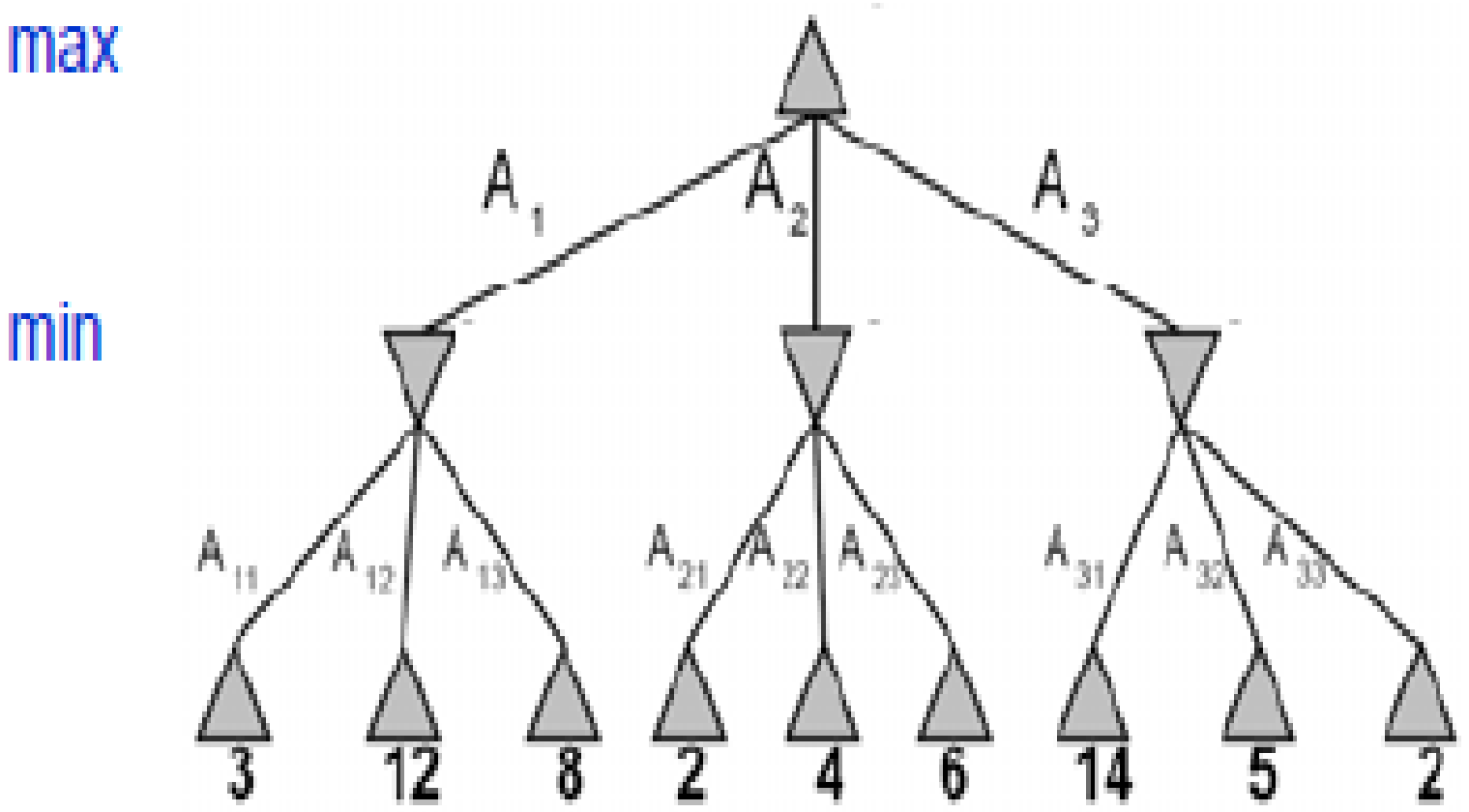
Game playing strategies

- Minimax strategy
- Alpha-Beta Pruning

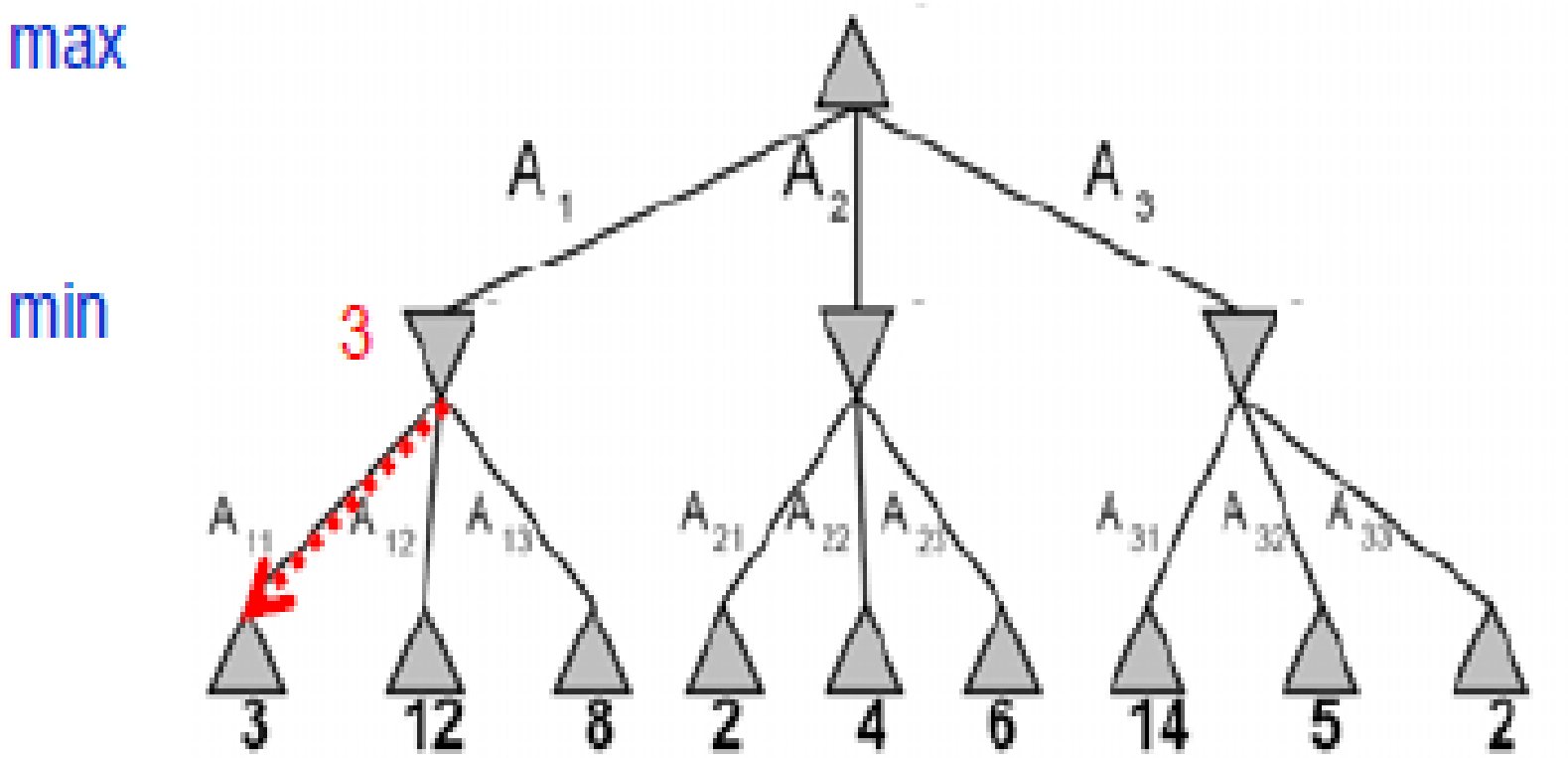
Minimax Strategy

- It is a simple look ahead strategy for two person game playing.
- One player “maximizer” tries to maximize the utility function
- Other player “minimizer” tries to minimize the utility function
- The plausible move generator generates the necessary states for further evaluation and the static evaluation function “ranks” each of the positions.
- To decide one move, it explores the possibilities of winning by looking ahead to more than one stop. This is called ply. To decide the current move, game tree would be explored two levels farther.

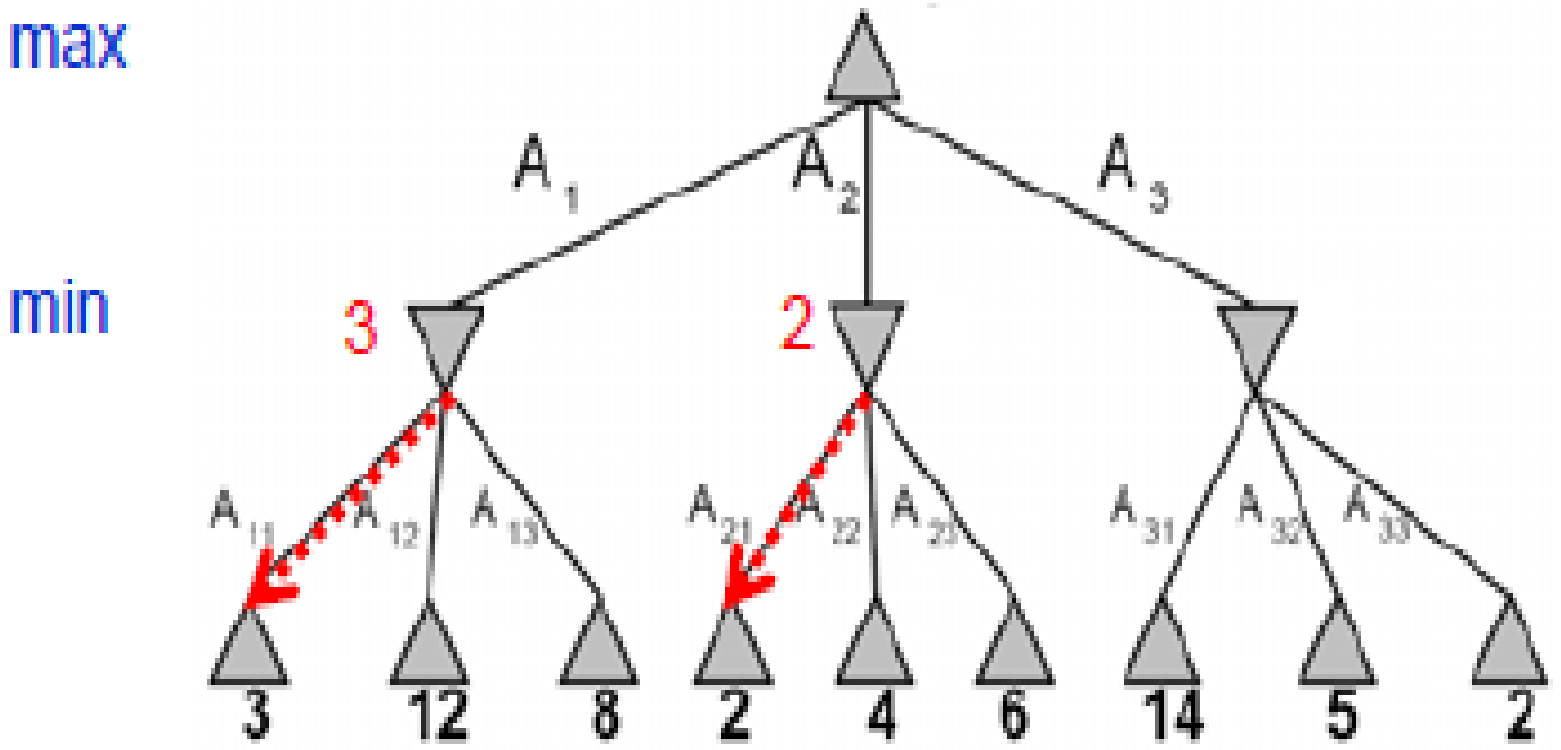
MiniMax Example



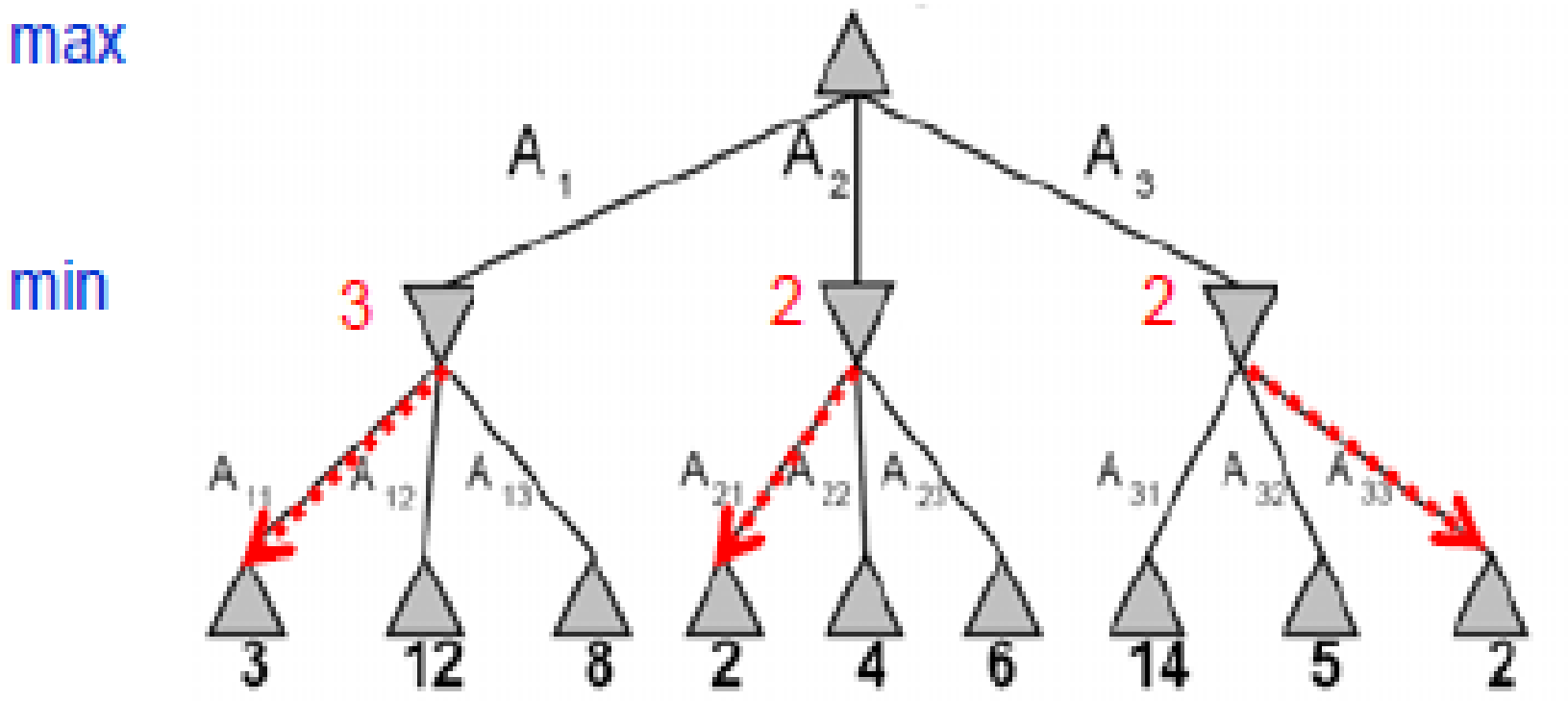
MiniMax Example



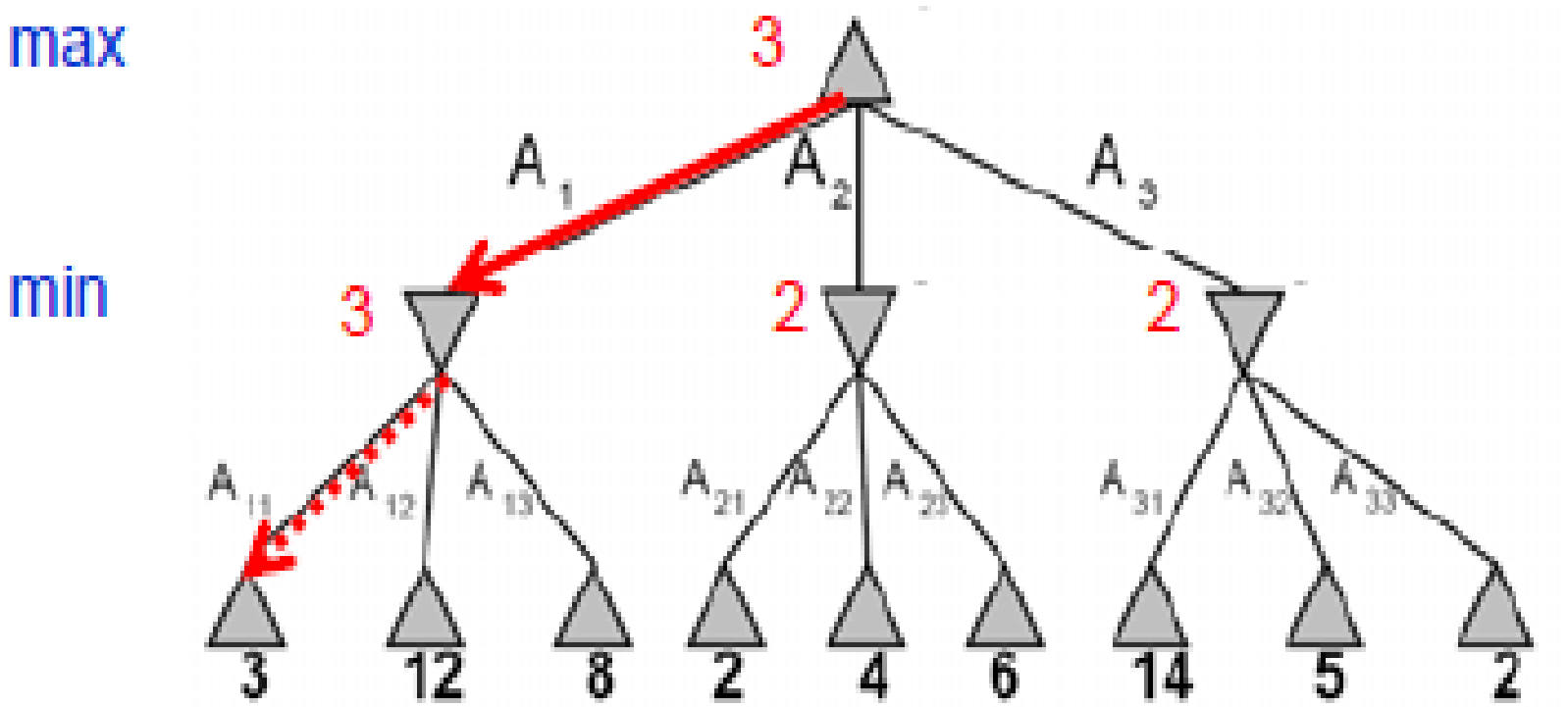
MiniMax Example



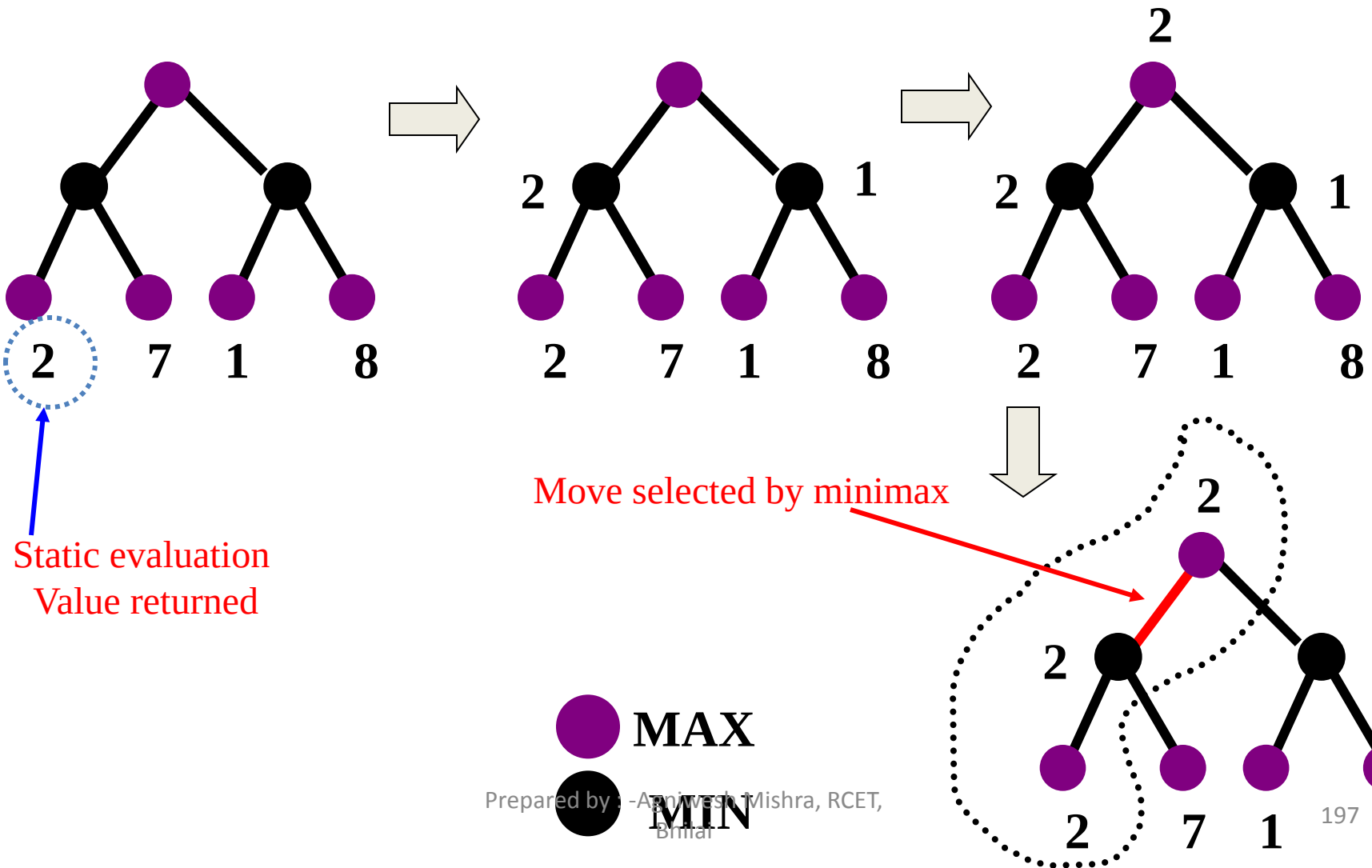
MiniMax Example



MiniMax Example



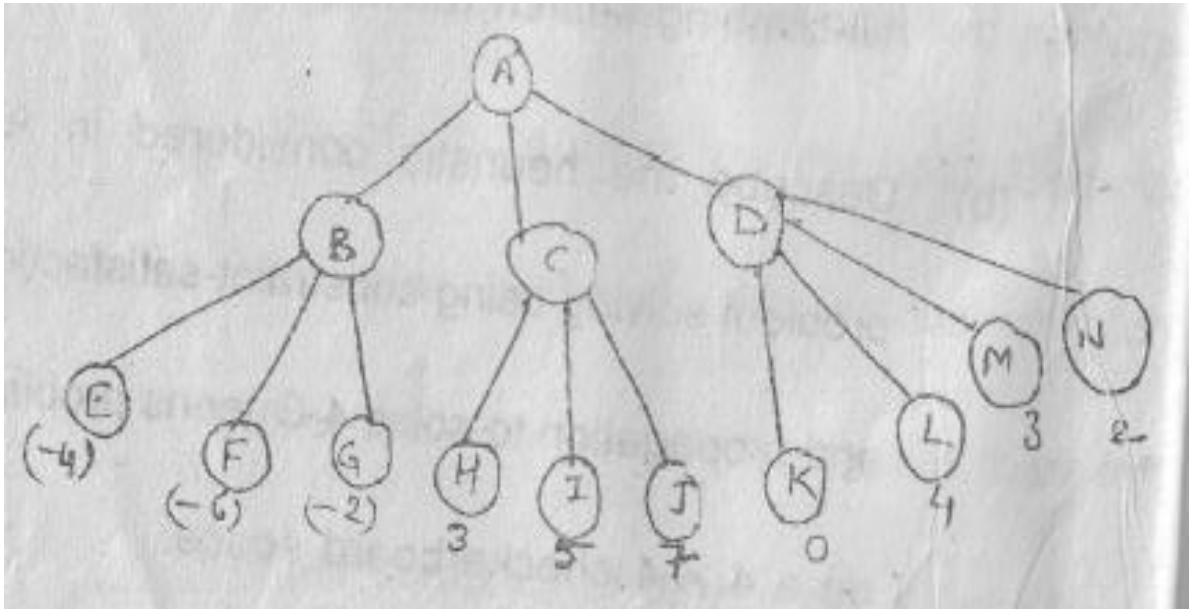
Minimax Algorithm Illustrated



Minimax Algorithm

```
function MINIMAX(N) is
begin
    if N is a leaf then
        return the estimated score of this leaf
    else
        Let N1, N2, .., Nm be the successors of N;
        if N is a Min node then
            return min{MINIMAX(N1), .., MINIMAX(Nm)}
        else
            return max{MINIMAX(N1), .., MINIMAX(Nm)}
end MINIMAX;
```

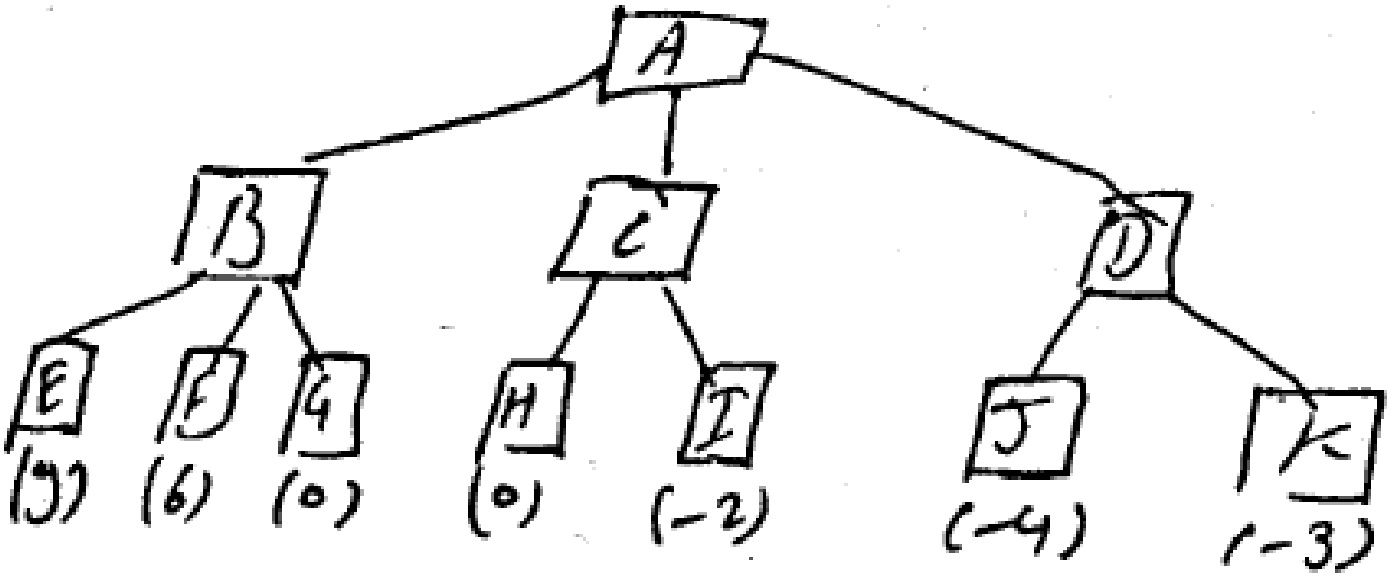
Example 1: Considering the following game tree search space



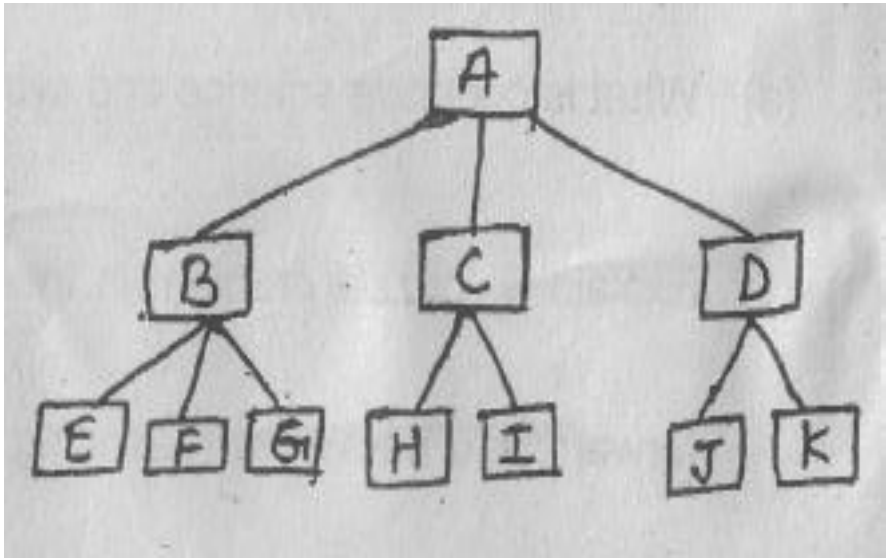
•Which move should be chosen under min-max search procedure, if the first move is a maximizing move?

Example 2: Considering the following game tree search space

– If the first player is a maximizing player, what move should be chosen under the min-max strategy?



Example 3: Considering the following game tree search space



Find out optimal path and value of the following example with the help of min-max algorithm

Alpha-Beta Pruning

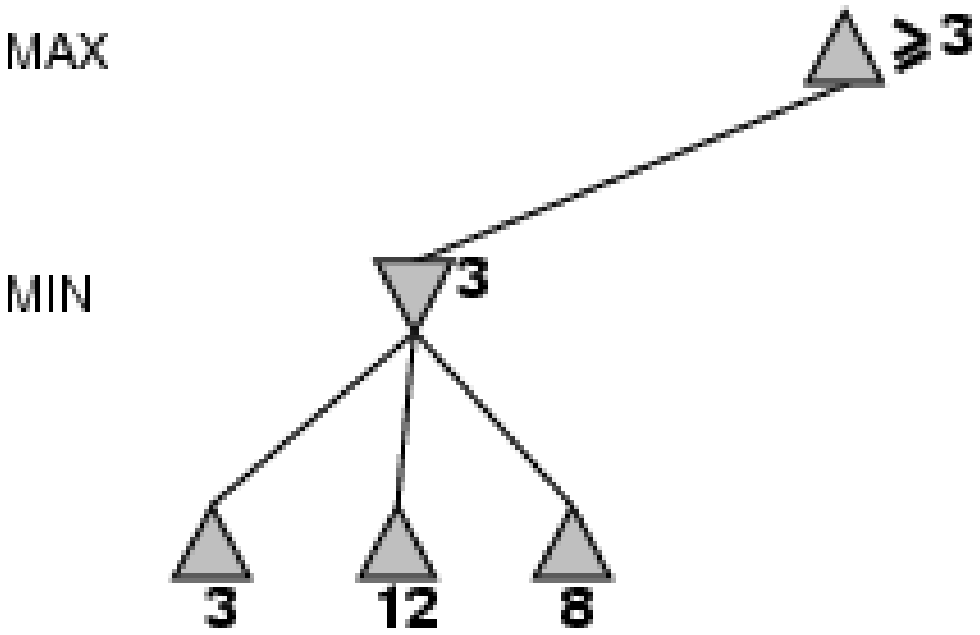
- The problem with Mini-Max algorithm is that the number of game states it has to examine is exponential in the number of moves.
- The Alpha-Beta Pruning helps to arrive at correct Min-Max algorithm decision without looking at every node of the game tree.
- Applying an alpha-cutoff means we stop search of a particular branch because we see that we already have a better opportunity elsewhere.
- Applying a beta cutoff means we stop search of a particular branch because we see that the opponent already has a better opportunity elsewhere.
- Applying both forms is alpha beta pruning.

Alpha Beta Procedure

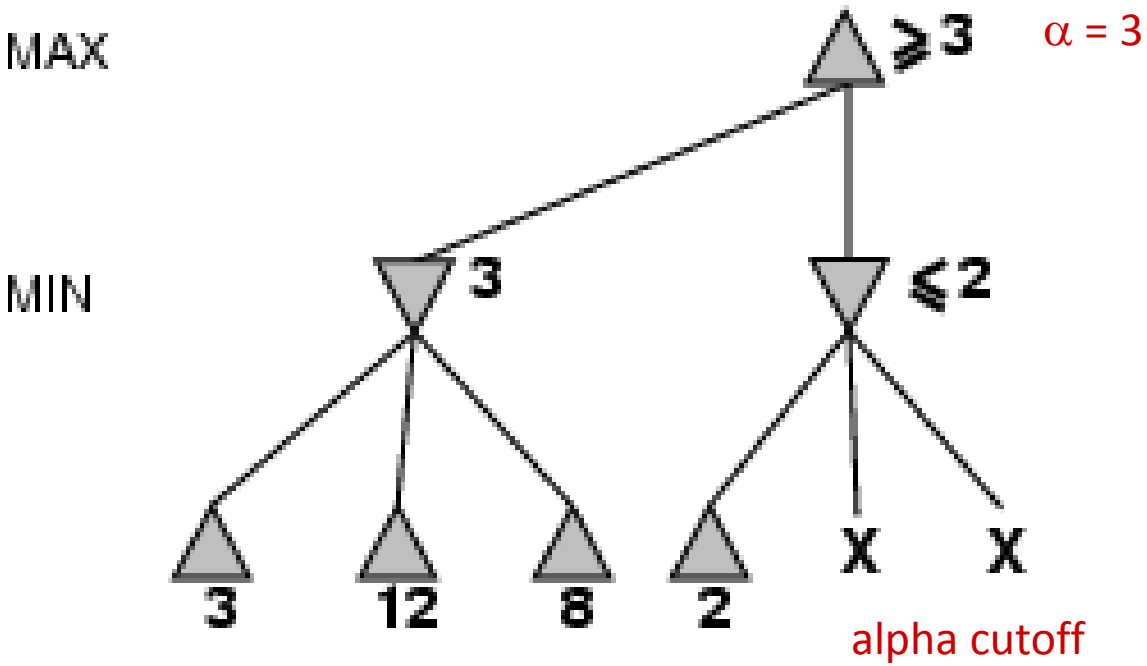
- Depth first search of game tree, keeping track of:
  - Alpha: Highest value seen so far on maximizing level
  - Beta: Lowest value seen so far on minimizing level

- Pruning
  - When Maximizing,
    - do not expand any more sibling nodes once a node has been seen whose evaluation is smaller than Alpha
  - When Minimizing,
    - do not expand any sibling nodes once a node has been seen whose evaluation is greater than Beta

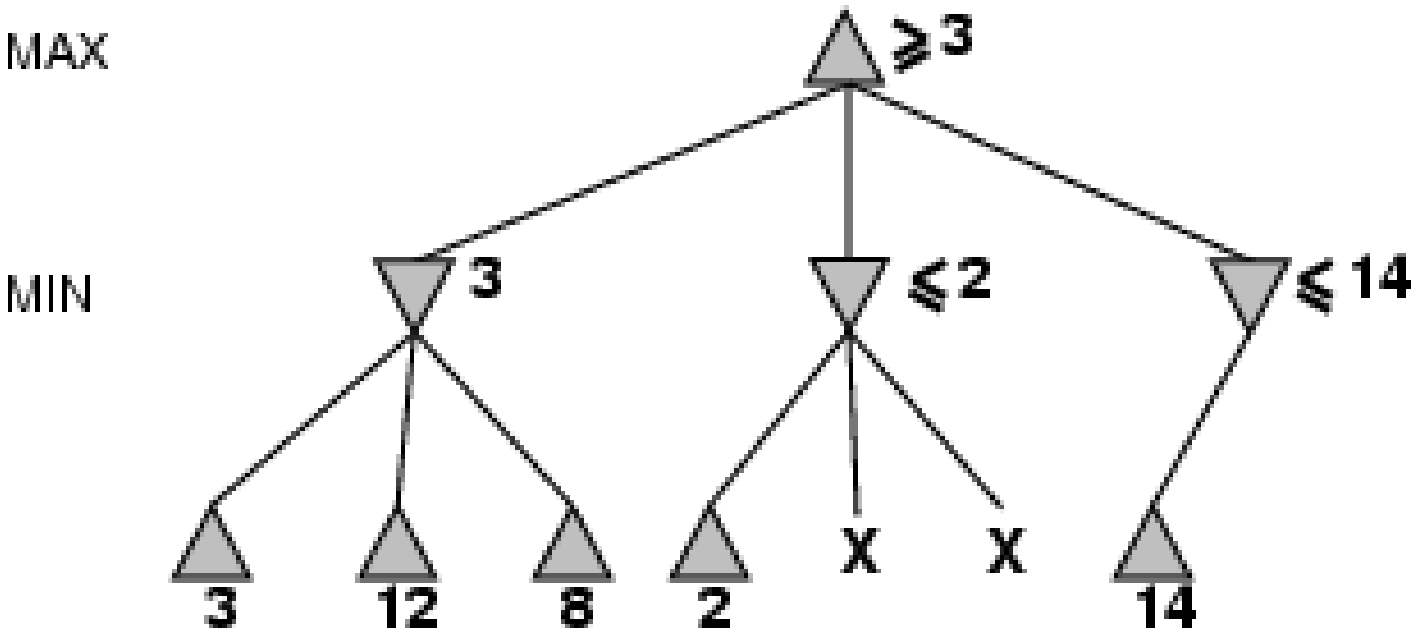
α-β pruning example



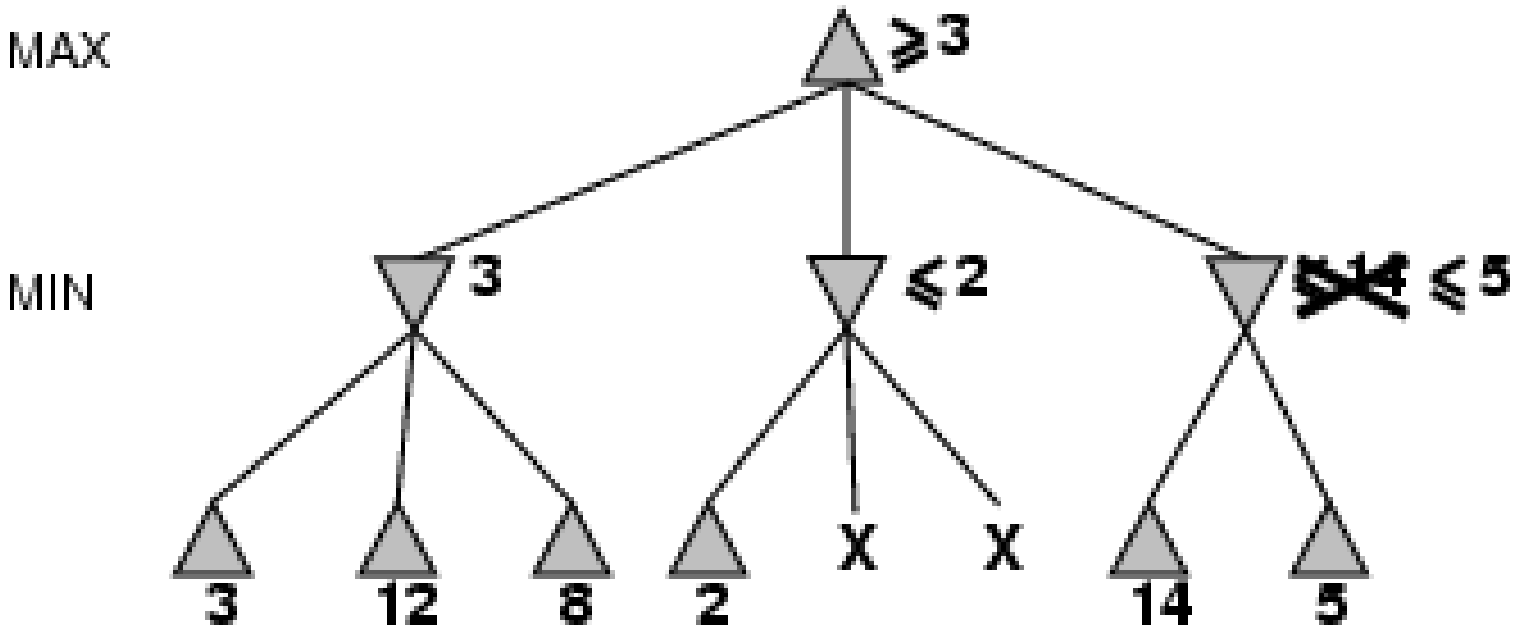
α-β pruning example



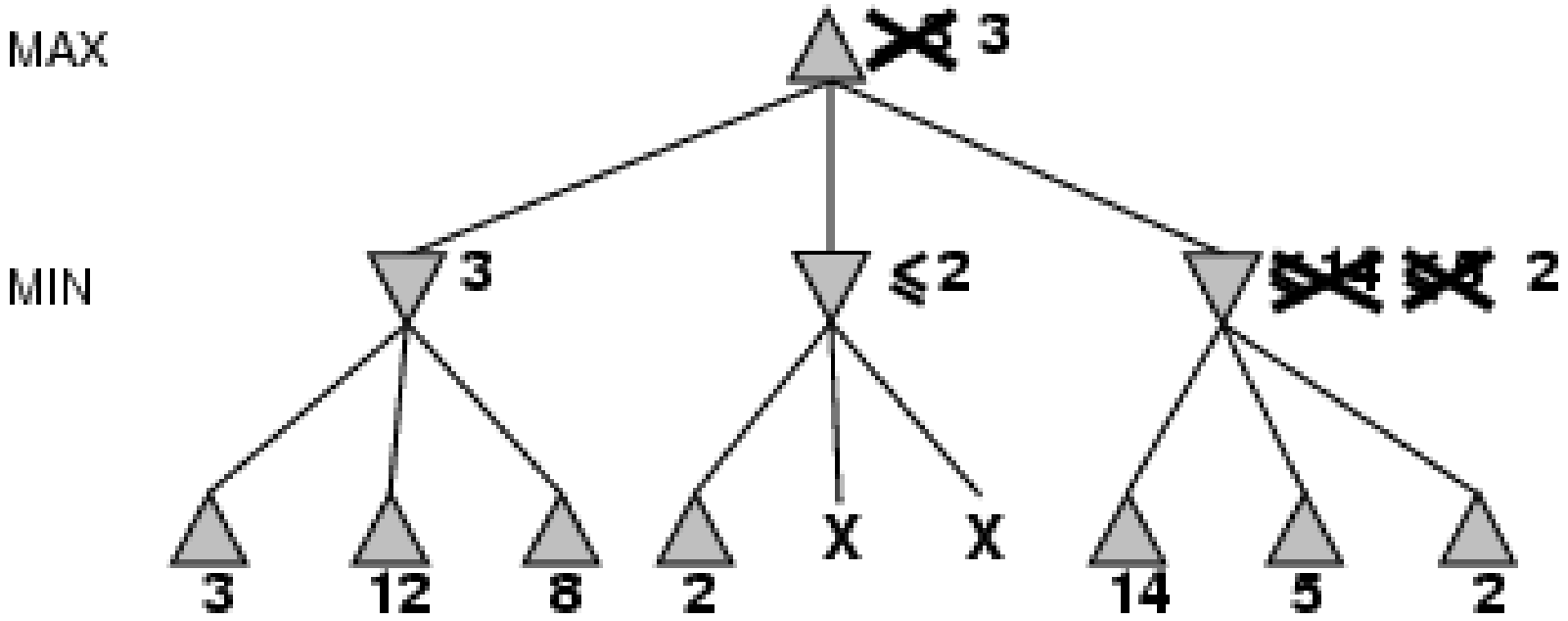
α-β pruning example



α-β pruning example



α-β pruning example

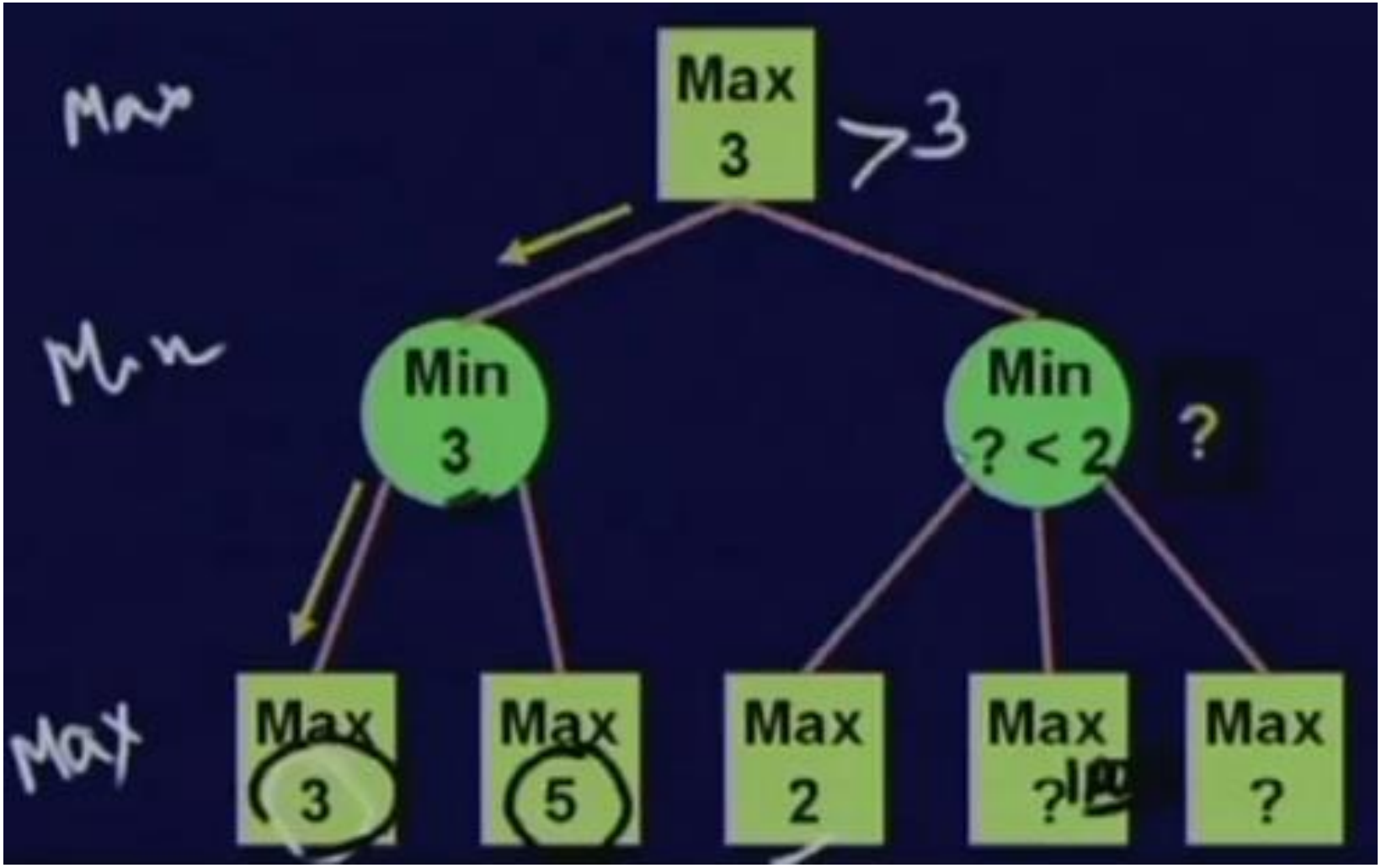


Alpha-beta algorithm

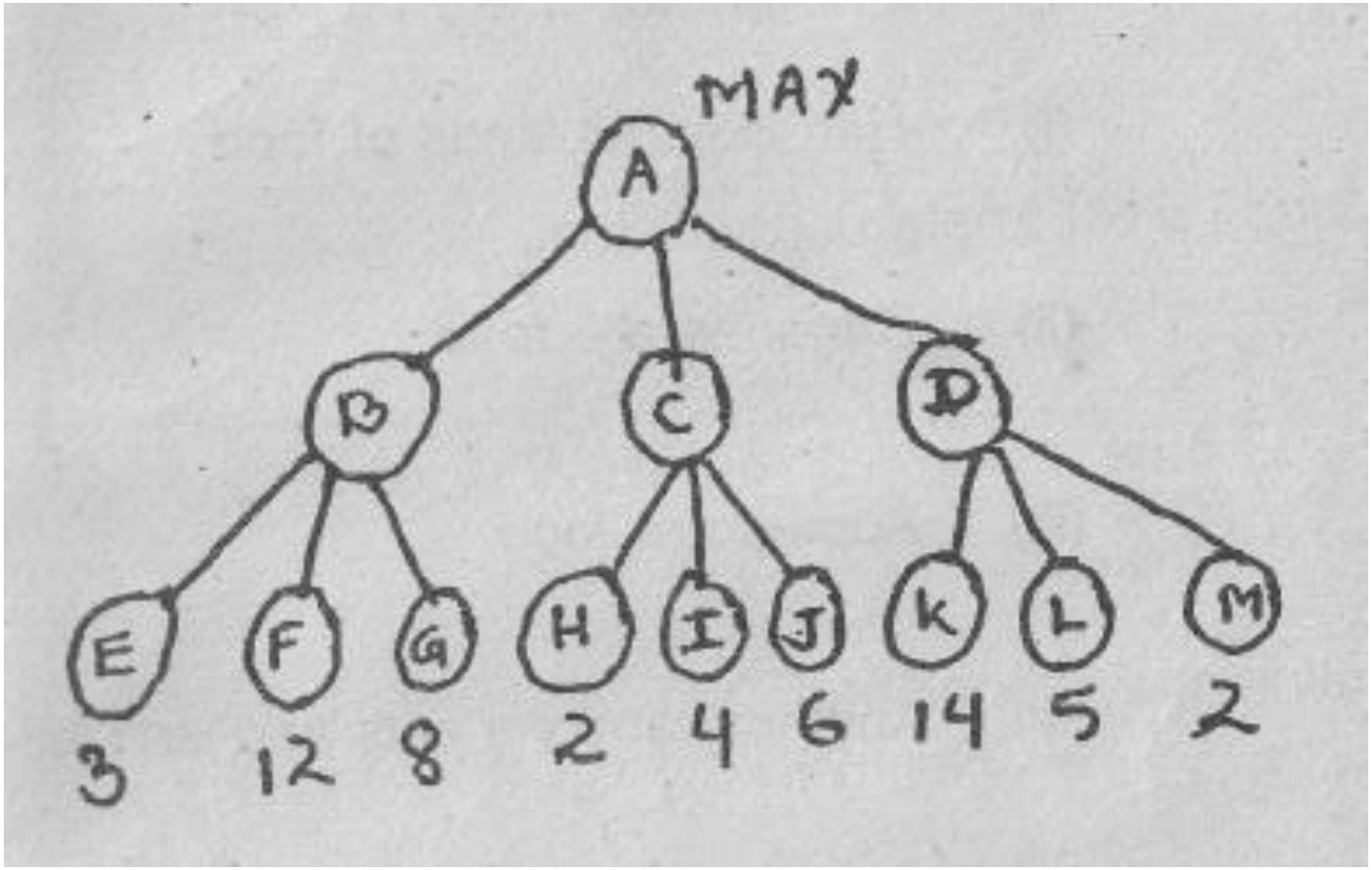
```
function MAX-VALUE (state, game, alpha, beta)
   ;; alpha = best MAX so far; beta = best MIN
if CUTOFF-TEST (state) then return EVAL (state)
for each s in SUCCESSORS (state) do
   alpha := MAX (alpha, MIN-VALUE (state, game,
                alpha, beta))
   if alpha >= beta then return beta
end
return alpha

function MIN-VALUE (state, game, alpha, beta)
if CUTOFF-TEST (state) then return EVAL (state)
for each s in SUCCESSORS (state) do
   beta := MIN (beta, MAX-VALUE (s, game, alpha, beta))
   if beta <= alpha then return alpha
end
return beta
```

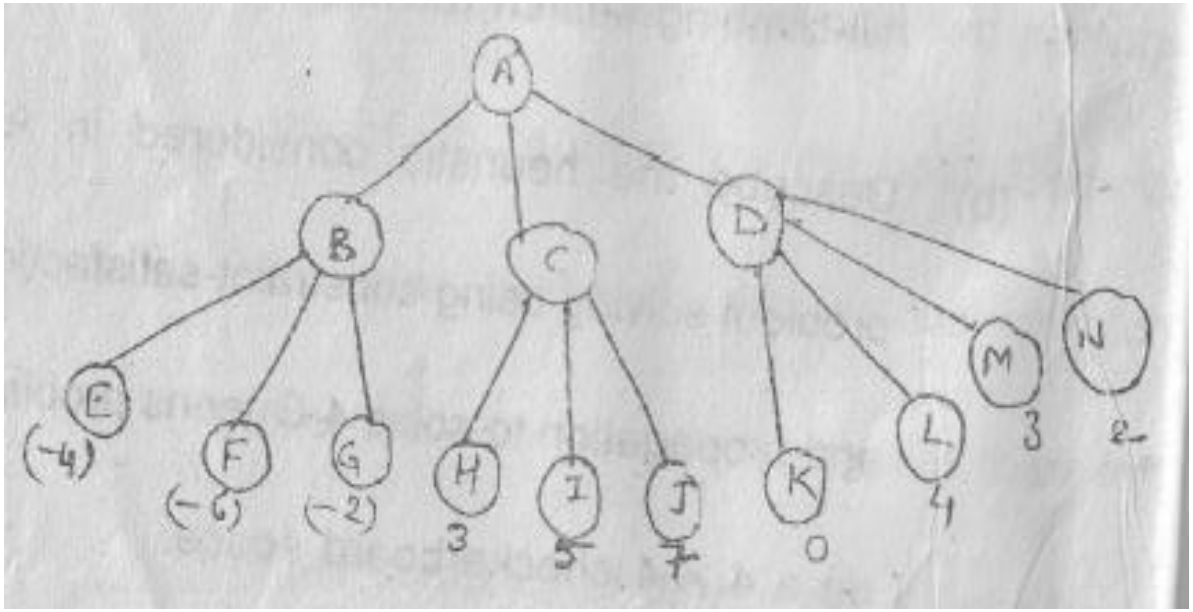
α-β pruning example



Example 1: Prune this tree



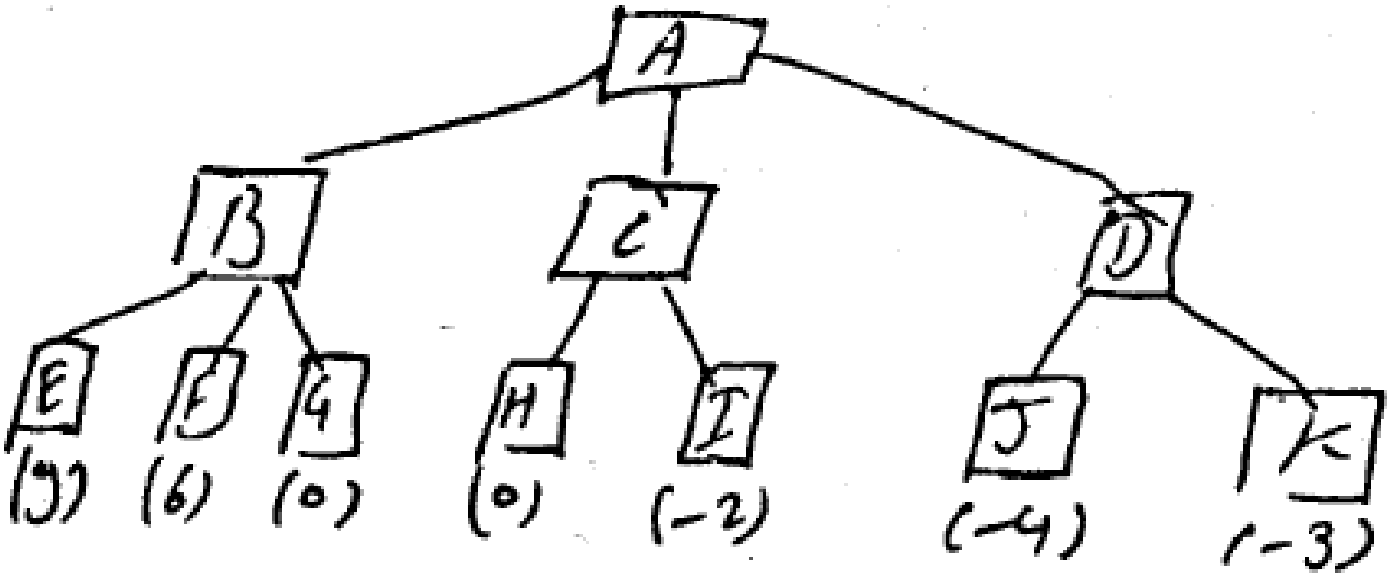
Example 2: Considering the following game tree search space



•What node would not be needed to be examined using alpha-beta pruning technique?

Example 3: Considering the following game tree search space

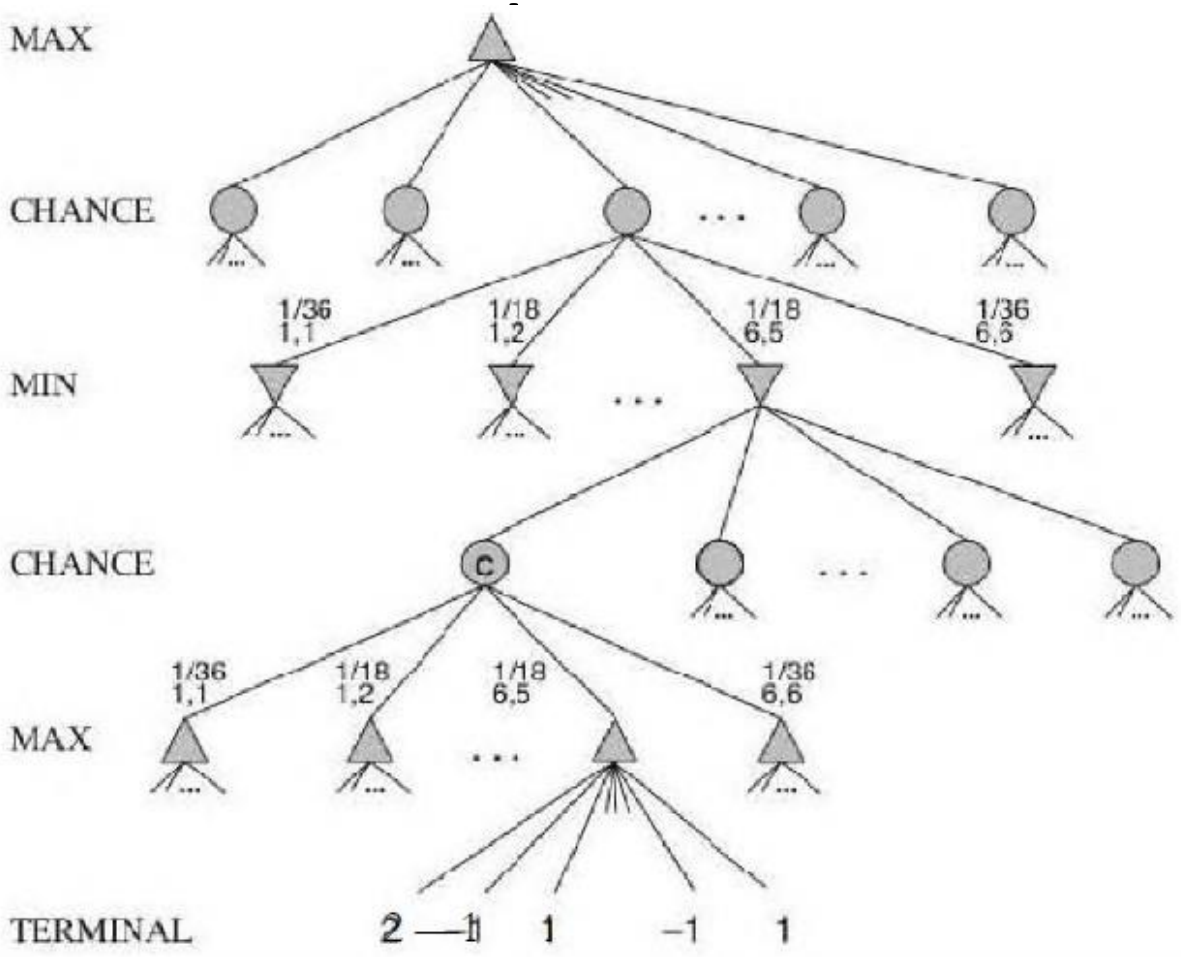
– What nodes should not be needed to be examined using alpha-beta pruning technique?



Games of chance

- In real life, many unpredictable external events can put us into unforeseen situations.
- Many games mirror this unpredictability by including a random element, such as the throwing of STOCHASTIC GAMES dice. We call these stochastic games or games of chance.
- Backgammon is a typical game that combines luck and skill. Dice are rolled at the beginning of a player's rum to determine the legal moves in the backgammon position.
- In game tree in backgammon must include chance nodes in addition to MAX and MIN nodes.

Schematic game tree for a backgammon position



Games of chance

- The next step is to understand how to make correct decisions. Obviously, we still want to pick the move that leads to the best position. However, positions do not have definite minimax values. instead, we can only calculate the expected value of a position: the average over all possible outcomes of the chance nodes.
- This leads us to generalize the minimax value for deterministic games to an expecti_minimax value for games with chance nodes. Terminal nodes and MAX and MIN nodes (for which the dice roll is known) work exactly the same way as before. For chance nodes we compute the expected value, which is the sum of the value over all outcomes, weighted by the probability of each chance action.